UAS E2EEA Developer Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAS E2EEA Developer Guide..... 4
2. Introduction to UAS E2EEA for Developers..... 5
2.1. Application Integrations..... 9
2.1.1. Pre-Authentication..... 11
2.1.2. Password Stored in UAS..... 12
2.1.3. Password Stored in Application..... 17
2.1.4. Client-side Integration..... 21
2.2. Add-on Module: Device-based Password Login Module (Biometric Authentication)..... 24
3. UAS E2EEA Implementation with HSM..... 25
3.1. Implementation Planning..... 26
3.2. UAS E2EEA with SafeNet ProtectServer HSM Implementation..... 32
3.2.1. Deployment..... 33
3.2.2. Configuration..... 41
3.2.3. SafeNet JCProv Failover..... 53
3.3. UAS E2EEA with SafeNet Payment HSM Implementation..... 58
3.3.1. Deployment..... 59
3.3.2. Configuration..... 62
3.4. UAS E2EEA with Thales payShield 10k HSM Implementation..... 70
3.4.1. Deployment..... 71
3.4.2. Configuration..... 72
3.5. UAS E2EEA with Utimaco CryptoServer HSM Implementation..... 82
3.5.1. Deployment..... 83
3.5.2. Configuration..... 94
3.6. UAS Pseudo E2EEA Implementation..... 102
3.6.1. Configuration..... 103
3.7. UAS E2EEA Password Migration..... 113
4. UAS E2EEA Implementation Using API..... 117

4.1. Pre-Authentication..... 118
4.2. Password Stored in UAS..... 121
4.3. Password Stored in Application..... 136
4.4. Add-on Module: Device-based Password Login Module (Biometric Authentication)..... 146
5. UAS E2EEA Client Integration..... 155
6. Appendices..... 174

1. Preface - UAS E2EEA Developer Guide

UAS End-to-End Encryption Authentication (E2EEA) creates a secure channel from the user's endpoint device up to the service provider's Hardware Security Module (HSM). It provides encryption and authentication for the passwords during the user's login and allows developers and administrators to integrate with HSM and the front-end component without any complicated code.

Intended Reader

This guide is intended for experienced application developers and security administrators.

For feedback or questions regarding this document, contact doc@i-sprint.com.

Document Scope

This document covers the following topics:

- UAS E2EEA Concept
- UAS E2EEA Application Integrations
- UAS E2EEA Integration with HSMs
- UAS E2EEA Implementation Using REST API and Client-side Library

2. Introduction to UAS E2EEA for Developers

This document provides the API integration details of AccessMatrix Universal Authentication Server (UAS) End-to-End Encryption Authentication (E2EEA) solution. After reading this document, readers are expected to:

- Understand the concept of UAS E2EEA.
- Understand the use case and workflow of UAS E2EEA.
- Be able to integrate UAS E2EEA with HSMs (or configure pseudo-E2EEA without an HSM).
- Be able to implement UAS E2EEA by integrating related REST APIs and client libraries.

Terminology

This table lists some common terms that are used in this document. It is not intended as a comprehensive definition for each item.

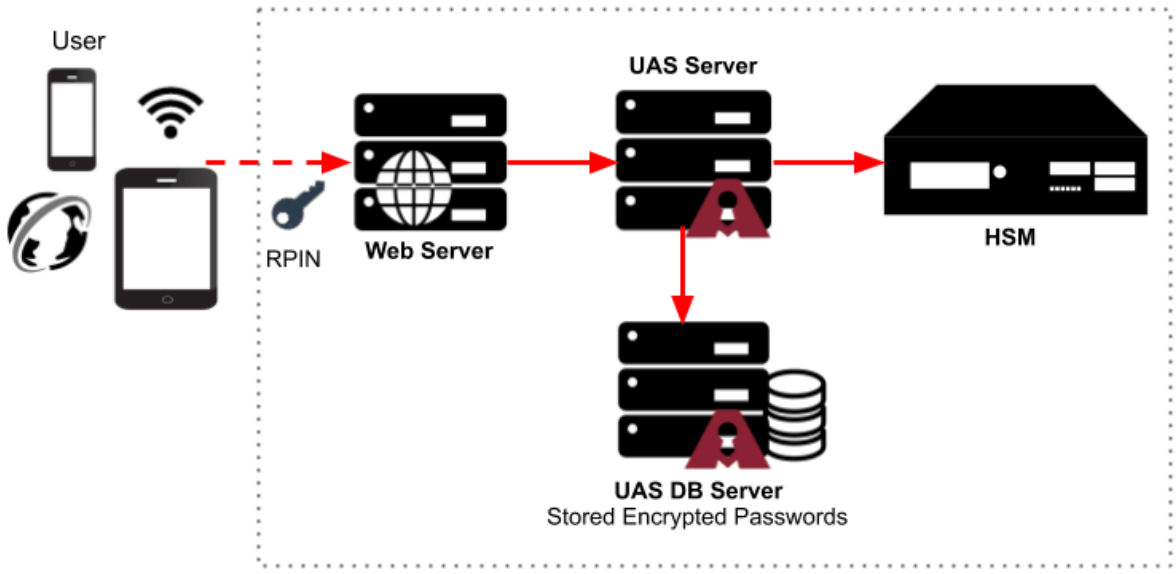
Term	Description
RSA encrypted PIN (RPIN)	The encrypted data is generated from the E2EEA client library to protect the client's confidential data during transit.
TPV (Transformed Pin Value)	Transformed Pin value which is the actual encrypted pin block.
Stored Transformed Pin Value (STPV)	UAS formatted TPV for storage purposes.
For Password Stored in UAS	The user's encrypted password (refers to STPV) will be stored and managed in the UAS Database.
For Password Stored in Application	The user's encrypted password (refers to STPV) will be stored and managed in the User Application Database.
Device-Based Password	The password will be stored and derived from the device secure storage (i.e. biometric verification) and then authenticate to UAS using the Device-based Login Module in the Admin Console.
Salt	In the E2EEA solution, a salt is random data that is used to safeguard the passwords (or STPVs) in storage from risking exposure of clear passwords during authentication.
Local Master Key (LMK)	Local Master Key is used to encrypt operational keys used for encryption, such as Terminal Pin Key (TPK) and Zone Pin Key (ZPK) etc. Local Master Keys are secret, internal to the HSM, and do not exist outside of the HSM.

Term	Description
Terminal Pin Key (TPK)	Terminal Pin Key (TPK) is a data-encrypting key used to encrypt PINs for transmission, within a local network, between a terminal and the terminal data acquirer. For local storage, it is encrypted under one of the LMK pairs.
Zone Pin Key (ZPK)	Zone Pin Key (ZPK) is a data-encrypting key used to encrypt PINs for transfer between communicating parties. For local storage, it is encrypted under one of the LMK pairs.

UAS E2EEA Concept

UAS End-to-End Encryption Authentication Solution (UAS E2EEA) creates a secured channel between the user's access device and Hardware Security Module (HSM). Within this channel, the Password is encrypted at the user's access device. It can only be decrypted for verification by the HSM located in a physically secure location within the organization.

UAS E2EEA is implemented with users residing in the UAS repository, which could be a default user store in UAS or from an external user store, where the users' encrypted passwords (or STPV) will be stored in the UAS database.



However, it is also possible to implement E2EEA so that the encrypted password (or STPV) could be managed and stored by the user application. UAS does not manage the user repository in this case.

Note: The AccessMatrix REST API only supports the **HTTP POST** method.

RPIN Lifecycle

RPIN stands for RSA-encrypted PIN.

The PIN entered by the user at the application front-end is appended with a random number obtained from the server to construct the PIN block. This random number is typically generated by the HSM.

The PIN block is then encrypted with the RSA public key using Optimal Asymmetric Encryption Padding (OAEP) to produce the RPIN. The resulting RPIN is submitted to UAS server for user login. It can only be decrypted by the corresponding RSA private key.

After the RPIN has been decrypted and processed, it is discarded. Thus, the RPIN is only used to securely transport the user's PIN from the application front end to the server side and is never stored permanently.

STPV Storage in UAS

STPV stands for stored transformed PIN value. It refers to the encrypted PIN that is stored permanently in the database for verification purposes.

The STPV is encrypted by a symmetric key that is stored in the HSM. The cryptographic algorithm is typically AES with CBC mode and PKCS#7 padding. After the RPIN has been decrypted, the clear PIN is used to construct the TPV (transformed PIN value). The TPV is then encrypted to form the STPV. The encryption key is stored in the HSM and is only accessible by the HSM.

Typically, the TPV contains the irreversible hash or digest of the PIN instead of the clear PIN itself. The hash algorithm used is usually SHA-256 (optionally, SHA-512 can be selected as well). The hash is irreversible and one-way only. PIN verification is done by computing the hash of the user's input and comparing it with the stored hash in the decrypted STPV.

Password Storage Options

The encrypted passwords (STPVs) can be stored in UAS or the user's application. See the differences below.

Stored in UAS

Stored in UAS option is the implementation has the encrypted user's password stored and managed in the UAS repository. UAS will perform password checking and validation based on the setting of the specified password policies.

Salts are used to safeguarding the passwords (or STPVs) in storage during the hashing operation to produce STPVs. For the implementation of the Password Stored in UAS option, the user salt is provided by UAS and transparent to the application.

> **Note**: Salt is used in hashing operations to produce STPVs during the E2EEA operations. The same salt is used to provide the same user across the operations. Therefore, it must be immutable and unique for each user.

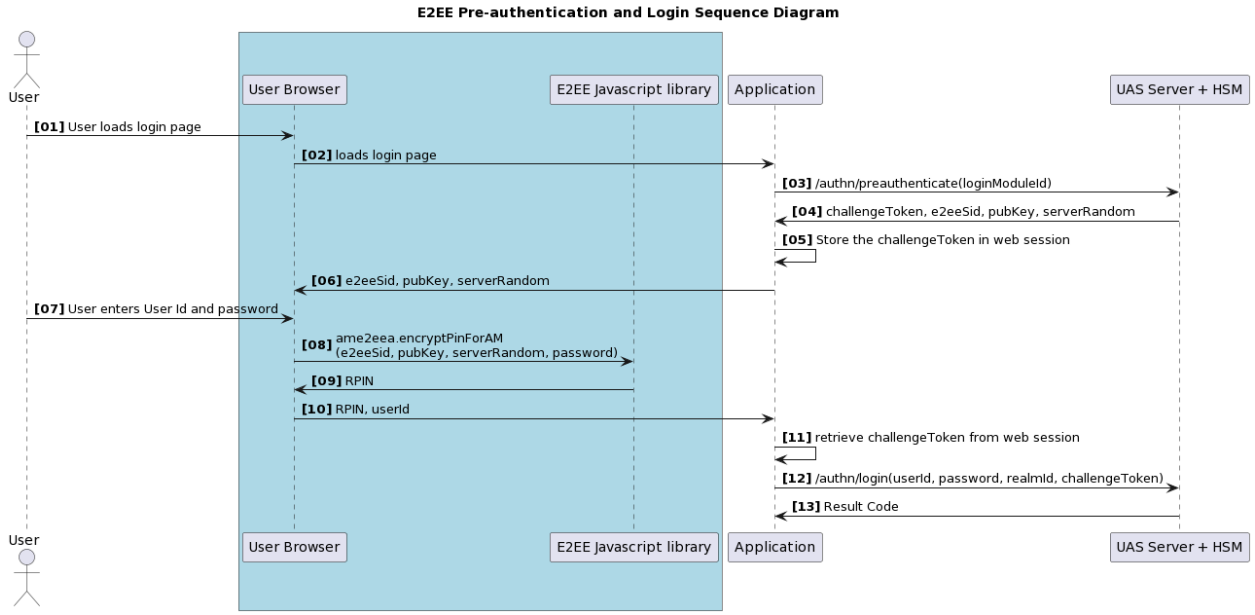
Stored in Application

Stored in Application option is the implementation that allows the application to leverage the E2EEA implementation and having the encrypted user's password stores and manages in the application repository. The user's identity will not be populated in UAS. The enforcement of the password policies like the password expiry, bad logins and others must be enforced by the application.

For the Stored in Application option, the application must provide the User Salt.

2.1. Application Integrations

Refer to the sequential diagram below. The pre-authentication must be done first to prevent a session replay attack. UAS server will generate an E2EEA Session-Id and the Hardware Security Module (HSM) will generate a unique challenge code (a.k.a server-random number) and randomly pick an RSA public key to be used for the E2EEA session.



E2EEA is supported on the following login modules:

Login Module	Description
SafeNet PHEFT Login	SafeNet PHEFT Login is an implementation of login module handler or controller class to manage the E2EEA use cases between a client device and UAS where all authentications and verifications are done in Safenet PHEFT HSM. There are two supported communication modes between UAS and Safenet PHEFT HSM, using a socket connection.
SafeNet ProtectServer E2EEA Login Module	SafeNet ProtectServer E2EEA Login Module is an implementation of a login module handler or controller class to manage the E2EEA use cases between a client device and UAS where all authentications and verifications are done in SafeNet ProtectServer HSM.
E2EE via PKCS#11 Login Module	E2EE via PKCS#11 Login Module is an implementation of login module handler or controller class to manage the E2EEA use cases between a client device and UAS where all authentications and verifications are done in an HSM machine that supports PKCS #11 (or cryptoki) interface.

Login Module	Description
Pseudo E2EEA Login Module	Pseudo E2EEA Login Module is an implementation of login module handler or controller class to manage the E2EEA use cases between a client device and UAS where all authentications and verifications are done in UAS instead of Hardware Security Module (HSM). However, the HSM is optionally used for key management.
Device-based Password Login Module	Device-based Password Login Module is an implementation of login module handler or controller class to manage the E2EEA use cases between a user device and UAS where the user is authenticated by using her on-device biometric that will derive a password, and the password verification is done by using the other UAS login module, e.g. SafeNet ProtectServer E2EEA Login Module.
Thales payShield Login Module	Thales payShield Login Module is an implementation of login module handler or controller class to manage the E2EEA use cases between a client device and UAS where all authentications and verifications are done in Thales payShield HSM.
Utimaco CryptoServer E2EEA Login Module	Utimaco CryptoServer E2EEA Login Module is an implementation of login module handler or controller class to manage the E2EEA use cases between a client device and UAS where all authentications and verifications are done in Utimaco Crypto Server HSM.

2.1.1. Pre-Authentication

Pre-Authentication is a mandatory step to obtain the E2EEA parameters, including the pre-session required to call the UAS E2EEA API.

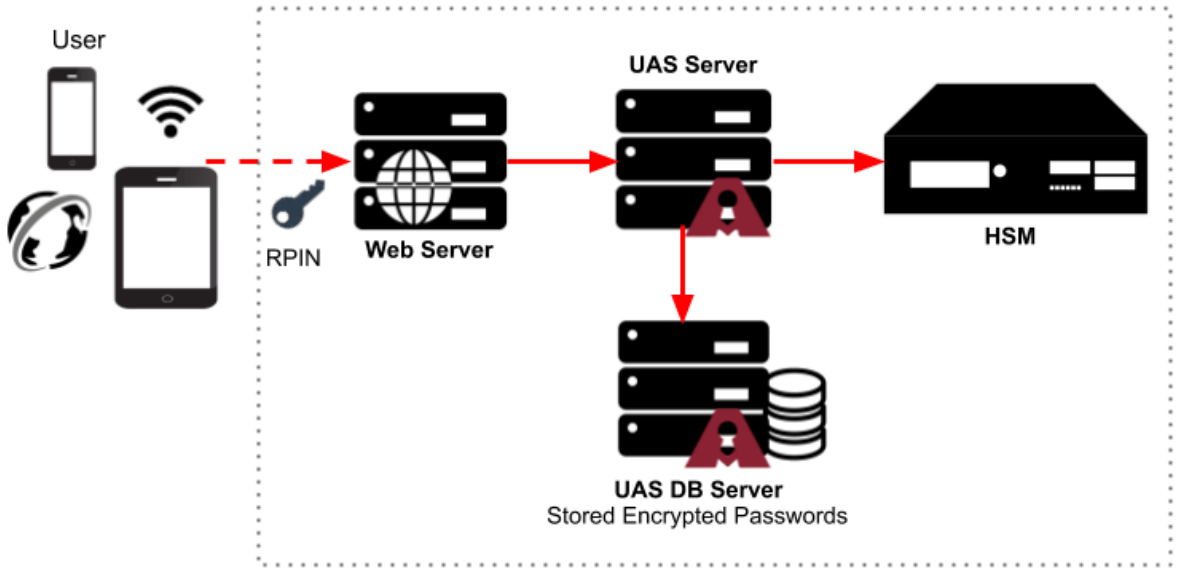
The following E2EE parameters are returned from the Pre-Authentication API call:

E2EE Parameter	Description
e2eeSid	**e2eeSid** is the pre-session literal required for calling UAS E2EEA API.
expiredAt	**expiredAt** is the expiry time of e2eeSid
serverRandom	**serverRandom** is a random literal generated by the UAS or HSM
oaepHashAlgo	**oaepHashAlgo** is a cryptograpy hash algorithm for RSA encryption padding
pubKey	**pubKey** is a RSA public key value
pubKeyIndex	**pubKeyIndex** is an index of the RSA public key
loginModuleUUID	**loginModuleUUID** is a unique ID of the UAS login module
passwordQualityPolicy	**passwordQualityPolicy** is a base64 encoded password quality policy configuration.

Note: The E2EE parameter **passwordQualityPolicy** is returned from the Pre-Authentication API only if the HSM does not have the capability to enforce password quality; thus, the password quality checking must be done at the client library. For example, this parameter will be returned with the Thales payShield HSM.

2.1.2. Password Stored in UAS

This implementation allows the user's applications to leverage the E2EEA solution without managing users' identities and passwords. Instead, user management and security are handled by the UAS Server as well as the password policy.



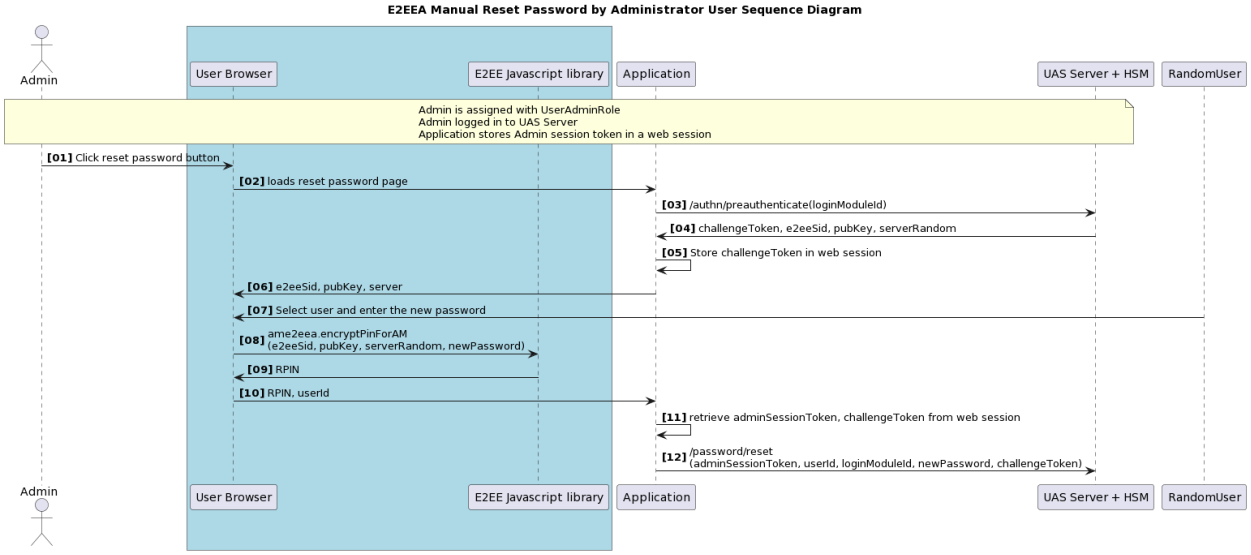
Reset

An administrator user assigned with UserAdminRole can reset the password of another user with the UAS reset password API; refer to UAS Administration Guide.

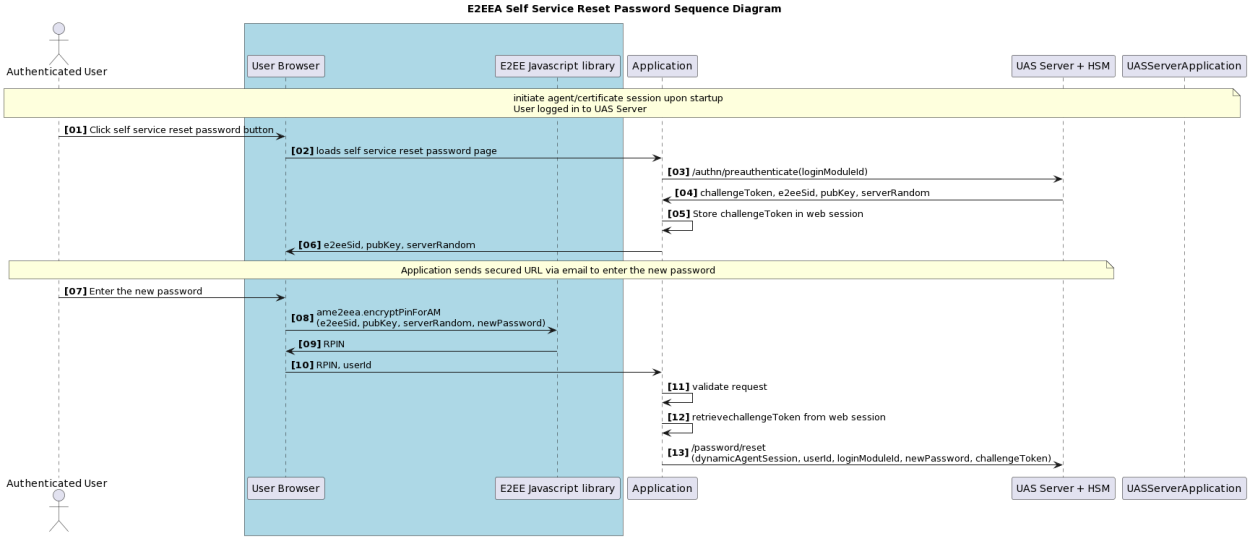
Reset password is useful for the use cases like self-service reset password or manual reset password by the administrator when the user forgot their password or enabled the device registration.

To give you some examples, below are the common usage of the E2EEA reset password.

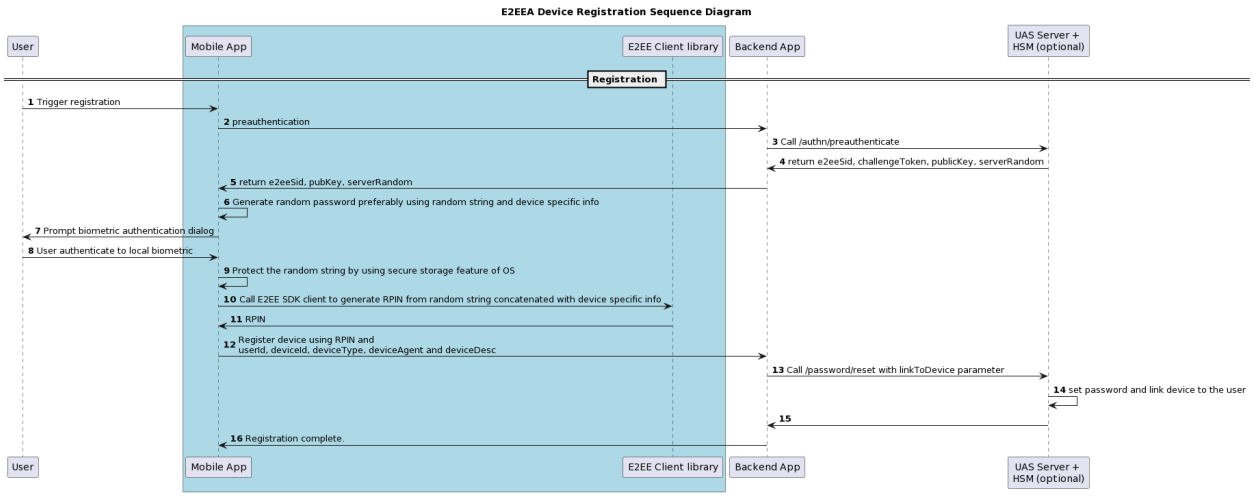
Usage Example: Manual E2EEA Reset Password by Administrator User



Usage Example: Self Service E2EEA Reset Password



Usage Example: Device Registration with E2EEA



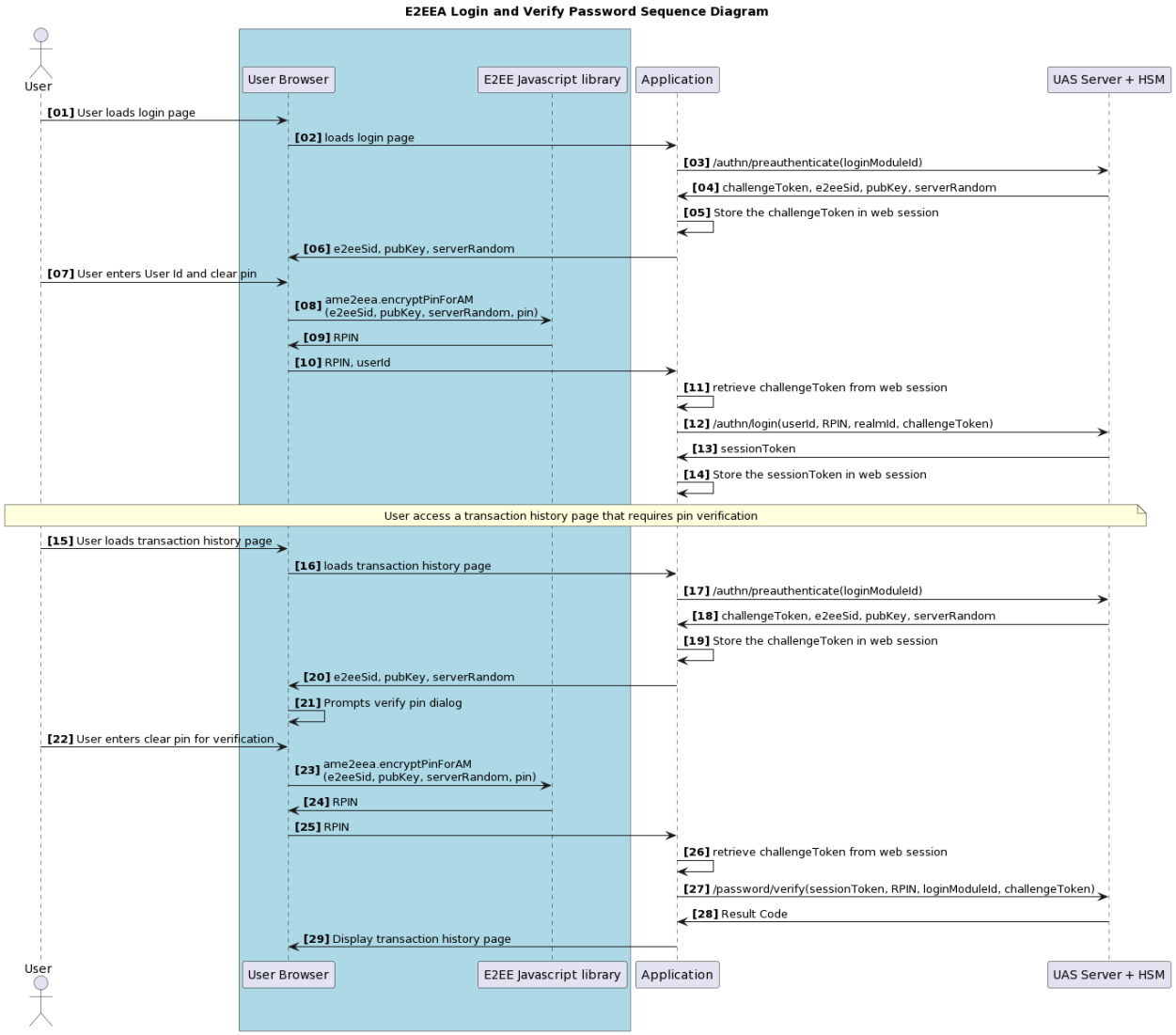
In addition to the above use cases, UAS supports **System Assigned** password generation options where the password is not provided by the user but generated in HSM. The generated password will be sent through a secured channel to the user. **System Assigned** password generation options will be explained further in Reset Password.

Authentication

Authentication is the process of verifying whether someone is who it is declared to be. UAS provide E2EEA API to do the authentication and verify the user password.

The example below shows the usage of E2EEA login and verify the password.

Usage Example: E2EEA Login and Verify PIN When User Access i-Banking Transaction History

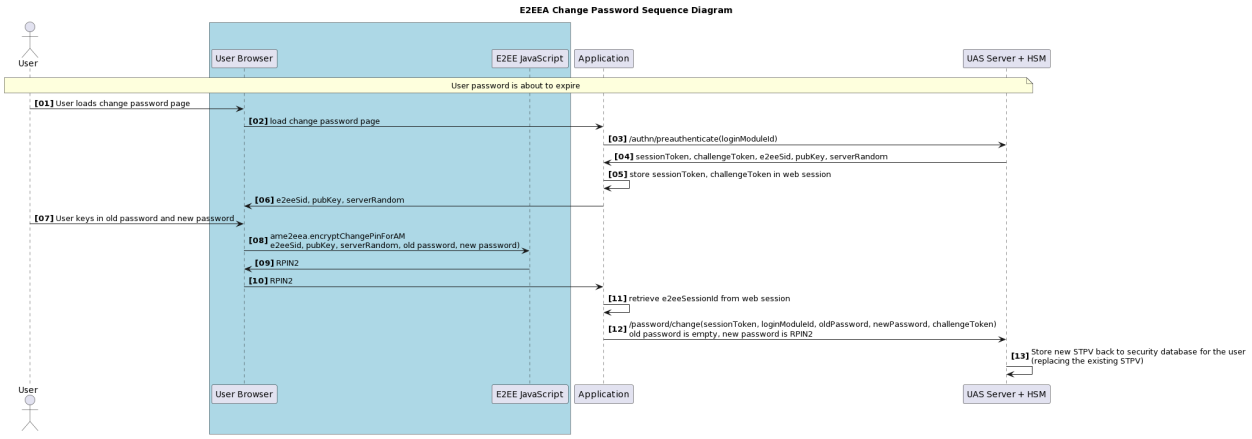


Change

For security reasons, the UAS user is advised to change the password periodically, or the application may enforce to change the password when the user password is about to expire.

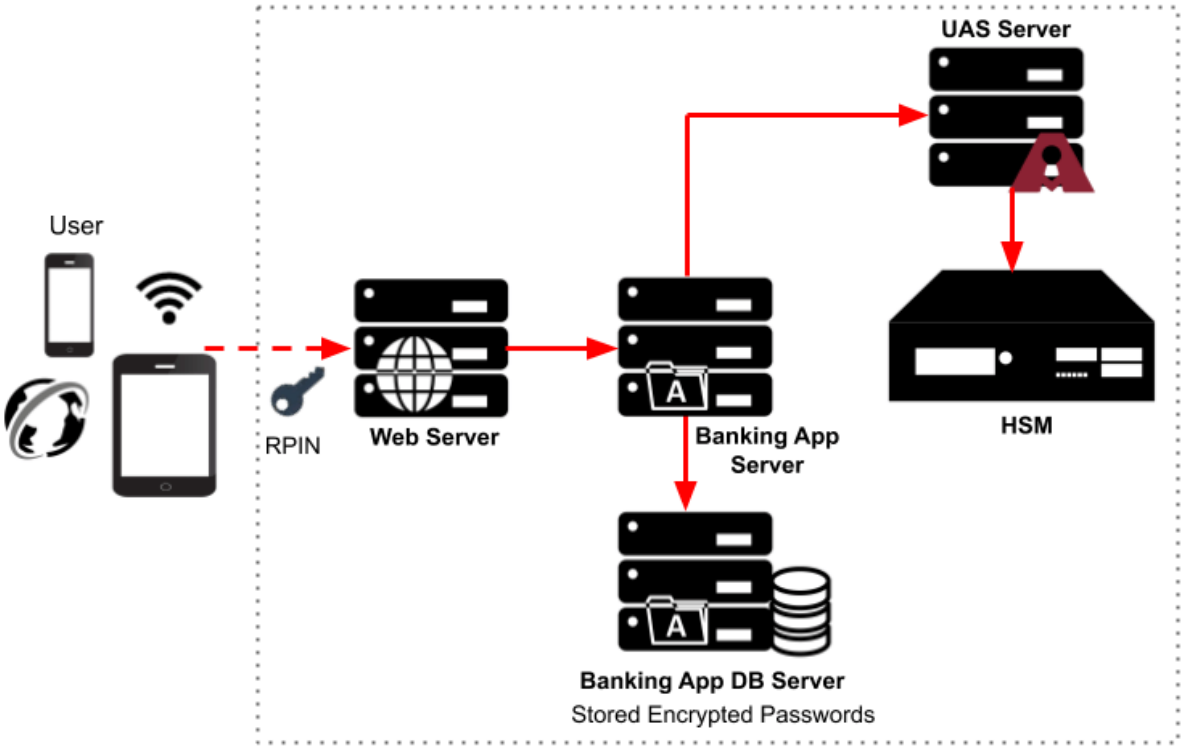
The example below shows a usage of E2EEA change password when the user password is about to expire.

Usage Example: E2EEA change password when the user password is about to expire



2.1.3. Password Stored in Application

This implementation allows users' applications to leverage the E2EEA solution while managing and storing the encrypted user password in the application repository. The user Id information will not be populated in UAS. However, the user's application needs to enforce the password policies like password expiry, bad login, and others.

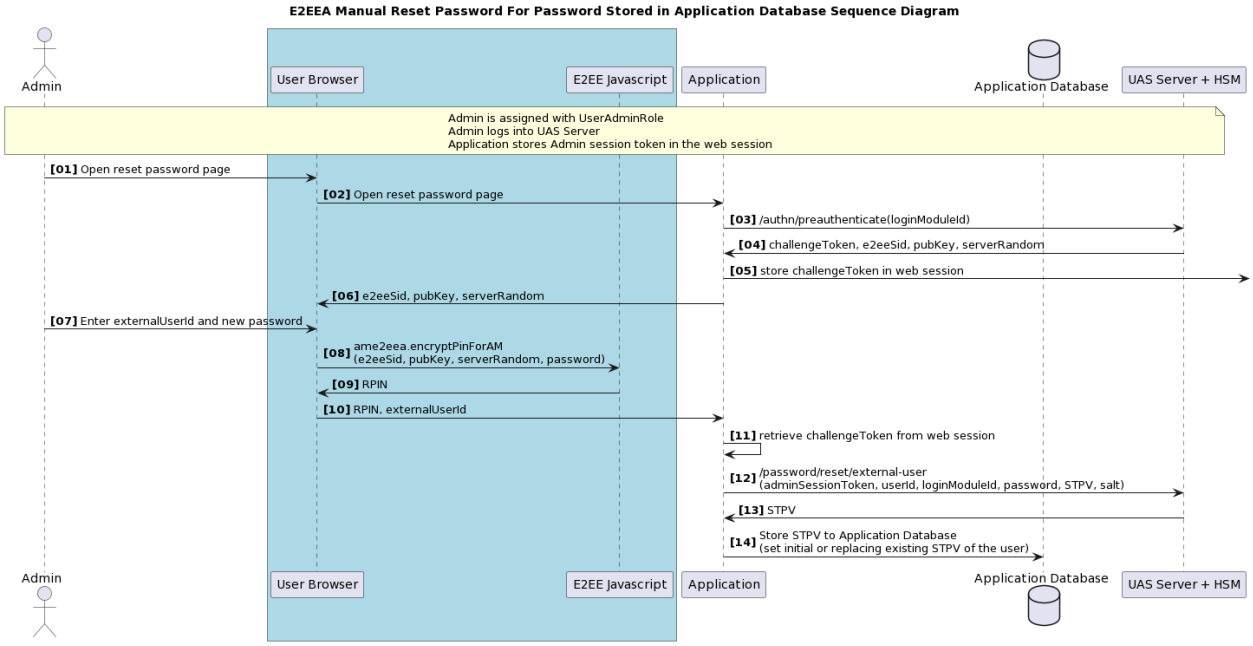


Reset

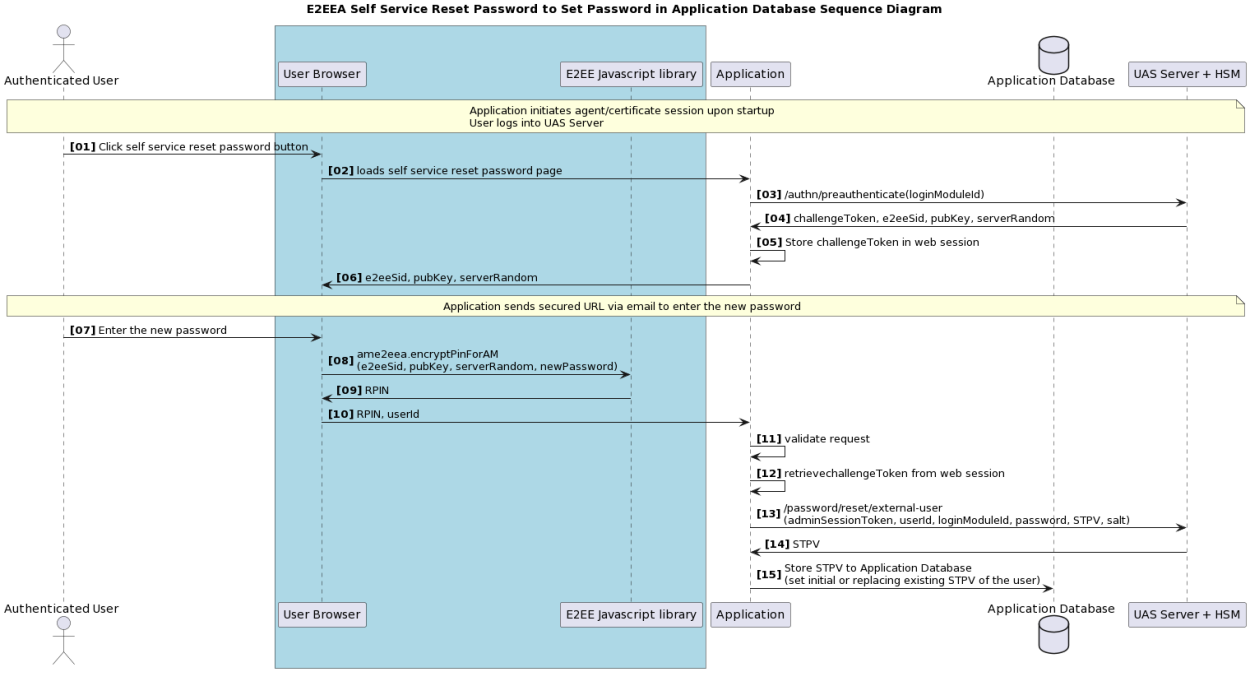
The backend application may choose to store STPV in its own storage. The user's application can leverage the UAS E2EEA reset external password API to generate the STPV and store it. The UAS E2EEA reset external password API needs a cryptography salt to safeguard password hash in storage and protect from the pre-computed hash attack such as rainbow tables.

The salt value must be unique for every user and long enough without a colon ":" character. The example below shows the usage of E2EEA reset external password API to set an initial password in the application database.

Usage Example: E2EEA Manual Reset Password to Set Initial Password in Application Database



Usage Example: E2EEA Self Service Reset Password to Set Password in Application Database

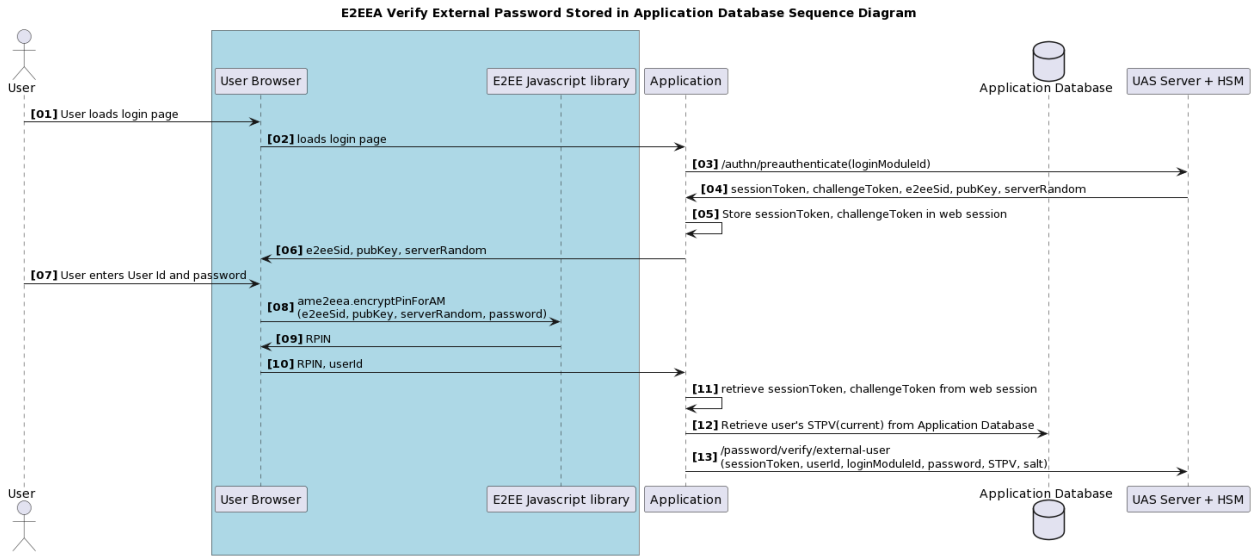


Authentication

The backend user application can verify the STPV value in the application's protected storage with UAS E2EEA verify external password API. The user's application only needs to provide the current STPV and its salt value.

The example below shows usage of E2EEA verifies external password API to verify the STPV stored in the application database.

Usage Example: E2EEA Verify External Password Stored in the Application Database

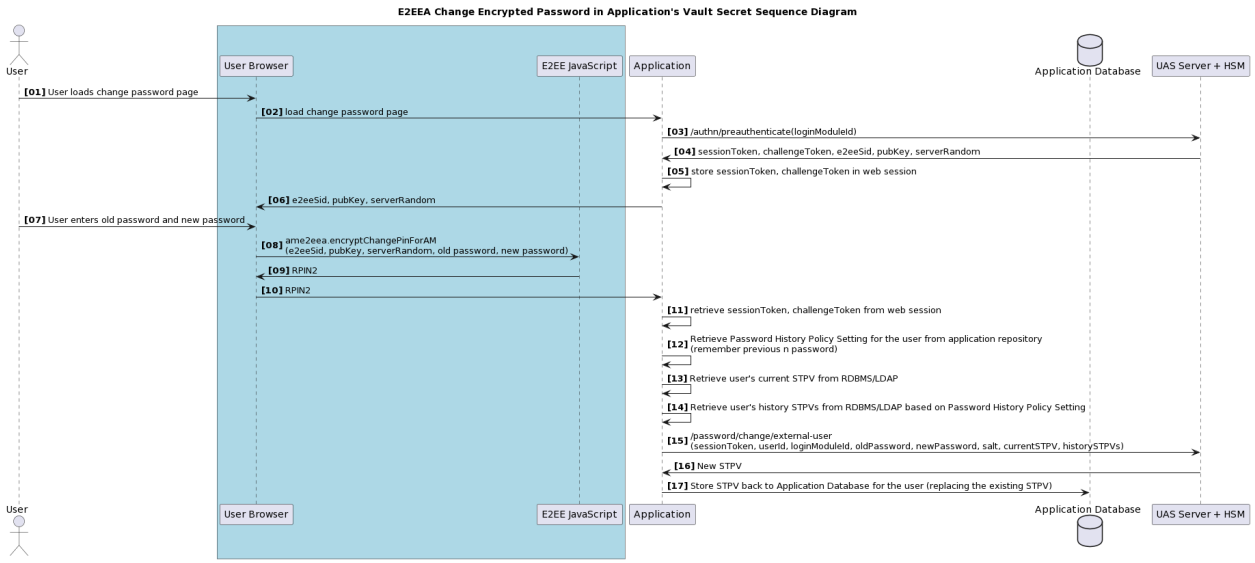


Change

The backend user application can change the STPV value in the application's protected storage with a new value using UAS E2EEA change external password API. The user's application only needs to provide the current STPV together with its salt value. If the application stored the historical STPV, the application may provide historical STPV to UAS E2EEA change the external password API to get the benefit of password history checking in UAS.

The example below shows the usage of E2EEA changes external password API to change the current STPV stored in the application database.

Usage Example: E2EEA Change Encrypted Password in Application Database



2.1.4. Client-side Integration

This is the integration between the E2EEA client library using web javascript, java or iOS, and the client-side's application (web browser, Android or iPhone application). The user information is encrypted on a user device using RPIN (RSA-encrypted PIN) to protect a user's clear password.

Web / Javascript

UAS E2EEA product shipped with the sample E2EEA web project, including the standalone javascript library **ame2eea.js**. The sample project is packaged as a web application archive (war), and the filename is **AccessMatrix-VERSION-e2eejs-sample.war**.

Follow the steps below to deploy the sample E2EEA in your web project.

1. In this example, we assume UAS Server is already deployed, and the SafeNetPHEFTE2EE login module is configured.
2. Edit the configuration file **e2ee-service.properties** in the *WEB-INF/classes* folder of your sample E2EEA web project.
3. Specify the following:
    a. Realm ID in E2EEREALMID
    b. Login Module ID in E2EELoginModule_ID
    c. UAS Server hostname/IP in HOSTS
    d. (Optional) The remaining properties, depending on your UAS Server deployment (see the example below).
4. Deploy the sample E2EEA web project in your preferred web application server.
5. Try E2EEA reset password from the browser by accessing **resetPassword.jsp**, for example, *http://mye2eesample/e2ee/resetPassword.jsp*.

Below is a sample configuration in **e2ee-service.properties:**

Properties (.properties files)

```
#---- for E2EE Sample Pages -------
E2EEREALMID=SafeNetE2EE
E2EELoginModule_ID=SafeNetPHEFTE2EE
E2EEOAEPHASH_ALGO=SHA-256
```

```
# ---- Uncomment these if PIN Block Format are
different for login, change and reset password
# E2EEPINBLOCKFORMATRESET_PWD=ISO-0
# E2EEPINBLOCKFORMATCHANGE_PWD=NONE-SHORT
# E2EEPINBLOCKFORMATLOGIN=NONE-SHORT
# ---- Config to set up HTTPS REST ------- #
HOSTS=https://testserver:8443/am5/rest
TRUST_STORE=C:/Tomcat 6.0/webapps/e2eeJS-sample/WEB-
INF/classes/testtrust.keystore
AGENT_KEYSTORE=C:/Tomcat 6.0/webapps/e2eeJS-sample/WEB-
INF/classes/testagent.keystore
TRUSTSTORE_PWD=passphrase
KEYSTORE_PWD=passphrase
ConnTimeoutSecs=30
ADMINLOGINID=usoagent
ADMIN_PASSWORD=
ADMINLOGINREALMID=40153
```

Note: For the Thales payShield Login Module, there are 2 PIN block formats involved:

- Reset Password uses the ISO-0 format
- Login and Change Password use the NONE-SHORT format

Thus, the E2EEPINBLOCKFORMATRESET_PWD, E2EEPINBLOCKFORMATCHANGE_PWD, and E2EEPINBLOCKFORMATLOGIN properties must be configured accordingly.

Java

UAS E2EEA shipped with the Java client library packaged with java archive (jar) to perform RSA encryption and generate RPIN on the java client side. The jar filename is **e2eeAndroid.jar**.

The following example shows how to import **e2eeAndroid.jar** in Android Studio for Android Application development.

1. Get the **e2eeAndroid.jar** file from the AccessMatrix CD installer, or you may copy it from the *WEB-INF/lib* folder in your UAS Server.
2. Put the **e2eeAndroid.jar** file somewhere in your system.
3. Copy the **e2eeAndroid.jar** file and Paste it into the libs folder under the app folder of your project.

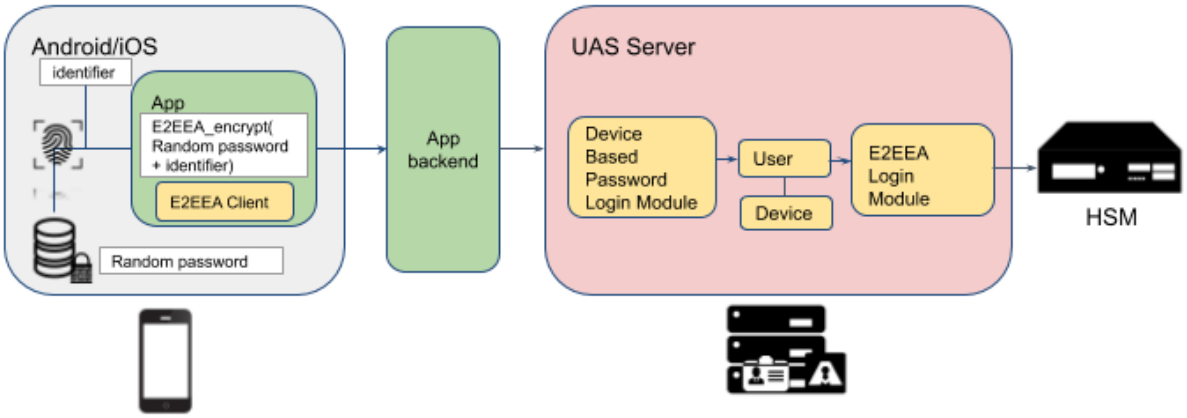
   Note: If you are unable to find libs in your Android Studio, then you might be viewing your project in the “Android” view. On the left side of your screen, there is a dropdown menu. Click on it and select "Project".

4. Navigate to **File > Project Structure > Dependencies**.
5. Click the **+** (plus) button displayed on the right side and select "Jar Dependency".
6. Provide the path to the library, for example, *libs\e2eeAndroid.jar*.
7. Click **OK** to save the changes.

2.2. Add-on Module: Device-based Password Login Module (Biometric Authentication)

Local (on device) biometric verification is now commonly provided by phone vendors. FIDO is commonly accepted as the proper solution; however, it comes with complexity. UAS aims to provide a less complex alternative yet still provide a seamless user experience using the phone's fingerprint/face authentication.

The device enrollment or registration is done by generating a random password inside the phone and setting it to UAS by calling the E2EEA password reset API and passing in the device Id. The generated random password is stored in a protected area on the mobile device. For example, for iOS, the random password will be stored in the keychain, while for Android, the application will generate a key secured in the Android Keystore and used by the application to encrypt the password.



Refer to *AccessMatrix Common Administration Guide - Device-based Password Login Module* for more information on the login module.

3. UAS E2EEA Implementation with HSM

The AccessMatrix UAS End-to-End Encryption Authentication (AccessMatrix UAS E2EEA) solution enables application developers to easily integrate E2EE authentication with an HSM and front end components for password encryption during user logins. This is done through the loading of a provided UAS E2EEA Functionality Module (FM) into the HSM. Once integrated, the following operations can generally be performed within the HSM for added encryption/decryption:

- Hashing
- Protection of Master Encryption Key
- Random Number Generation
- Initial PIN Generation
- PIN Verification

Alternatively, AccessMatrix also provides a non-HSM Pseudo-End-to-End Encryption Authentication (E2EEA) solution. This solution implements an end-to-end application layer encryption to protect passwords and other sensitive data between a client device and AccessMatrix UAS, wherein all authentication and verification is done within AccessMatrix UAS instead of an HSM. An HSM can still be optionally used for key management.

The following pages detail the steps required for the integration of UAS E2EEA with their respective HSMs:

- SafeNet ProtectServer HSM
- SafeNet Payment HSM
- Thales payShield 10k HSM
- Utimaco CryptoServer HSM
- UAS Pseudo-E2EEA (non-HSM alternative)

3.1. Implementation Planning

Implementation Checklist

Below are the checklists for implementing the UAS E2EEA solution with various HSMs:

	Tasks	Reference
SafeNet ProtectServer HSM	Install Oracle JDK/JRE	Install Oracle JDK/JRE
	Install SafeNet ProtectServer HSM Drivers	Install SafeNet ProtectServer HSM Drivers
	Initialize HSM Adapter	Initialize Adapter
	Initialize Token Slot	Initialize Token
	Deploy E2EEA FM to SafeNet ProtectServer HSM	Deploy E2EEA FM to SafeNet ProtectServer HSM
	Create E2EEA Keys	Create E2EEA Keys
	Secure AccessMatrix Server License	Upload E2EEA License
	Configure E2EEA Login Policy	Configure E2EEA Login Policy
	Configure E2EEA Login Module	Configure E2EEA Login Module
	Create Authentication Realm	Create Authentication Realm
	Assign Authentication Realm to User	Assign Authentication Realm to User
SafeNet Payment HSM	**Tasks**	**Reference**
	Install Oracle JDK/JRE	Install Oracle JDK/JRE
	Install SafeNet Payment HSM Drivers	Install SafeNet Payment HSM Client Drivers
	Secure AccessMatrix Server License	Upload E2EEA License
	Configure SafeNet PHEFT Login Module	Configure SafeNet PHEFT Login Module
	Create Authentication Realm	Create Authentication Realm
	Assign Authentication Realm to User	Assign Authentication Realm to User

	Tasks	Reference
SafeNet ProtectServer HSM	Install Oracle JDK/JRE	Install Oracle JDK/JRE
	Install SafeNet ProtectServer HSM Drivers	Install SafeNet ProtectServer HSM Drivers
	Initialize HSM Adapter	Initialize Adapter
	Initialize Token Slot	Initialize Token
	Deploy E2EEA FM to SafeNet ProtectServer HSM	Deploy E2EEA FM to SafeNet ProtectServer HSM
	Create E2EEA Keys	Create E2EEA Keys
	Secure AccessMatrix Server License	Upload E2EEA License
	Configure E2EEA Login Policy	Configure E2EEA Login Policy
	Configure E2EEA Login Module	Configure E2EEA Login Module
	Create Authentication Realm	Create Authentication Realm
	Assign Authentication Realm to User	Assign Authentication Realm to User
Thales payShield 10k HSM	**Tasks**	**Reference**
	Install Oracle JDK/JRE	Install Oracle JDK/JRE
	Secure AccessMatrix Server License	Upload E2EEA License **Note:** Thales payShield 10k HSM itself requires LIC014 in order to support E2EEA.
	Configure Thales payShield Login Module	Configure Thales payShield Login Module
	Create Authentication Realm	Create Authentication Realm
	Assign Authentication Realm to User	Assign Authentication Realm to User
Utimaco CryptoServer HSM	**Tasks**	**Reference**
	Install Oracle JDK/JRE	Install Oracle JDK/JRE
	Install CryptoServer Host Software	Install CryptoServer Host Software
	Set Up Utimaco CryptoServer LAN Device	Set Up Utimaco CryptoServer LAN Device

	Tasks	Reference
SafeNet ProtectServer HSM	Install Oracle JDK/JRE	Install Oracle JDK/JRE
	Install SafeNet ProtectServer HSM Drivers	Install SafeNet ProtectServer HSM Drivers
	Initialize HSM Adapter	Initialize Adapter
	Initialize Token Slot	Initialize Token
	Deploy E2EEA FM to SafeNet ProtectServer HSM	Deploy E2EEA FM to SafeNet ProtectServer HSM
	Create E2EEA Keys	Create E2EEA Keys
	Secure AccessMatrix Server License	Upload E2EEA License
	Configure E2EEA Login Policy	Configure E2EEA Login Policy
	Configure E2EEA Login Module	Configure E2EEA Login Module
	Create Authentication Realm	Create Authentication Realm

	Tasks	Reference
UAS Pseudo E2EEA	Secure AccessMatrix Server License	Upload E2EEA License
	Configure Login Policy	Configure Pseudo E2EEA Login Policy
	Configure Password Expiry Policy	Configure Pseudo E2EEA Password Expiry Policy
	Configure Password Quality Policy	Configure Pseudo E2EEA Password Quality Policy
	Create Key Provider	Configure Key Provider
	Create Encryption Key	Configure Encryption Key
	Configure Pseudo E2EEA Login Module	Configure Pseudo E2EEA Login Module
	Create Authentication Realm	Create Authentication Realm
	Assign Authentication Realm to User	Assign Authentication Realm to User

Implementation Planning

This section introduces the key considerations to be assessed before implementing UAS E2EEA with an HSM.

Planning prior to environment setup is critical for the successful implementation of UAS E2EEA. Understanding the customer's nature of business, studying the customer's requirements and its existing environments will help to identify the actions to make prior to actual implementation.

Security Considerations

- **Encryption Keys**

  Recommended key lengths:

  - Asymmetric key pairs: must be at least 2048-bit for key length for RSA (note that the longer the key, the slower the performance is).
  - Storage key: must be a symmetric key that is at least AES 256-bit or equivalent.
  - Wrapping key: must be a symmetric key that is at least AES 256-bit or equivalent.

- **HSM Key Attributes**

  Depending on the model, the HSM may have key attributes that define the usage of the key. The key attributes can be restricted as needed, but do note that AccessMatrix E2EEA requires certain attributes to be set for it to function properly, such as encryption and decryption attributes. Also, note that while some key attributes may make the keys more secure such that they cannot be exported out, they may affect the key backup process. Refer to the respective HSM documentation for more information.

- **(For Pseudo E2EEA Implementation) HSM Usage**

  For better security and protection, an HSM can be used to store the Storage key and Wrapping key. If the Storage key is from an HSM key, then the encryption and decryption of the stored password hash will be done by the HSM. Thus, for every password verification, the Pseudo E2EEA login module will ask the HSM to decrypt to get the password hash. If Wrapping key is from an HSM key, the RSA public-private key pairs will be protected by the HSM key when the key pairs are stored in the database.

Integration Considerations

- **E2EEA Client Component**

  E2EEA client component is meant to be loaded and called in the user device (browser or mobile app). It is not meant to be run at the application's server-side. Calling the client component's encryption function in the application server-side does not provide end-to-end protection of the user password.

- **E2EEA Session Replication Delay**

  E2EEA requires the preauthentication step, which creates an E2EEA session used to prevent replay attacks. With multiple UAS instances, this E2EEA session is replicated by UAS via Ehcache replication. This replication typically happens very quickly (within 1 second). Thus, it is best that preauthentication is only called when the login page is loaded (instead of when the user presses the login button). If preauthentication is called when the user has entered the password and presses the login button, there is the risk of the E2EEA session replication not happening fast enough. When the E2EEA session is not yet replicated to another UAS instance, that UAS instance will not be able to verify the login request successfully.

Performance Considerations

RSA decryption is the most expensive process in the E2EEA flow. Higher RSA key lengths will also reduce performance significantly. Thus, there must be a balance between security and performance. It is best to get information on the expected peak performance (e.g. a system may have a requirement of 50 authentications/sec during peak hour), and then referring to our benchmark report to check if the performance requirement with the selected key size is satisfactory. However, as mentioned in the security consideration, certain key lengths are required to meet the security requirement (e.g. RSA key must be at least 2048-bit, AES must be at least 256-bit).

- **(For Pseudo E2EEA Implementation) With or without HSM**

  Without an HSM, all encryption and decryption is done by the AccessMatrix Server. Using an HSM key for Wrapping key does not affect performance much as usually the call to HSM to decrypt RSA key pairs are done only once in the lifetime of the login module. Using an HSM key for Storage key may affect performance. However, in general, symmetric-based encryption/decryption using Storage key does not take much processing power.

- **Adding more UAS/HSM instances**

  Additional UAS instances and HSMs can also be deployed to handle higher performance. Adding

UAS instances should have a higher impact due to the RSA processing that is done by UAS instances.

3.2. UAS E2EEA with SafeNet ProtectServer HSM Implementation

The following pages provide detailed instructions regarding the implementation of the UAS E2EEA solution with a SafeNet ProtectServer HSM:

- Deployment
- Configuration
- SafeNet JCProv Failover

3.2.1. Deployment

This section describes the installations and deployments required for UAS E2EEA FM with SafeNet ProtectServer HSM implementation.

Install Oracle JDK/JRE

<table>
<tr>
<td>Windows</td>
<td>
Notes:

It is assumed that the SafeNet ProtectServer HSM device is already installed and configured before performing the following steps in this section. Refer to UAS SafeNet ProtectServer HSM Deployment Guide for the deployment details.
Install the supported Oracle JDK/JRE version in both UAS AccessMatrix server and HSM Client server (if both are deployed separately). Refer to the AccessMatrix Supported Platforms document to know the supported JDK/JRE version.

Download and install the JDK executable from Oracle website.
Set an environment variable with the following details:
<code>Variable name: JAVA_HOME</code>
<code>Variable value: <JDKInstallationFolder> (e.g. C:\Program Files\Java\jdk1.8.0_181)</code>
You will also be required to add bin location of your java binaries to the systems PATH variable. Edit PATH variable under System variables and append string ";%JAVA_HOME%\bin" in variable value and save it.

Notes:
If you do not install the Java environment, the following two batch files will not execute:

gCTAdmin.bat
KMU.bat

</td>
</tr>
<tr>
<td>Linux</td>
<td>
Notes:

It is assumed that the SafeNet ProtectServer HSM device is already installed and configured before performing the following steps in this section. Refer to UAS SafeNet ProtectServer HSM Deployment Guide for the deployment details.
Install the supported Oracle JDK/JRE version in both UAS AccessMatrix server and HSM Client server (if both are deployed separately). Refer to the AccessMatrix Supported Platforms document to know the supported JDK/JRE version.

Download and install the JDK rpm file from Oracle website.
<code>rpm -ivh <jdk_package>.rpm</code>

</td>
</tr>
</table>

	2. Add the Java location path in JAVA_HOME. For example:
	`export JAVA_HOME=<jdk or jre installation path>`
	`export PATH=$JAVA_HOME/bin:$PATH`

Install SafeNet ProtectServer HSM Client Drivers

Windows	**Notes:** • Prior to client driver deployment, ensure that AccessMatrix server is already installed. You may refer to *AccessMatrix Common Deployment Guide* for installation steps, if necessary. • In this guide, the term <*UAS_DIR*> will be used to refer to the installation directory of AccessMatrix UAS on Windows platforms using Apache Tomcat. Ensure that the AccessMatrix UAS server is installed prior to integration of E2EEA FM with SafeNet ProtectServer HSM. • Retrieve the Windows client drivers from SafeNet ProtectServer HSM package. In this guide, PTKC 5.7.0 version is installed. • In this guide, the drivers will be installed on Windows (64-bit) and the term <*HSMDRIVER_DIR*> will be used to refer to the installation directory of SafeNet ProtectServer drivers. • Ensure that the **.NET Framework version 3.5 and 4.5 and Microsoft Visual C++ (MSVC) 2010** redistributable runtime packages are installed. Install the following SafeNet ProtectServer Client drivers: **a. Network HSM Access Provider** This package enables connection to one or multiple HSMs. It acts as a data abstraction layer for which you can add multiple front-end Application Program Interfaces (APIs). • Run the **PTKnethsm.msi** file and follow the on-screen instructions to proceed with the installation. • During installation, you will be prompted to enter the IP Address or name of the SafeNet ProtectServer HSM. • Enter the HSM IP Address or Fully Qualified Name (FQDN) and an installation complete window will appear. (Ensure that the IP address here must belong to the HSM device 0. • Click **Close** to exit the installation wizard. The driver's binary directory is automatically appended to PATH environment variable. • After installation, run the **hsmstate** command in the command prompt to verify whether HSM Access Provider is installed successfully. The response should be: `HSM device 0: HSM in NORMAL MODE. RESPONDING. Usage Level-0%` **Note**: The HSM IP Address configuration is stored in the *HKEY_LOCAL_MACHINE\SOFTWARE\Safenet\HSM\NETCLIENT\ET_HSM_NETCLIENT_SERVERLIST* registry key. **b. ProtectToolkit C (PTKC) Runtime**

	1. Run the **PTKcprt.msi** file and follow the on-screen instructions to proceed with the installation. 2. An installation complete window will appear. 3. Click **Close** to exit the installation wizard. The driver's installation directory is automatically appended to PATH environment variable. 4. Copy **jcprov.jar** from *<HSMDRIVER_DIR>\Protect Toolkit C RT* (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT*) to *<UAS_DIR>\tomcat\webapps\am5\WEB-INF\lib*. 5. Restart AccessMatrix UAS server. **c. (Optional) ProtectToolkit C (PTKC) Software Development Kit (SDK)** **Note**: Installation of this driver is optional. It is only required to install on the machine used to sign and deploy the FM. For normal AccessMatrix Server runtime, it is not required to install this component. This toolkit provides the PKCS#11 interface and talks to the access provider, which in turn routes the request to the HSM. 1. Run the **PTKcpsdk.msi** file and follow the on-screen instructions to proceed with the installation. 2. During installation, you will be prompted to select a Cryptoki Provider. 3. Select **HSM (local adapter or remote server)** as the Cryptoki Provider. 4. An installation complete window will appear. 5. Click **Close** to exit the installation wizard. The driver's library and binary directories are automatically appended to PATH environment variable.

Linux	**Notes**:
	• Retrieve the Linux client drivers from SafeNet ProtectServer HSM package. In this guide, PTKC 5.7.0 version is installed.
	• The term *<UAS_DIR>* will be used to refer to the installation directory of AccessMatrix UAS on Linux platform using Apache Tomcat. Ensure that the AccessMatrix UAS server is installed prior to integration of E2EEA FM with SafeNet ProtectServer HSM.
	
	a. Network HSM Access Provider
	
	This package enables connection to one or multiple HSMs. It acts as a data abstraction layer for which you can add multiple front-end Application Program Interfaces (APIs).
	
	1. Install this driver by using the following command:
	`rpm -ivh <nethsm_package>.rpm`
	For example:
	`rpm -ivh PTKnethsm-5.7.0-RC2.x86_64.rpm`
	
	b. ProtectToolkit C (PTKC) Runtime

1. Install this driver by using the following command:
   `rpm -ivh <cprt_package>.rpm`
2. For example:
   `rpm -ivh PTKcprt-5.7.0-RC2.x86_64.rpm`
3. After installation, run the following command:
   `ln -s /opt/safenet/protecttoolkit5/ptk/lib/libjcprov.so <UAS_DIR>/tomcat/bin/libjcprov.so`
4. Restart AccessMatrix UAS server.

c. (Optional) ProtectToolkit C (PTKC) Software Development Kit (SDK)

> **Note**: Installation of this driver is optional. It is only required to install on the machine used to sign and deploy the FM. For normal AccessMatrix Server runtime, it is not required to install this component.

1. Install this driver by using the following command:
   `rpm -ivh <cpsdk_package>.rpm`
   Example:
   `rpm -ivh PTKcpsdk-5.7.0-RC2.x86_64.rpm`

Perform the following steps after installation:

1. Add the PATH and LD_LIBRARY_PATH environment variables with the SafeNet ProtectServer HSM directories. For example:
   `export PATH=$PATH:/opt/safenet/protecttoolkit5/ptk/bin)`
   `export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/safenet/protecttoolkit5/ptk/lib)`
2. Add the SafeNet NETCLIENT Server List by adding the following line:
   `export ET_HSM_NETCLIENT_SERVERLIST=<HSM_IP>:<HSM_PORT>`

   > **Note**: Substitute HSM_IP and HSM_PORT with the actual HSM IP address and port.
3. Save the changes.
4. Run `hsmstate` command to verify whether HSM Access Provider is installed successfully. The response should be:
   `HSM device 0: HSM in NORMAL MODE. RESPONDING. Usage Level-0%`

> **Important Note**: Perform the same steps above (i.e. setting of environment variables) if the OS user used to install the drivers is not the same with the OS user used to start the AccessMatrix server.

Initialize Adapter

> **Note**: Perform the following steps if Adapter is not yet initialized.

1. Launch **Safenet, INc. -- Adapter Management** by running the **gCTAdmin** file (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\gCTAdmin.bat*).
2. The token label is automatically set as "Admin Token" and it will occupy the last slot. Enter the **SO PIN** and **User PIN**. Click **OK** button to confirm.

Initialize Token

The following steps will initialize a token.

1. Launch **Safenet, INc. -- Adapter Management** by running **gCTAdmin** (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\gCTAdmin.bat*).
2. Enter **User PIN** and proceed to login.
3. After successful login, select **Edit > Tokens...** menu. The **Manage Tokens** dialog box is displayed.
4. Select "uninitialized token" from **Slot** drop-down list and click on **Initialize** button to initialize the token slot. An **Initialize Token** dialog box is displayed.

   > **Note**: If no uninitialized token is available, proceed to the next section to create a slot before initializing a token.

5. Enter the **Token Label** (e.g. "E2EEAKEYS"), **Security Officer PIN** and **User PIN**.
6. Click **OK** button to confirm the token initialization.
7. Click **Done** button to close the **Manage Tokens** dialog box.

Create Token Slot

The following steps will create a new token slot.

> **Note**: Perform steps 1 and 2 only if **Safenet, INc. -- Adapter Management** is not displayed yet.

1. Launch **Safenet, INc. -- Adapter Management** by running **gCTAdmin** (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\gCTAdmin.bat*).
2. Enter **User PIN** and proceed to login.
3. After successful login, select **File > Create Slots...** menu.
4. Enter the number of slots to create and click **OK**.

   The GCTAdmin will restart to reflect the newly created tokens with uninitialized slots. For example: If an Administrator enters "1" in the field, and click **OK**, one token will be created with an uninitialized slot.

Delete Token Slot

The following steps will delete a token slot.

> **Note**: Perform steps 1 and 2 only if **Safenet, INc. -- Adapter Management** is not displayed yet.

1. Launch **Safenet, INc. -- Adapter Management** by running **gCTAdmin** (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\gCTAdmin.bat*).
2. Enter **User PIN** and proceed to login.
3. After successful login, select **File > Delete Slots...** menu.
4. Select the slot to delete and click **OK**. The GCTAdmin will restart to reflect the change made.

Deploy E2EEA FM to SafeNet ProtectServer HSM

> **Note**: Skip this section if the E2EEA FM is already deployed to SafeNet ProtectServer HSM.

Secure the E2EEA Functionality Module (FM) file from i-Sprint's E2EE FM Builds repository. The SafeNet ProtectServer HSM binaries are named **E2EEFM.PSE2.bin.version**, e.g. FM version 1.06 is **E2EEFM.PSE2.bin.0106**. Copy this FM binary file to a deployment folder (<*FM_Location*>), e.g. *C:\E2EEA_Files*.

E2EEA FM must be signed with trusted certificate before it can be deployed to SafeNet ProtectServer HSM. In order to do this, certificate for E2EEA FM must be generated and imported as trusted certificate. Then, the FM must be signed with the trusted certificate.

> **Important Note:** Ensure that ProtectToolkit C (PTKC) Software Development Kit (SDK) driver is installed before performing the steps below.

1. Open the command prompt and change the directory to the deployment folder (e.g. *C:\E2EEA_Files*).
2. Create a signing certificate by following the command below:
    a. Run `ctcert c` to create a 256-bit certificate and provide a certificate name to identify the certificate for FM purpose. The example below uses "am5-fm" as certificate name:
    `ctcert c -s0 -k -trsa -z2048 -lam5-fm`
    b. Enter the **User Pin** of the token slot.
    c. Enter the Common Name, Organization, Organization Unit, Locality, State and Country of the certificate based on preference, e.g.

    Code

```
Common Name: E2EEFMCert
Organization: i-Sprint
Organizational Unit: i-Sprint
Locality: BEDOK
State: SG
Country: SG
```

d. Run `ctcert l` to check if the certificate is successfully created.

3. Run `ctcert x` to export the newly created certificate to a file, e.g. **am5-fm.cer**:

   `ctcert x -s0 -lam5-fm -fam5-fm.cer`

4. Execute `ctcert i` to import the newly exported certificate into the Admin Slot.

   a. Run `ctcert i -s1 -lam5-fm -fam5-fm.cer`.

   b. Enter **User Pin** of the Admin token slot.

   c. Run `ctcert l` to check if the certificate is successfully imported.

   e.g. `ctcert l -s1`

   > **Note**: Admin slot is usually the last token slot. It usually has the name "AdminToken" along with the serial number of the HSM, e.g. AdminToken (493567).

5. Set trusted certificate in admin token slot by executing `ctcert t` command.

   a. Run `ctcert -s1 -lam5-fm`.

   b. Enter the **Security Officer (SO) Pin** of the Admin token slot.

   c. Run `ctcert l` to check if the certificate is successfully trusted.

   e.g. `ctcert l -s1`

   > **Note**: Asterisk in the leftmost side of the certificate name indicates trusted.

6. Sign the FM file using the newly created FM Certificate (i.e. **am5-fm.cer**).

   a. Run `mkfm -k` to create a signed FM file:

   `mkfm -k<Slotlabel>/am5-fm -f <FMLocation>\E2EEFM.PSE2.bin.0108 –o E2EEFM.PSE2.bin.0108.signed`

   Example:

   `mkfm -kE2EEAKEYS/am5-fm -f E2EEFM.PSE2.bin.0108 –o E2EEFM.PSE2.bin.0108.signed`

   b. Enter the **User Pin** of the token slot 0.

7. Load the signed FM into the HSM Device by executing the following steps:

a. Run the `ctconf -j` command to upload the signed FM to the HSM Device.
   `ctconf -jE2EEFM.PSE2.bin.0108.signed -bam5-fm`

b. Enter the **User Pin** of the Admin token slot.

c. Verify FM upload status by executing `ctconf -s`.

8. Run `ctconf -v` to check the FM in the HSM configuration settings.

3.2.2. Configuration

This section details the configuration steps relating to the implementation of UAS E2EEA FM with a SafeNet ProtectServer HSM.

Setup Steps

Before you start the implementation, it is assumed that you have completed the planning and implementation checklists, and you have understood the concepts behind the UAS E2EEA FM with SafeNet ProtectServer HSM solution. Perform the following steps to configure UAS E2EEA with a SafeNet ProtectServer HSM:

- **Create E2EEA Keys**
  A hardware security module (HSM) is a physical computing device that safeguards and manages digital keys for strong authentication and provides cryptoprocessing. Refer to Create E2EEA Keys to know the required keys for this UAS E2EEA solution.
- **Upload E2EEA License**
  Server license is used to enable general and product-specific features. For E2EEA license, refer to Upload E2EEA License for details.
- **Configure E2EEA Login Policy**
  E2EEA Login Policy is a basic login policy that defines the behavior settings, precondition checks, post-event actions of login, reset password and change password operations. Refer to Configure E2EEA Login Policy for details.
- **Configure SafeNet ProtectServer E2EEA Login Module**
  SafeNet ProtectServer E2EEA Login Module provides SafeNet ProtectServer-based E2EE login method for the user who may reside in either an internal or external user store. Refer to Configure SafeNet ProtectServer E2EEA Login Module for details.
- **Create Authentication Realm** Authentication Realm associates user credentials for authentication based on a lookup module and login module(s) where user entries can be created into local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). Refer to Create Authentication Realm for details.
- **Assign Authentication Realm to User**
  The user is an entity that accesses the system and may belong to a default store (local repository) or an external user store, e.g. AD (Active Directory). Assign the configured

authentication realm to the user by adding it to the **Login Accounts** tab. Refer to Assign Authentication Realm to User for details.

Create E2EEA Keys

The following keys are required to be generated for E2EEA FM with SafeNet Protect Server HSM solution:

- **Public-Private Key Pair**

  The key used to encrypt the client-side PIN.
- **Storage PIN Encryption Key**

  The key used to encrypt the stored PIN.

The following table describes the attributes that can be set when creating a key using the Key Management Utility (KMU). For different customers, there will have different expectations for their keys. The list below is customizable according to the requirements of different customers.

Note: Where "-" is indicated, set the value based on the customer's requirements.

Key Attribute	Public Key	Private Key Storage	PIN Encryption Key
Persistent	TRUE	TRUE	TRUE
Extractable	FALSE	FALSE	FALSE
Derive	FALSE	FALSE	FALSE
Export	FALSE	FALSE	FALSE
Verify	FALSE	FALSE	FALSE
Sensitive	FALSE	TRUE	TRUE
Exportable	FALSE	FALSE	FALSE
Encrypt	TRUE	FALSE	TRUE
Wrap	FALSE	FALSE	FALSE
UnWrap	FALSE	TRUE	FALSE
Modifiable	-	-	-
Private	FALSE	TRUE	TRUE

Key Attribute	Public Key	Private Key Storage	PIN Encryption Key
Sign	FALSE	FALSE	FALSE
Decrypt	FALSE	TRUE	TRUE
Import	FALSE	FALSE	FALSE

Meaning of Key Attributes

Key Attribute	Description
Persistent	Stores the object on non-volatile memory. Persistent objects can be accessed after session termination. Disabled and selected by default. (Unclickable)
Extractable	An extractable key can be wrapped (encrypted with another key) and extracted from the HSM.
Derive	Indicates that the key can be used in key derivation functions.
Export	Indicates the key may be used to export other keys (similar to the wrap function).
Verify	Indicates that the key may be used for verifying signatures or MAC values.
Sensitive	If a key is sensitive, the key's value may not be revealed in plain text. Once a key becomes sensitive it cannot be modified to be non-sensitive.
Exportable	An exportable key may be wrapped (encrypted with another key), but only with keys marked with the **Export** attribute.
Encrypt	Indicates that the key may be used for encryption.
Wrap	Indicates that the key may be used to wrap (i.e. extract) other keys.
UnWrap	Indicates that the key may be used to unwrap keys.
Modifiable	Indicates whether or not the object is modifiable, that is, if the object’s attributes may be modified after creation.
Private	Defines whether the object is protected by the user PIN. A private object is only accessible to an application that has supplied the user PIN.
Sign	Indicates that the key may be used for signing.
Decrypt	Indicates that the key may be used for decryption.
Import	Indicates the key may be used to import other keys.

Generate Public Key

Perform the following steps to generate the public key (e.g. "RSA01") to the initialized E2EEA token slot (e.g. "E2EEAKEYS"):

1. Launch **Key Management Utility** by running KMU (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\KMU.bat*).
2. Select the previously initialized token (e.g. "E2EEAKEYS") and enter the **User PIN**.
3. Click **Options > Create > Key Pair** in the menu. The **Generate Key Pair** dialog box is displayed.
4. Select the **Key Pair Type**, e.g. "RSA".
5. Enter the **Subject** value, e.g. "AM5 E2EEA Keys".
6. Enter the **Key Size (bits)** value, e.g. "2048".
7. Enter the labels for **Public Key** and **Private Key** fields, e.g. "RSAO1".
8. Specify what attributes a key will have by checking/unchecking the key attribute checkboxes.
9. Click **OK** to confirm the configuration.

Note: Repeat steps 3 to 9 to create another RSA key.

Generate Storage PIN Encryption Key

Perform the following steps to generate the storage PIN encryption key (e.g. "AES01") to the initialised E2EEA token slot (e.g. "E2EEAKEYS"):

1. Launch Key Management Utility by running KMU (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\KMU.bat*).
2. Select the previously initialized token (e.g. "E2EEAKEYS") and enter the **User PIN**.
3. Click **Options > Create > Secret Key** in the menu. The **Generate Secret Key** dialog box is displayed.
4. Select the **Mechanism**, e.g. "AES".
5. Enter the **Label** value, e.g. "AES01".
6. Enter the **Key Size (bits)** value, e.g. "256".
7. Specify what attributes a key will have by checking/unchecking the key attribute checkboxes.
8. Click **OK** to confirm the configuration.

Note: Repeat steps 3 to 7 to create another AES key.

Upload E2EEA License

Before doing any configuration in the Admin Console, secure the server license required to implement UAS E2EEA FM with SafeNet ProtectServer HSM. Ensure that **E2EEA Base** license feature is set to **Yes** to enable E2EEA FM support and to display "SafeNet ProtectServer E2EEA Login Module" in the login module implementation list. To load the server license:

1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
2. Select **Server Id** and click **Edit**.
3. Select the **Licensed Features**
4. Click **Upload Licence File**
5. Save the changes.
6. Restart the AccessMatrix server for the new license to take effect.

Configure E2EEA Login Policy

To create a new basic login policy, navigate to **Administration Dashboard > Security Policy > Authentication** and click **Login Policy** on the left pane. Click the **Create** button and select the "Basic Login Policy" implementation from the **Choose Login Policy Implementation** page. Then, click **Next** and provide the details for the login policy attributes before saving the changes.

Attribute Name	Description
*Login Policy Id	Unique login policy Id, e.g "E2EEALoginPolicy".
*Login Policy Name	Name of login policy, e.g. "E2EEALoginPolicy".
Description	Description of login policy.
Version	Login policy entry version.
Implementation	Type of login policy implementation, i.e. "Basic Login Policy".
Attributes Tab	
Login Account	
Check login account status	If set to true then login account status will be checked and will display corresponding error message if counter already reached the set bad login count. Otherwise, user will be allowed to login if correct

Attribute Name	Description
	password is entered regardless if the login account status was already disabled. Default: true
Disable login account if bad login count reaches	Sets the maximum bad login count. If bad login count reaches the specified number, then login account will be disabled. Default: 3
Allow resetting password without force password change	Allows password reset without forcing a password change. The behavior will depend on the selected option: • "Disabled" - Disallow password reset. • "Enabled in API only" - Allows password reset in API only. • "Enabled in API and Admin Console" - Allows password reset in both API and Admin Console. Default: Disabled
Allow enabling of login account without resetting password	Allows login account activation without resetting the password. Default: false **Note**: This attribute is applicable only for **Modify Account** button which is hidden by default. This button can change the login account's status. Uncomment the Modify Account button tag in **field-definitions.xml** to display the button.
Increment bad login count for the wrong password used during change password	Specifies the action to be done if change password failed due to an incorrect password. • "Do not increment" - No action to be taken. • "Increment same bad login counter as login" - Increment the bad login count if change password failed due to an incorrect password. • "Increment separate bad login counter" - This will create or increment a separate counter if a change password failed due to an incorrect password. Default: Do not increment
Auto Unlock	
Enable Auto Unlock By Time	If enabled, then locked accounts will be unlocked after the specified time in **Auto Unlock Time Duration** attribute. Default: false
Auto Unlock Time Duration (in minutes)	Sets the time duration when auto unlock will be executed. Value should be from 1 to 1440. Default: 30
Others	

Attribute Name	Description
Include new password in change and reset password event	Setting this to true will include the new password in event object for successful change and reset password events. Default: false **Note:** Only enable this if required, e.g. for provisioning.

Configure SafeNet ProtectServer E2EEA Login Module

To create a new login module, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane. Click the **Create** button and select the "SafeNet ProtectServer E2EEA Login Module" implementation from the **Choose Login Module Implementation** page. Then, click **Next** and provide the details for the login module attributes before saving the changes.

Attribute	Description
*Login Module Id	Unique Id of login module, e.g. "E2EEPSLM".
*Login Module Name	Name of login module, e.g. "E2EE PS Login Module".
Description	Description of login module.
Version	Login module entry version.
Implementation	Type of login module implementation, i.e. "SafeNet ProtectServer E2EEA Login Module".
Handler	Handler of the login module, i.e. "SafenetPSGLoginModule".
Configuration Tab	
Login Configuration	
Login Policy	The login policy specifies if bad login count should be incremented and if login account should be locked upon exceeding maximum bad login count, as well as other policy controls. If no login policy is attached, then the corresponding policy controls like incrementing bad login count will not take effect. Example: E2EEALoginPolicy
Authentication Level	(Unused) This is an attribute of login module used for UAM product. Default: 1
HSM Basic Configuration - Require restart to take effect	

Attribute	Description
*HSM User PIN	This is the same PIN you used to login to Key Management Utility (KMU).
HSM Slot ID	For JHSM (not by default) network access only: The slot containing your keys for this module. Default: 0
*HSM Slot Name	For JCProv (default) network access only: The slot or token labels for each HSM; if label is not unique, prefix with HSM serial no.(e.g. "408801:Slot01"). Example: E2EEAKEYS
Server-based HSM Slot Name	This is to configure HSM slot/token labels based on server id. If server id is not mentioned here, it will use the default HSM Slot Name configuration. Syntax: serverId1,label1,label2,...|serverId2,label1,label2,...|... Example: am2,slot1,408801:slot2|am3,408801:slot2,slot1
*Public Key Names	List of public key names or IDs to use to encrypt the client-side PIN. • During preauthentication, one of these public keys will be randomly selected Example: RSA01
RPIN OAEP Hash Algorithm	Values: • "SHA1 (default)" • "SHA256" Available from version 5.6.1-GA-E05, 5.6.2-GA
*Storage PIN Encryption Key Names	List of PIN encryption key names or IDs to use for encrypting the stored PIN. • If more than one is specified, one of these keys will be randomly selected to encrypt the storage PIN. • The storage PIN will contain the key index so that the correct key can be used during verification. Example: AES01
PIN Protection Key Index/Label	If you are using E2EEA ATM PIN translation function, you need to configure the key used for encrypting the translated PIN here.
Password Policies	

Password Quality Policy	The password quality policy to use.
	Note: If **Check password quality policy in HSM** is set to "true":
	• For AccessMatrix versions below 5.3.3.3304, only Password Age and Password History of the policy are checked by AccessMatrix server.
	• For AccessMatrix versions 5.3.3.3304 onwards, besides Password Age and History, the following password policy checks are also performed in the HSM:
	  ○ Min length
	  ○ Max length

Attribute	Description
	○ Min upper case ○ Min lower case ○ Min numeric ○ Min alphabetic ○ Min non-alphanumeric ○ Allow Repeat character ○ Number of consecutive repeating char allowed ○ Allow sequence of alphabetic char ○ Allow sequence of numeric
Password Quality Policy (System Assigned)	If a policy is specified for this attribute, it will apply to all system-generated passwords instead of the policy configured in **Password Quality Policy**. Specifically, this policy will apply to "System Assigned", "System Assigned (pending)", "System Assigned PDF" or "System Assigned PDF (pending)" passwords. The policy specified in **Password Quality Policy** will still apply for non-system-generated passwords (e.g. user-configured passwords). If no policy is specified for this attribute, the policy specified in **Password Quality Policy** will apply for system-generated passwords as well.
Check password quality policy in HSM (if supported by HSM)	Setting to "true" will check the password quality policy in HSM. Default: false
Password Expiry Policy	The password expiry policy to use, e.g. "DefaultPasswordExpiryPolicy".

Create Authentication Realm

To create a new Authentication Realm, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane. Click **Create** and provide the details for the authentication realm attributes before saving the changes.

Attribute Name	Description
*Authentication Realm Id	Unique Id of authentication realm, e.g. "E2EEAPR".
*Authentication Realm Name	Name of authentication realm, e.g. "E2EEA ProtectServer Realm".
Description	Description of authentication realm.
Version	Authentication realm entry version.

Attribute Name	Description
Configuration Tab	
Automatically assigned to users	Allow this authentication realm to be automatically assigned to all users.
User Lookup Module	Lookup module is used to validate user identity (i.e. user search in user store(s) or other validation methods). Example: "SystemLookup"
First(1st) Login Module	The first login method to use for user authentication, e.g. "E2EE PS Login Module (E2EEPSLM)".

Assign Authentication Realm to User

A user may belong to a default store or external user store. You may refer to *AccessMatrix Common Administration Guide - Manager User Stores* to know about the different types of user stores and how to configure it. To create a new user in default store, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Provide values to the following attributes that are necessary for E2EEA implementation before saving the changes.

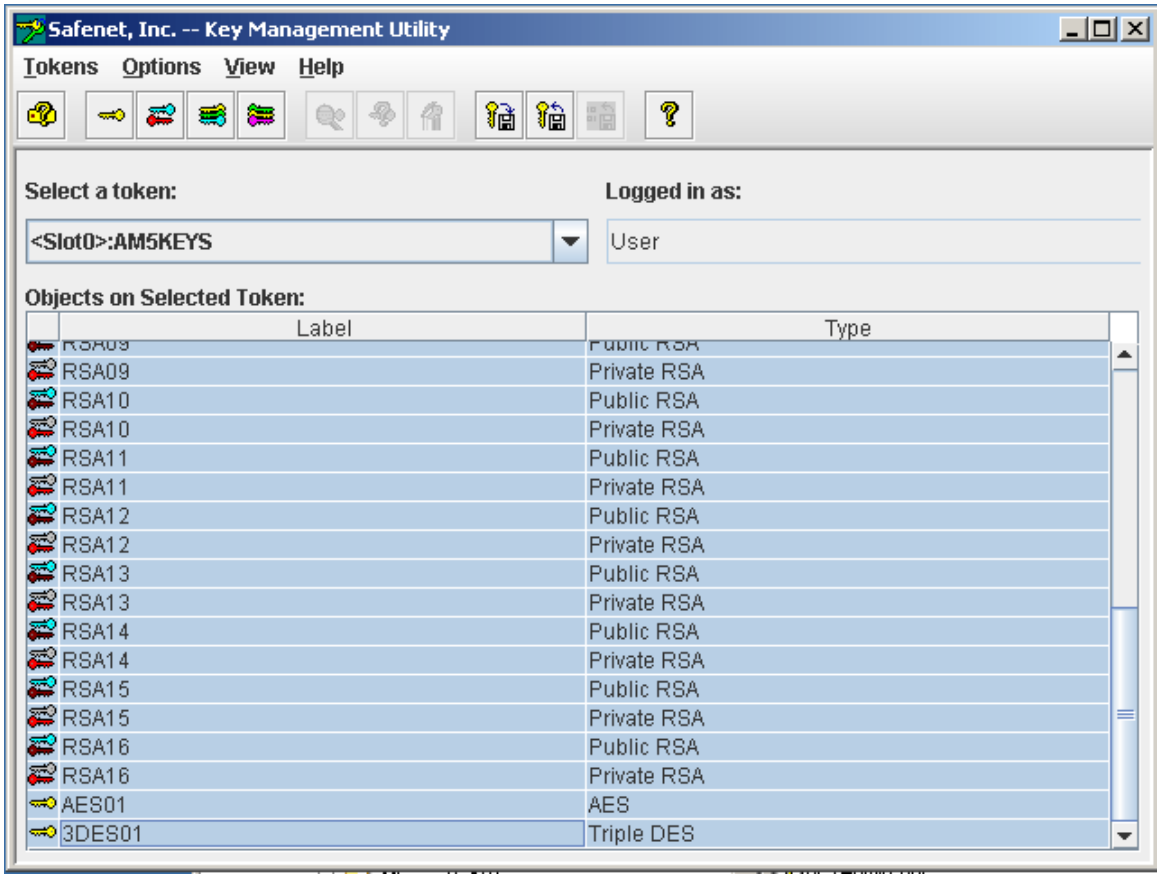
Attribute Name	Description
*User Id Unique	User Id.
*User Name	Name of user.
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of user store the user belongs to, i.e. "Default Store".
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to user. Click **Add** to display the **Choose Authentication Realm** dialog box. Select the authentication realm created in previous section (e.g. "E2EEA ProtectServer Realm (E2EEAPR)") and click **OK**. Both authentication realm and the login module(s) associated to it will be assigned to the user. Save the changes.	

You may refer to *AccessMatrix Common Administration Guide - Manager Users* for detailed information about users.

Back Up Token to Smart Card

> **Note**: This step is optional. This will back up token objects - RSA Public/Private keys, AES keys, and certificates.

1. Launch **Key Management Utility** by running *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\KMU.bat* or by clicking the KMU shortcut icon.
2. Select **AM5KEYS** token. The **Enter PIN** dialog box is displayed.
3. Enter the **AM5KEYS User PIN** and click **OK**.
4. Select token objects to back up to Smart Card. Use mouse to click on object. To select multiple objects, use Ctrl and Shift key with mouse click.



5. Select the **Export** option from **Options** menu.
6. Select **<Random key>** as the wrapping key and select the **Write to smart card** option. Select **<Slot1>:<No Token>** in the **Selected Smartcard** list box. Specify the **Batch Name** and enter "2" or higher in **No. Custodians**. Click the **OK** button to continue.

7. KMU will prompt to enter **User Name** and **Smartcard PIN**. The 1st custodian should specify the user name and smart card PIN, and then click the **OK** button to continue.
8. The 1st Custodian should insert the Smart Card into card reader and click the **OK** button.
9. Click the **OK** button to confirm initializing 1st Custodian’s Smart Card.
10. Remove the 1st Custodian's Smart Card and insert the 2nd Custodian's Smart Card into the card reader. Then, click the **OK** button.
11. The 2nd Custodian should enter **User Name** and **PIN**, then click the **OK** button.
12. Click the **OK** button to confirm the initialization of the 2nd Custodian’s Smart Card.
13. Remove Smart Cards and store securely.

3.2.3. SafeNet JCProv Failover

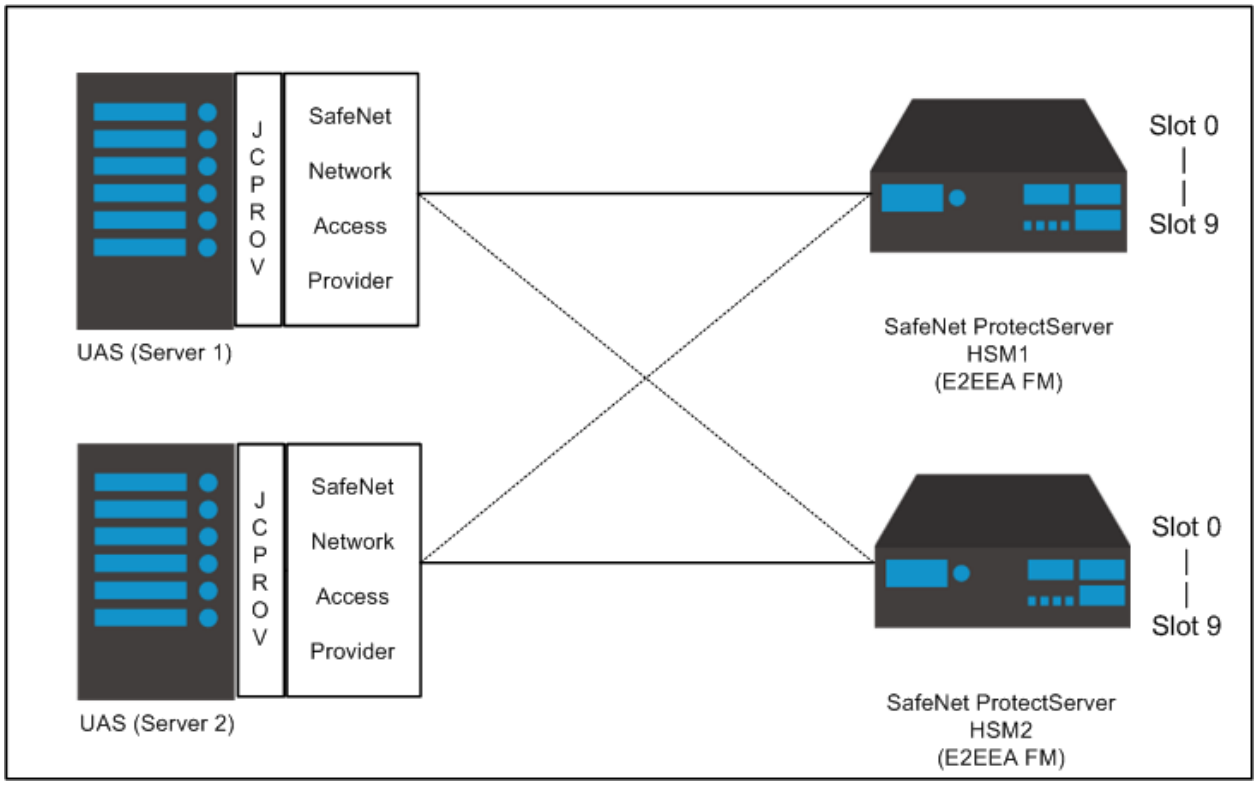
SafeNet JCProv API

AccessMatrix UAS server uses SafeNet JCProv API to integrate with i-Sprint's E2EEA FM on SafeNet ProtectServer HSM. SafeNet Network Access Provider is used to access a remote HSM and it allows a configuration of multiple SafeNet ProtectServer HSMs. The HSM IP Address and port configurations are stored as part of the Network Access Provider configuration.

The JCProv API sees all of the combined slots of the HSMs (e.g. both HSM1 and HSM2 have 10 slots each, JCProv API shall see 20 virtual slots where 0-9 virtual slots correspond to 0-9 slots of HSM1 and 10-19 virtual slots belong to 0-9 slots of HSM2). The combining of slots happens during JCProv API initialization. Therefore, based on previous example, if during initialization only secondary HSM is up, JCProv API shall only return 10 slots (virtual slot 0-9).

For every AccessMatrix request to HSM, AccessMatrix will open an HSM session, do the HSM operation, and close the session. When opening a session, AccessMatrix needs to specify the virtual slot to open. Thus, AccessMatrix can choose which HSM to connect to by indicating the slot Id.

SafeNet does not support failover when FM is used and recommends i-Sprint to make use of the virtual slots as a way to achieve failover.



Failover Concept & Consideration

AccessMatrix UAS server can choose which HSM to connect to by specifying the slot Id. Based on previous example (20 virtual slots (slot 0-19), it corresponds to HSM1 slot 0-9 and HSM2 slot 0-9), AccessMatrix UAS server can connect to HSM1 slot 0 by specifying slot Id 0 and connect to HSM2 slot 0 by specifying slot Id 10.

The configuration of primary and secondary slots must be done using slot/token label as slot Id may shift around. AccessMatrix UAS server shall use slot/token label to find the primary and secondary slot Ids.

The failover can be achieved by first opening a session to a slot belonging to primary HSM. Then, do the required HSM-related operation. If somehow the operation fails with an error that is "unexpected by AM" (discussed at later point), then AccessMatrix shall:

- Close the existing session to primary HSM
- Write to a specific log file with error code
- Open a new session to a slot belonging to secondary HSM and retry the required operation

When the problem on primary HSM has been rectified and the HSM is brought online, JCProv API does not automatically recreate the connection. In order for JCProv to get the primary HSM connection again, JCProv API needs to be reinitialized/restarted. The best and safest way to achieve this is to restart AccessMatrix UAS server.

AccessMatrix UAS server provides a feature to reinitialize JCProv API without server restart. JCProv reinitialization process requires all JCProv-related activities to be halted. This process may take few seconds depending on the load at that time. This reinitialization feature will affect all JCProv API-dependent modules in AccessMatrix server. For example, if AccessMatrix has two modules, E2EEA and Key Management, or if AccessMatrix has two E2EEA login modules, then reinitialization feature will affect each other (affect all JCProv-dependent modules). And note that this feature is done on best-effort basis to workaround on the JCProv API limitation; thus, if JCProv API still fails to reestablish the connection, then there is nothing that AccessMatrix UAS server can do.

Conditions for Failover

The condition that triggers HSM failover is any error that is unexpected by AccessMatrix UAS server, such that AccessMatrix server cannot fulfill any HSM-related request. This error is referring to the error

returned by the HSM, not by the E2EEA FM. Thus, HSM shall return OK (no error) when E2EEA FM returns wrong OTP, out-of-sync, etc.

HSM errors that are expected by AccessMatrix UAS server:

- CRYPTOKI_ALREADY_INITIALIZED (expected during initialization)
- USER_ALREADY_LOGGED_IN (expected when AccessMatrix UAS server is opening a session)
- USER_NOT_LOGGED_IN (expected when AccessMatrix UAS server is doing HSM operation)
- ENCRYPTED_DATA_LEN_RANGE

The failover conditions above are parameterized in AccessMatrix UAS server configuration file so that it is easily tuned in testing and production operation.

Failover Logic

Note: The following example is used to explain the failover logic.

In this example, AccessMatrix UAS server is configured with these token labels: E2EE-HSM1, E2EE-HSM2.

Initialization

1. Initialize HSM (JCProv initialize).
2. Call JCProv to list all available tokens and their information (token label, slot Id, HSM serial number).
3. Initialize the High availability (HA) environment with token labels = E2EE-HSM1, E2EE-HSM2.
    a. Validate token labels, E2EE-HSM1 and E2EE-HSM2, and get the corresponding slot Ids. If any of the token label cannot be found, then print error to log/raise an alert.
    b. Put the resolved slot Ids in a list for HA purpose.
    c. Read the primary HSM from configuration and indicate the HSM in the list as the active HSM.

HSM Operation

1. Open the session to HSM (the active HSM).
2. Do the necessary operation (invoke E2EE FM).
3. Close the session.

> **Note**: If any unexpected error occurred on step 1 or 2 of HSM operations:
>
> a. Print the error to log/raise an alert.
> b. Make the next HSM in the list as the active HSM. If no more available HSM, then throw an exception to the caller.
> c. Redo from step 1 (open session).

Constraints & Assumptions

- It is best if slot/token labels are configured to be unique across HSMs. If this is not done, the HSM serial number will be used as prefix of the label.
- i-Sprint E2EEA FM uses HSM session to access/use any required key from the opened slot/token. Thus, it does not need to know the slot index of any required key.

Configurations

SafeNet ProtectServer HSM Configuration

The timeout configuration determines how fast JCProv returns an error to AccessMatrix UAS server when there is a problem when invoking an HSM function. It is better to set it to lower value, especially for the read timeout. However, the value may be affected by the environment. If possible, set the read and write timeout to 10-15 seconds.

- ET_HSM_NETCLIENT_SERVERLIST
    - Specifies multiple HSM IPs and ports (separated by space)
    - JCProv orders the virtual slots based on the order specified in this configuration
- ET_HSM-NETCLIENT_READ_TIMEOUT_SECS (default value is 300)
- ET_HSM_NETCLIENT_WRITE_TIMEOUT_SECS (default value is 60)
- ET_HSM_NETCLIENT_CONNECT_TIMEOUT_SECS (default value is 60)

AccessMatrix Server Configuration

More than one token label is required to configure the failover. For i-Sprint E2EE FM, the login module configuration allows administrator to specify multiple token labels. In order to enable reinitialization feature, this property can be set in **amsystem.properties** (default value is false):

am.crypto.e2ee.SafenetPSG.auto-reinit

It is best to have unique token labels across multiple HSMs. However, if the token labels to be used are somehow not unique, it is still possible to configure failover by adding HSM serial number as a prefix to the token label: <hsm_serial_no>:<token_label>. For example: 400081:E2EESlot, 400190:E2EESlot

Below are the HSM errors that are expected by AccessMatrix UAS server (that will NOT trigger failover):

- CRYPTOKI_ALREADY_INITIALIZED (expected during initialization)
- USER_ALREADY_LOGGED_IN (expected when AccessMatrix UAS server is opening a session)
- USER_NOT_LOGGED_IN (expected when AccessMatrix UAS server is doing HSM operation)

To override the list of expected error:

- Set am.crypto.JCProv.failover.ignored-error-codes in **amsystem.properties**.
- The error codes are Safenet ProtectServer HSM error codes. For multiple error codes, separate them by comma.

AccessMatrix has a special log category for failover-related:

- com.isprint.am.util.JCProvUtil.Failover

3.3. UAS E2EEA with SafeNet Payment HSM Implementation

The following pages provide detailed instructions regarding the implementation of the UAS E2EEA solution with a SafeNet Payment HSM:

- Deployment
- Configuration

3.3.1. Deployment

This section describes the installations and deployments required for UAS E2EEA FM with SafeNet Payment HSM implementation.

Install Oracle JDK/JRE

<table>
<tr>
<td>Windows</td>
<td>
Notes:

It is assumed that the SafeNet Payment HSM device is already installed and configured before performing the following steps in this section.
Install the supported Oracle JDK/JRE version in both UAS AccessMatrix server and HSM Client server (if both are deployed separately). Refer to the AccessMatrix Supported Platforms document to know the supported JDK/JRE version.

1. Download and install the JDK executable from Oracle website.
2. Set an environment variable with the following details:
<code>Variable name: JAVA_HOME</code>
<code>Variable value: <JDKInstallationFolder> (e.g. C:\Program Files\Java\jdk1.8.0_181)</code>
3. You will also be required to add bin location of your java binaries to the systems PATH variable. Edit PATH variable under System variables and append the string ";%JAVA_HOME%\bin" in variable value. Save it.
</td>
</tr>
<tr>
<td>Linux</td>
<td>
Notes:

It is assumed that the SafeNet Payment HSM device is already installed and configured before performing the following steps in this section.
Install the supported Oracle JDK/JRE version in both UAS AccessMatrix server and HSM Client server (if both are deployed separately). Refer to AccessMatrix Supported Platforms document to know the supported JDK/JRE version.

1. Download and install the JDK rpm file from Oracle website.
<code>rpm -ivh <jdk_package>.rpm</code>
2. Add the Java location path in JAVA_HOME. For example:
<code>export JAVA_HOME=<jdk or jre installation path></code>
export PATH=$JAVA_HOME/bin:$PATH
</td>
</tr>
</table>

Install SafeNet Payment HSM Client Drivers

The connection to SafeNet Payment HSM can be done directly via socket, therefore, installation of HSM drivers are not required. However, ensure that **General authentication HSM Id** is configured in Configure SafeNet PHEFT Login Module.

Note: For AccessMatrix versions before 5.6.0-GA, refer to the following section for the steps to be performed.

HSM Client Drivers Installation For AccessMatrix Versions Before 5.6.0-GA

Prior to performing the steps below, it is assumed that AccessMatrix was already installed and SafeNet Luna Payment HSM was already set up.

1. Install the SafeNet SDK in AccessMatrix server computer.

   Note: The recommended version is **MK2SDK-3.02.00.exe**.

   a. Run the installation file (**mk2sdk<version>.exe**).

   b. Follow the installation wizard. The destination directory of the SDK will be asked.

2. Install the SafeNet Security Communication Module in AccessMatrix server computer.

   Note: The recommended version is **ptkComm-5-09-00.exe**.

   a. Follow the installation wizard. The destination directory will be asked.

   b. Restart the computer.

To check the version:

- For PTK MK2, go to *<SafeNet installation folder>/Protecttoolkit MK2*. Right click on **kvcDiff.exe** and select **properties > details.**
- For PTK comm, go to *<SafeNet installation folder>/Protecttoolkit Comm*. Right click on **CommsConfig.exe** and select **properties > details.**

Perform the following steps to configure HSM:

1. Add HSM configuration in the Security Module Configuration.

   a. Go to **Safenet > PTK EFT COMM > Security Configuration Module**. A **Security Module Host Comms Configuration Utility** will be displayed.

   b. Click the **Add HSM** button. The **Security Module Configuration** dialog will be displayed.

2. Type a name in the **Name** field, e.g. "HSM1".

   Note: This is the same name that will be supplied in Admin Console configuration.

   a. Click **Configure** button. The **TCP/IP Comms Config** dialog will be displayed.

   b. Input the **HSM IP Address** and **Port** values, then click the **OK** button.

c. Click the **OK** button from **Security Module Configuration** dialog. The **Security Module Host Comms Configuration Utility** dialog will be displayed.

d. Click **Options > Security Module Global Options**. The **Options for all Security Modules** is displayed.

e. Select **Registry** button under **Configuration Storage Location**. Then, click the **OK** button to close the dialog.

f. Click **Exit** button.

3. HSM connection can be tested using ESM Tester. Go to **Safenet > PTK EFT COMM > ESM Tester**. The **Safenet Security Module host function tester** will be displayed.

   a. Select the name created in previous steps from ESM name combo box, e.g. "HSM1".

   b. Select **ALL** from the the Trace msg combo box.

   c. Click **Open** and browse for esm test files. Test files are located in the SafeNet install location, e.g. *C:\Program Files\Safenet\Protecttoolkit Comm\Test Files*. Click on the *EE0801* folder and select **EE0801_01.esm**.

   d. Click the **Go** button. If there are values for Received and Expect, then the connection to HSM is successful.

> **Note**: If **File** is selected as the **Configuration Storage Location**, check java.library.path in **am.log**, then copy all ini files from *<SafeNet installation folder>/Protecttoolkit Comm* and put them into the corresponding java library path. For example, in Windows environment, where Tomcat is used as Web Application server of AccessMatrix:

- If AccessMatrix is started by command prompt and `java.library.path=C:\Program Files\Java\jdk1.8.0_181\bin`
  - Copy all ini files from *<SafeNet installation folder>/Protecttoolkit Comm* and put them into *C:\Program Files\Java\jdk1.8.0_181\bin.*
- If AccessMatrix is started by service and `java.library.path=[AccessMatrixHome]\tomcat\bin`
  - Copy all ini files from *<SafeNet installation folder>/ Protecttoolkit Comm* and put them into *<AccessMatrixHome>\tomcat\bin.*

3.3.2. Configuration

This section details the configuration steps relating to the implementation of UAS E2EEA FM with a SafeNet Payment HSM.

Setup Steps

Before you start the implementation, it is assumed that you have completed the planning and implementation checklists, and you have understood the concepts behind the UAS E2EEA FM with SafeNet Payment HSM solution. Perform the following steps to configure UAS E2EEA with a SafeNet Payment HSM:

- **Upload E2EEA License**
  Server license is used to enable general and product specific features. For E2EEA license, refer to Upload E2EEA License for details.
- **Configure SafeNet Payment E2EEA Login Module**
  SafeNet Payment E2EEA Login Module provides SafeNet Payment-based E2EE login method for the user who may reside in either an internal or external user store. Refer to Configure SafeNet PHEFT Login Module for details.
- **Create Authentication Realm**
  Authentication Realm associates user credentials for authentication based on a lookup module and login module(s) where user entries can be created into local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). Refer to Create Authentication Realm for details.
- **Assign Authentication Realm to user**
  User is an entity that accesses the system and may belong to a default store (local repository) or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to the user by adding it to the **Login Accounts** tab. Refer to Assign Authentication Realm to User for details.

Upload E2EEA License

Before doing any configuration in the Admin Console, secure the server license required to implement UAS E2EEA FM with SafeNet ProtectServer HSM. Ensure that **E2EEA Base** license feature is set to "Yes" to enable E2EEA FM support and to display "SafeNet ProtectServer E2EEA Login Module" in the login module implementation list.

To load the server license:

1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
2. Select **Server Id** and click **Edit**.
3. Select the **Licensed Features** tab and select the first radio button. Browse for the license file.
4. Click **Upload Licence File** to upload the license file to the server.
5. Save the changes.
6. Restart the AccessMatrix server for the new license to take effect.

Configure SafeNet PHEFT Login Module

To support E2EEA with SafeNet Payment HSM, there is a need to create a new login module or edit the default SafeNet PHEFT E2EE login module (SafeNetPHEFTE2EE). This login module provides SafeNet PHEFT login method for the user who may reside in either an internal or external user store.

To create a SafeNet PHEFT Login Module, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane. Click the **Create** button and select "SafeNet PHEFT Login" implementation from the **Choose Login Module Implementation** page. Then, click **Next** and provide the necessary values to the **Configuration** tab attributes before saving the changes.

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
*Login Module Id	Unique Id of login module, e.g. "PHSMLM".	N/A	Y
*Login Module Name	Name of login module, e.g. "PAYMENT HSM Login Module".	N/A	Y
Description	Description of login module.	N/A	N
Version	Login module entry version.	N/A	N/A
Implementation	Type of login module implementation, i.e. "SafeNet PHEFT Login".	N/A	N/A
Handler	Handler of the login module, i.e. "SafenetPHEFTLoginModuleHandler".	N/A	N/A
Configuration Tab			

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
Login Configuration			
Login Policy	This specifies if bad login count should be incremented and if login account should be locked upon exceeding maximum bad login count, as well as other policy controls. If no login policy is attached, then the corresponding policy controls like incrementing bad login count will not take effect. Default: DefaultBasicLoginPolicy	N	
Authentication Level	Unused. **Note:** This is an attribute of login module used for UAM product. Default: 1	N	
HSM Basic Configuration			
General authentication HSM Id	Applicable and mandatory only if connecting to HSM via Direct Socket Protocol. Specify as "host:port" format, eg. "192.168.1.44:80". For multiple HSMs behind a load balancer, then specify the load balancer IP address and port number.	N	
Connection timeout (seconds)	Applicable only if connecting to HSM via Direct Socket Protocol. Default: 30	N	
Public Key Index	Specifies the public key index for RPIN use.	N	Y
Cache Public Keys	For faster pre-authentication performance. If enabled, the AccessMatrix server will cache the public keys from HSM. If using this option, it is needed to restart the AccessMatrix server or reload the login module by editing/saving it if the public keys of the affected indexes are changed on the HSM.	N	
Server Random Length	Server Random Length is dependent on the password length, the longer the server random the shorter the password (clear) that can be supported. **Notes:** • The maximum password length supported by HSM is 30 and the	N	

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
	corresponding maximum server random length is 21. • The actual length of server random is in hexadecimal therefore it will double the value configured when it is used by the client application for encryption. Value should be from 8 to 21. Default: 16		
Server Random generated by AccessMatrix server instead of HSM	For faster pre-authentication performance. If enabled, HSM will only be used to seed the AccessMatrix server random generator and AccessMatrix server will generate the Server Random instead of HSM.	N	
RPIN OAEP Hash Algorithm	For HSM software version below M090800E, set to "SameAsTPV" - the algorithm will be based on the one for TPV Hash Algorithm. For software version M090800E and above, specify the exact algorithm. E2EEA client library must be configured to match the algorithm configured here. Options: • "SameAsTPV" • "SHA1" • "SHA224" • "SHA256" • "SHA384" • "SHA512"	N	Y
TPV Hash Algorithm	**Important Note:** The RSA public key(s) size (configured in HSM) must be set to 2048 if intending to use SHA-2 hash algorithms (e.g. SHA224, SHA256, SHA384, SHA512). Options: • "SHA256" • "SHA1" • "SHA384" • "SHA512" • "SameAsTPV" • "SHA224" Default: SHA1	N	
KTPV (Storage Key) Index	Refers to the key index in HSM. Value should be from 1 to 99.	N	Y

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
KTPV Algorithm	Selects either "DES3"' or "AES". Default: DES3	N	
Legacy KTPV Index	Legacy KTPV Index would be used during first-time password verification upon E2EEA migration from the older version, provided the migrated password does not contain the KTPV index. Value should be from 1 to 99.	N	
Password Policies			
Password Quality Policy	Only Password Age and Password History of the policy are checked by the AccessMatrix server. It is used to validate the new password during the reset password and change password. Default: E2EEPHEFTPasswordQualityPolicy	N	
Password Expiry Policy	Password expiry policy used to check password expiry during login. Default: DefaultPasswordExpiryPolicy	N	
Extra HSM return codes for wrong password	HSM return codes to be considered as wrong password in addition to default product behavior.	N	
Reset Password Options			
Allow Manual Password Reset	Allow manual reset of password. Default: true	Y	
Allow Reset By System Assigned	Password reset via immediate HSM password generation and printing.	Y	
Allow Reset By System Assigned (Pending)	Password reset via HSM password generation and printing using PIN mailer workflow.	Y	
Allow Reset By Random Password generation	Password reset via random password generation.	Y	
Reset Password Specific Configuration			

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
Password Generating HSM Id	Id of the SafeNet Luna Payment HSM. - If connecting to HSM via Direct Socket Protocol, specify as "host:port" format, e.g. "192.168.1.44:80".	Y	These HSM settings are mandatory provided reset password by **System Assigned** or **System Assigned (Pending)** option is allowed.
Printing HSM Id	Id of the Printing HSM. - If connecting to HSM via Direct Socket Protocol, specify as "host:port" format, e.g. "192.168.1.44:80".	Y	
PPK Index	PIN Protection Key Index used by Printing HSM. Value should be from 1 to 99. Default: 1	Y	
Pin Mailer Template	Specifies the message template of the pin mailer.	Y	
Excluded Characters	Characters to be excluded for password generated by System Assign or System Assigned (pending), System Assigned PDF or System Assigned PDF (pending), e.g. "01Ol".	Y	
Password Generation Policy			
Password Length	Password length for password generation. This is the password length of System Assigned, System Assigned (pending), System Assigned PDF or System Assigned PDF (pending). Value should be from 6 to 30. Default: 6	Y	
Password Type	PIN type used by HSM password generation. This is the password type of System Assigned, System Assigned (pending), System Assigned PDF or System Assigned PDF (pending). • "0" - Numeric • "1" - Alpha-numeric (upper & lower case alpha) • "2" - Upper case alpha and numeric • "3" - Lower case alpha and numeric Default: 0	Y	
Pin Mailer Configuration			

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
PIN Mailer Work Flow Type	Specifies the PIN mailer work flow type.	Y	This setting is mandatory provided **Allow Reset by System Assigned (Pending)** option is allowed.
ATM PIN Translation			
PPK Index for ANSI format PIN	PIN Protection Key Index used for ANSI/ISO-0 PIN. Value should be from 1 to 99.	N	Y if using PIN Translation
PPK Index for PinPad (IBM-3624) format PIN	PIN Protection Key Index used for PinPad/IBM-3624 PIN. Value should be from 1 to 99.	N	
Account Number Mask	Masking account number in log file and audit record. Mask consists of '9' for digit to display and any other masking character. E.g. mask "XX*99999*XXX" and account number "123456789012" will produce "XX*45678*XXX"	N	

Create Authentication Realm

To create a new Authentication Realm, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane. Click **Create** and provide the details for the authentication realm attributes before saving the changes.

Attribute Name	Description
*Authentication Realm Id	Unique Id of authentication realm, e.g. "PE2EEAR".
*Authentication Realm Name	Name of authentication realm, e.g. "PAYMENT HSM E2EEA Realm".
Description	Description of authentication realm.
Version	Authentication realm entry version.

Attribute Name	Description
Configuration Tab	
Automatically assigned to users	Allow this authentication realm to be automatically assigned to all users.
User Lookup Module	Lookup module is used to validate user identity (i.e. user search in user store(s) or other validation methods), e.g. "SystemLookup".
First(1st) Login Module	The first login method to use for user authentication, e.g. "PAYMENT HSM Login Module (PHSMLM)".

Assign Authentication Realm to User

A user may belong to a default store or external user store. You may refer to *AccessMatrix Common Administration Guide - Manage User Stores* to know about the different types of user stores and how to configure it.

To create a new user in default store, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Provide values to the following attributes that are necessary for SafeNet PHEFT HSM E2EEA implementation before saving the changes.

Attribute Name	Description
*User Id	Unique user Id.
*User Name	Name of user.
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of user store the user belongs to, i.e. "Default Store".
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to user. Click **Add** to display the **Choose Authentication Realm** dialog box. Select the authentication realm created in previous section (e.g. "PAYMENT HSM E2EEA Realm (PE2EEAR)") and click **OK**. Both authentication realm and the login module(s) associated to it will be assigned to the user. Save the changes.	

You may refer to *AccessMatrix Common Administration Guide - Manager Users* for detailed information about users.

3.4. UAS E2EEA with Thales payShield 10k HSM Implementation

The following pages provide detailed instructions regarding the implementation of the UAS E2EEA solution with a Thales payShield 10k HSM:

- Deployment
- Configuration

3.4.1. Deployment

This section describes the installations and deployments required for UAS E2EEA FM with Thales payShield 10k HSM implementation.

Install Oracle JDK/JRE

<table>
<tr>
<td>Windows</td>
<td>
Notes:

It is assumed that the Thales payShield 10k HSM device is already installed and configured before performing the following steps in this section.
Install the supported Oracle JDK/JRE version in both UAS AccessMatrix server and HSM Client server (if both are deployed separately). Refer to AccessMatrix Supported Platforms document to know the supported JDK/JRE version.

1. Download and install the JDK executable from Oracle website.
2. Set an environment variable with the following details:
<code>Variable name: JAVA_HOME</code>
<code>Variable value: <JDKInstallationFolder> (e.g. C:\Program Files\Java\jdk1.8.0_181)</code>
3. You will also be required to add bin location of your java binaries to the systems PATH variable. Edit PATH variable under System variables and append the string ";%JAVA_HOME%\bin" in variable value. Save it.
</td>
</tr>
<tr>
<td>Linux</td>
<td>
Notes:

It is assumed that the Thales payShield 10k HSM device is already installed and configured before performing the following steps in this section.
Install the supported Oracle JDK/JRE version in both UAS AccessMatrix server and HSM Client server (if both are deployed separately). Refer to AccessMatrix Supported Platforms document to know the supported JDK/JRE version.

1. Download and install the JDK rpm file from Oracle website.
<code>rpm -ivh <jdk_package>.rpm</code>
2. Add the Java location path in JAVA_HOME. For example:
<code>export JAVA_HOME=<jdk or jre installation path></code>
<code>export PATH=$JAVA_HOME/bin:$PATH</code>
</td>
</tr>
</table>

3.4.2. Configuration

This section details the configuration steps relating to the implementation of UAS E2EEA FM with a Thales payShield 10k HSM.

Setup Steps

Before you start the implementation, it is assumed that you have completed the planning and implementation checklists, and you have understood the concept of UAS E2EEA with Thales payShield 10K HSM.

- **Upload E2EEA License**
  E2EE license is required to enable this feature, refer to Upload E2EEA License for the details.
- **Configure Thales payShield Login Module**
  Login module provides a login or authentication method to verify user login credentials. Refer to Configure Thales payShield Login Module for the configuration details.
- **Create Authentication Realm**
  Authentication Realm associates user credentials for authentication based on a lookup module and login module(s). User entries can be created into the local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). Refer to Create Authentication Realm for the details.
- **Assign Authentication Realm to User**
  User is an entity that accesses the system and may belong to a default store (local repository) or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to the user by adding it to the "Login Accounts" tab. Refer to Assign Authentication Realm to User for the details.

Upload E2EEA License

Before doing any configuration in the Admin Console, secure the server license required for E2EEA with Thales payShield 10K HSM support. Ensure that **E2EEA Base** license features are set to "Yes" to enable E2EEA with Thales payShield 10K HSM support and display the "Thales payShield Login Module" in the login module implementation list.

To load the server license:

1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
2. Select the Server Id and click **Edit**.
3. Select **Licensed Features** tab and select the first radio button. Browse for the license file.
4. Click **Upload Licence File** to upload the license file to the server.
5. Save the changes.
6. Restart the AccessMatrix server for the new license to take effect.

Configure Thales payShield Login Module

To support E2EEA with Thales payShield 10K HSM, you must create a new login module that provides Thales payShield login method for the user who may reside in either an internal or external user store.

To create a Thales payShield Login Module, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane. Click **Create** button and select the "Thales payShield Login Module" implementation from the **Choose Login Module Implementation** page. Then, click **Next** and provide necessary values to the **Configuration** tab attributes before saving the changes.

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
*Login Module Id	Unique Id of login module, e.g. "ThalesPSLM".	N/A	Y
*Login Module Name	Name of login module, e.g. "Thales payShield HSM Login Module".	N/A	Y
Description	Description of login module.	N/A	N
Version	Login module entry version.	N/A	N/A
Implementation	Type of login module implementation, i.e. "Thales payShield Login Module".	N/A	N/A
Handler	Handler of the login module, i.e. "ThalesPayshieldLoginModule".	N/A	N/A
Configuration Tab			
Login Configuration			

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
Login Policy	The login policy specifies if bad login count should be incremented and if login account should be locked upon exceeding maximum bad login count, as well as other policy controls. If no login policy is attached, then the corresponding policy controls like incrementing bad login count will not take effect.	N	
Authentication Level	Strength of authentication from 1 (lowest) to 10 (highest). Default: 1	N	
Connection			
*Thales payShield HSM Host, Port and Number of Connections	Define the connection settings to the HSM. Specify the value in "host:port:number of connections" format, e.g. 192.168.1.44:80:2. For the number of connections, consult the administrator for Thales payShield HSM. Unless set otherwise by the HSM administrator, the default number of connections should be set to "2".	N	Y
*Message Header	The value can be any length from 1 to 255 characters and it is configured at HSM installation.	N	Y
Enable TLS	Enable TLS for mutual authentication between AccessMatrix and HSM. Once enabled, the HSM server certificate must be added to the default JRE truststore or javax.net.ssl.truststore Default: false.	N	
Keystore	Path to the keystore file containing the HSM client certificate.	N	
Keystrore Passphrase	The passphrase of the keystore.	N	
Private Key Passphrase	Client private key passphrase in the keystore.	N	
Connection timeout (seconds)	Socket connection timeout value in seconds. "0" means infinite timeout.	N	

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
	Value should not be less than 0. Default: 15		
Heartbeat Interval (minutes)	Interval of heartbeat connection (in minutes) to HSM to keep the connection alive. The value must be lower than the idle timeout value configured in HSM. If the socket connection is unhealthy (temporarily disconnected), this value will also be used as the time interval for reconnecting the socket. Value should not be less than 1. Default: 5	N	
Maximum Connection Retry	Specify the maximum number of retry attempts for HSM connection. "0" means no retry. Value should not be less than 0. Default: 3	N	
Local Master Key Index	Specify the local master key to use. Empty means AccessMatrix will use the default local master key index configured in HSM. Value should be from 0 to 99.	N	
Message Trailer	Specify the message trailer to be appended at the end of the host command. AccessMatrix will check if the host command and response has the same message trailer. Value can be any length from 0 to 16 characters.	N	
Password Policies			
Password Quality Policy	Specify the password quality policy to validate the password entered by the user on the client side. **Note:** Password quality will only be enforced on the client because the HSM does not support checking and enforcing the password quality.	Y	
Password Generation Policy (The Settings Below Must Comply With the Password Policies)			

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
Password Length	Length of the generated password. This length must follow the PIN Length configuration in HSM. Value should be from 6 to 10. Default: 8	Y	
Asymmetric Cipher (affects generated key pairs in future)			
Algorithm	Default: RSA	N	
Key size	Default: 2048	N	
RSA encrypted PIN (RPIN) OAEP Hash Algorithm	Default: SHA256	N	
Number of Key Pairs	Value should be from 4 to 16. Default: 4	N	
Pin Block uses PAN/Token	Determines if the PIN block uses the PAN or token format in PIN translation operations. • "Token" - The source PIN block format is a token and destination PIN block format is a PAN. • "PAN" - The source PIN block format is a PAN. No destination PIN block is specified (as it is not required). Default: Token	N	

Next, for printing a pin mailer with a printer attached to the HSM, select the **Print Pin Mailer** tab and provide necessary values for the Pin Mailer attributes before saving the changes.

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
Print Pin Mailer tab			
Password Type	Type of characters in a generated password. For system assigned password sent to the printer, the type is always		

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
	"Mixed Case Alphanumeric". Further notes from Thales payShield Documentation: A Random Alphanumeric PIN with character set 0 - 9, A - Z and a - z. It should not contain characters “0” (ASCII 30H), “O” (ASCII 4FH), “1” (ASCII 31H) and “l” (ASCII 49H and 6CH) to avoid confusion. No digits should occur more than 4 times.		
Allow Reset By System Assigned	Password reset via immediate HSM password generation and printing. Default: false. See the next section on Configure a Printer on Thales payShield HSM.	Y	N
Thales payShield HSM Host, Port and Number of Connections	If you connect the printer to the primary HSM, no configuration is required here. Set up this configuration if you attach the printer to a non-primary HSM. Define the connection settings to the HSM with the attached printer. Specify the value in "host:port:number of connections" format, e.g. 192.168.1.44:80:2. For the number of connections, consult the administrator for Thales payShield HSM. Unless set otherwise by the HSM administrator, the default number of connections should be set to 2.	Y	N
Pin Mailer Template	The Print Format for the HSM PIN printing. The same format as Default Message Template. Example: Dear ${user.name}, Your user ID is ${user.id} and your PIN is <<E2EEPIN>>. Have a nice day!	Y	N
Message Header	If you connect the printer to the primary HSM, no configuration is required here.	Y	N

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
	Set up this configuration if you attach the printer to a non-primary HSM. The value can be any length from 1 to 255 characters and it is configured at HSM installation.		
Enable TLS	If you connect the printer to the primary HSM, no configuration is required here. Set up this configuration if you attach the printer to a non-primary HSM. Enable TLS for mutual authentication between AccessMatrix and HSM. Once enabled, the HSM server certificate must be added to the default JRE truststore or javax.net.ssl.truststore. Default: false	Y	N
Keystore	If you connect the printer to the primary HSM, no configuration is required here. Set up this configuration if you attach the printer to a non-primary HSM. Path to the keystore file containing the HSM client certificate.	Y	N
Keystore Passphrase	If you connect the printer to the primary HSM, no configuration is required here. Set up this configuration if you attach the printer to a non-primary HSM. The passphrase of the keystore. The passphrase of the keystore.	Y	N
Private Key Passphrase	If you connect the printer to the primary HSM, no configuration is required here. Set up this configuration if you attach the printer to a non-primary HSM. Client private key passphrase in the keystore.	Y	N
Connection timeout (seconds)	If you connect the printer to the primary HSM, no configuration is required here. Set up this configuration if you attach the printer to a non-primary HSM. Socket connection timeout value in seconds. "0" means infinite timeout. Value should not be less than 0. Default: 15	Y	N

Attribute Name	Description	For Password Generation Only (Y/N)	Mandatory (Y/N)
Heartbeat Interval (minutes)	If you connect the printer to the primary HSM, no configuration is required here. Set up this configuration if you attach the printer to a non-primary HSM. Interval of heartbeat connection (in minutes) to HSM to keep the connection alive. The value must be lower than the idle timeout value configured in HSM. If the socket connection is unhealthy (temporarily disconnected), this value will also be used as the time interval for reconnecting the socket. Value should not be less than 1. Default: 5	Y	N
Maximum Connection Retry	If you connect the printer to the primary HSM, no configuration is required here. Set up this configuration if you attach the printer to a non-primary HSM. Specify the maximum number of retry attempts for the HSM connection. "0" means no retry. Value should not be less than 1. Default: 3	Y	N
Local Master Key Index	If you connect the printer to the primary HSM, no configuration is required here. Set up this configuration if you attach the printer to a non-primary HSM. Specify the local master key to use. Empty means AccessMatrix will use the default local master key index configured in HSM. Value should be from 0 to 99.	Y	N
Message Trailer	If you connect the printer to the primary HSM, no configuration is required here. Set up this configuration if you attach the printer to a non-primary HSM. Specify the message trailer to be appended at the end of the host command. AccessMatrix will check if the host command and response has the same message trailer. The value can be any length from 0 to 16 characters.	Y	N

Configure a Printer on Thales payShield HSM

Refer to payShield 10k Installation and User Guide PDF document provided by Thales.

See *Appendix B.2: Configure the Printer Port* of the above-mentioned guide for the important notes below.

The payShield 10k is compatible with several types of printers:

- Serial printer (connected via a USB-to-serial converter cable),
- Parallel printer (connected via a USB-to-parallel converter cable)
- Native-USB printer

  It is recommended to use a native USB printer as it is the easiest to set up.

  Using a serial or parallel printer requires a converter cable supported by Thales payShield (preferably, the converter cable should be supplied by Thales).

Configuring the port is accomplished via entering the console command CP.

Refer to *Appendix A: Console Commands* of the same guide for details of the CP command.

> **Notes**:
>
> - CP denotes Configure Printer Port.
> - A printer must be connected to the HSM before the CP command is invoked.

Create Authentication Realm

To create a new Authentication Realm, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane. Click **Create** and provide the details for the authentication realm attributes before saving the changes.

Attribute Name	Description
*Authentication Realm Id	Unique Id of authentication realm, e.g. "PS2EEAR".
*Authentication Realm Name	Name of authentication realm, e.g. "Thales payShield HSM E2EEA Realm".
Description	Description of authentication realm.

Attribute Name	Description
Version	Authentication realm entry version.
Configuration Tab	
Automatically assigned to users	Allow this authentication realm to be automatically assigned to all users.
User Lookup Module	Lookup module is used to validate user identity (i.e. user search in user store(s) or other validation methods), e.g. "SystemLookup".
First(1st) Login Module	The first login method to use for user authentication, e.g. "Thales payShield HSM Login Module (ThalesPSLM)".

Assign Authentication Realm to User

A user may belong to a default store or an external user store. You may refer to *AccessMatrix Common Administration Guide - Manage User Stores* to know about the different types of user stores and how to configure them.

To create a new user in default store, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Provide values to the following attributes necessary for Thales payShield 10K HSM E2EEA implementation before saving the changes.

Attribute Name	Description
*User Id	Unique user Id.
*User Name	Name of user.
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of user store the user belongs to, i.e. "Default Store".
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to user. Click **Add** to display **Choose Authentication Realm** dialog box. Select the authentication realm created in previous section (e.g. "Thales payShield HSM E2EEA Realm (PE2EEAR)") and click **OK**. Both authentication realm and the login module(s) associated to it will be assigned to the user. Save the changes.	

You may refer to *AccessMatrix Common Administration Guide - Manage Users* for detailed information about users.

3.5. UAS E2EEA with Utimaco CryptoServer HSM Implementation

The following pages provide detailed instructions regarding the implementation of the UAS E2EEA solution with a Utimaco CryptoServer HSM:

- Deployment
- Configuration

3.5.1. Deployment

This section describes the installations and deployments required for UAS E2EEA FM with Utimaco CryptoServer HSM implementation.

Install Oracle JDK/JRE

<table>
<tr>
<td>Windows</td>
<td>
Notes:

It is assumed that the Utimaco CryptoServer HSM device is already installed and configured before performing the following steps in this section.
Install the supported Oracle JDK/JRE version in both UAS AccessMatrix server and HSM Client server (if both are deployed separately). Refer to AccessMatrix Supported Platforms document to know the supported JDK/JRE version.

Download and install the JDK executable from Oracle website.
Set an environment variable with the following details:
<code>Variable name: JAVA_HOME</code>
Variable value: <JDKInstallationFolder> (e.g. C:\Program Files\Java\jdk1.8.0_181)
You will also be required to add bin location of your java binaries to the systems PATH variable. Edit PATH variable under System variables and append the string ";%JAVA_HOME%\bin" in variable value. Save it.
Download the corresponding Java Cryptography Extension (JCE) Unlimited Strength Jurisdiction Policy File (e.g. jce_policy-8.zip). Extract the .jar files and copy them to <Java installation directory>\lib\security. The existing .jar files in the directory will be overwritten.

Note: Skip step 4 if you are using JDK Version 11 or above as the relevant files are bundled with these versions.
</td>
</tr>
<tr>
<td>Linux</td>
<td>
Notes:

It is assumed that the Utimaco CryptoServer HSM device is already installed and configured before performing the following steps in this section.
Install the supported Oracle JDK/JRE version in both UAS AccessMatrix server and HSM Client server (if both are deployed separately). Refer to AccessMatrix Supported Platforms document to know the supported JDK/JRE version.

Download and install the JDK rpm file from Oracle website.
<code>rpm -ivh <jdk_package>.rpm</code>
Add the Java location path in JAVA_HOME. For example:
<code>export JAVA_HOME=<jdk or jre installation path></code>
export PATH=$JAVA_HOME/bin:$PATH

</td>
</tr>
</table>

	3. Download the corresponding Java Cryptography Extension (JCE) Unlimited Strength Jurisdiction Policy File (e.g. **jcepolicy-8.zip**). Extract the .jar files and copy them to *$JAVAHOME/jre/lib/security*. The existing .jar files in the directory will be overwritten. **Note**: Skip step 3 if you are using JDK Version 11 or above as the relevant files are bundled with these versions.

Install CryptoServer Host Software

Windows	This section details the installation steps for the CryptoServer host software. The host software includes tools such as the CryptoServer Command-line Administration Tool (csadm), the CryptoServer Administration Tool (CAT), and the PKCS#11 CryptoServer Administration Tool (PKCS#11 CAT). 1. Run the **SecurityServer-<version number>.msi** file (e.g. **SecurityServer-4.45.3.0.msi**) located within the SecurityServer product CD. **Note**: The file is named **CryptoServerSetup-<version number>.exe for CryptoServer versions earlier than 4.40.0.** 2. Follow the on-screen instructions until you arrive at the **Choose Setup Type** page. 3. Select the **Complete** installation. Alternatively, select the **Custom** installation and ensure that the following features are selected: ○ PKCS#11 ○ CXI_C ○ JCE ○ EKM ○ Drivers ○ CryptoServer ○ cyberJack 4. Complete the installation by following the on-screen instructions.

Linux	This section details the installation steps for the CryptoServer host software. The host software includes tools such as the CryptoServer Command-line Administration Tool (csadm), the CryptoServer Administration Tool (CAT), and the PKCS#11 CryptoServer Administration Tool (PKCS#11 CAT).
	1. Create a */bin* directory in your user directory
	Note: Skip this step if there is already a */bin* directory on your system).
	`mkdir ~/bin`
	2. Copy the **casdm** file relevant for your operating system version (e.g. 32-bit or 64-bit) into the */bin* directory. The **casdm** file is located on the product CD at *Software/Linux/<your system version>/Administration/* .
	`cp <mount point of the product CD>/Software/Linux/<your system version>/Administration/csadm ~/bin`

	3. Add the */bin* directory to the path in the user configuration file in the shell that is being used:
	`export PATH=$PATH:~/bin`
	4. Copy the **cat.jar** file relevant for your operating system into your user directory. The **cat.jar** file is located on the product CD at *Software/Linux/<your system version>/Administration/* .
	`cp <path to the Product CD>/Software/Linux/<your system version>/Administration/cat.jar ~/`
	5. Start CAT with the following command:
	`java -jar ~/cat.jar`
	6. Copy the **p11cat.jar** file relevant for your operating system into your user directory. The **p11cat.jar** file is located on the product CD at *Software/Linux/<your system version>/Administration/* .
	`cp <path to the Product CD>/Software/Linux/<your system version>/Administration/p11cat.jar ~/`
	7. Start PKCS#11 CAT with the following command:
	`java -jar ~/p11cat.jar`

Set Up Utimaco CryptoServer LAN Device

This section introduces the steps needed to configure the CryptoServer LAN Device and connect it to the admin computer.

Configure CryptoServer LAN Device

Perform the following steps to configure the CryptoServer LAN device.

1. Log into the CryptoServer LAN device. The default login details are user: root and password: utimaco.
2. Enter the IPV4 Address of the default gateway.
3. Set up either a Dynamic or Static IP Address as required.
4. Enable SSH on the CryptoServer LAN device if it is disabled (SSH is enabled by default).
5. (Optional) Enable SSH login for the user root:
    a. Enter `vi /etc/ssh/sshd_config` into the device terminal.
    b. Change the default setting `PermitRootLogin no` to `PermitRootLogin yes`.
    c. Enter `/etc/init.d/sshd_restart` into the terminal.
    d. On the admin computer, perform SSH login by entering `ssh root@<HSMDeviceIP>` on the command line, followed by the password associated with the user "root".

Note: Refer to the Utimaco CryptoServer HSM documentation for detailed instructions on each step.

Connect CryptoServer LAN Device

Perform the following steps to connect the CryptoServer LAN device to the admin computer.

1. Launch CAT.
2. Click on **Devices** in the CAT toolbar.
3. Enter the IP address of the CryptoServer LAN in the **New Device** field.
4. Click on the **Add to List** button.
5. Click on the **Test** button to ensure that the CAT can connect to the CryptoServer LAN device.
6. Click on the **OK** button to save the settings.

Enable Utimaco CryptoServer Administration Using Smart Card or Keyfile

Smart Card

Enable Utimaco CryptoServer Administration Using Smart Card

This section introduces the steps needed to configure the Utimaco CryptoServer software to be used with smart cards.

Configure PIN Pad

Before enabling CryptoServer administration, perform the following steps to configure a compatible Utimaco PIN pad:

1. Connect the delivered PIN pad to the USB port of the admin computer.
2. Install the necessary PIN pad device drivers from the product CD if you have not yet done so. Refer to the Utimaco CryptoServer HSM documentation for detailed instructions on the installation process.
3. Launch CAT.
4. Click on **File** in the CAT menu bar.
5. Click on **Settings...** in the drop-down list and navigate to the **PIN Pad** tab in the new window.
6. Ensure that **Type** is set to **Autodetection** and **Port** is set to **USB Port**.
7. Click on the **OK** button to save the settings.

Generate New Authentication Token

Perform the following steps to generate a new authentication token.

Note: 3 smart cards are needed for this step (1 main smart card and 2 backup smart cards).

1. Launch CAT.
2. Click on **Authentication Token** in the CAT menu bar.
3. Select **Smartcard...** in the dropdown menu and navigate to the **Generate** tab.
4. Enter a key name into the **Key-Info** field.
5. Select "2048" for **RSA Key Length (bits)**.
6. Click on the **Generate...** button.
7. Follow the instructions displayed on the PIN pad - insert the main smart card into the PIN pad, followed by the 1st and 2nd backup smart cards as instructed. The default PIN is "123456".

Change Authentication Token

Perform the following steps to replace the existing authentication token with a new one.

Note: This step is only intended for administrators who wish to change their existing smart card with a new one.

1. Launch CAT.
2. Click on **Manage User** in the CAT toolbar.
3. Select the user "ADMIN".
4. Click on the **Change Token/Password...** button.
5. Select the **Smartcard Token** radio button and click the OK button to login as ADMIN to the CryptoServer.
6. Follow the instructions displayed on the PIN pad - insert the existing smart card into the PIN pad and enter the PIN. The default PIN is "123456".
7. Click on the **OK** button. This brings up the **Choose New User token** window.
8. Select the **Smartcard Token** radio button.
9. Click on the **OK** button.
10. Follow the instructions displayed on the PIN pad - insert the new smart card into the PIN pad and enter the PIN as instructed. The default PIN is "123456".

Create Master Backup Key (MBK)

Perform the following steps to create an MBK. An MBK is a 256-bit AES key used for backup, the encryption of external key storage, or the synchronization of CryptoServer in a server cluster.

Note: 3 backup smart cards are needed for this step.

1. Launch CAT.
2. Click on **Manage MBK** in the toolbar.
3. Enter a name in the **MBK Name** field.
4. Select the **m out of n** radio button.
5. Select "2" for **m (Shares)** and select "3" for **n (Shares)**.
6. Tick the **Automatic MBK Import** checkbox.

	7. Click on the **Generate...** button. This brings up the **Master Backup Key (MBK) Share Storage** window. 8. Select the **MBK Smartcard Token** radio button. 9. Click on the **OK** button. 10. Insert the first MBK smart card into the PIN pad and enter the PIN as instructed. 11. Click the **OK** button on the confirmation message. 12. Repeat steps 8-11 for the next 2 MBK smart cards. 13. Click on **Manage MBK** in the toolbar and navigate to the **Info** tab to check the details of the MBK stored in the CryptoServer. 14. Click on the **Card Info...** button to show details about the MBK share stored on the smart card currently inserted in the PIN pad.
Keyfile	## Enable Utimaco CryptoServer Administration Using Keyfile This section introduces the steps needed to configure the Utimaco CryptoServer software to be used with keyfiles. ### Generate New Authentication Token Perform the following steps to generate a new authentication token. 1. Launch CAT. 2. Click on **Authentication Token** in the menu bar. 3. Select "Keyfile" from the drop-down list and navigate to the **Generate** tab. 4. Select the **RSA** radio button. 5. Select a location on your device to save the keyfile to. 6. (Optional) Tick the **encrypt keyfile** checkbox. Encrypting your keyfile is strongly recommended. 7. Enter the admin password into the **Password** and **Confirm Password** fields. 8. Click on the **Generate** button. 9. Close the confirmation message by clicking on the **OK** button. ### Change Authentication Token Perform the following steps to replace the default authentication token with the newly-generated authentication token. 1. Launch CAT. 2. Click on **Login/Logoff** in the toolbar. This brings up the **Login/Logoff User** window. 3. Click on the **Login...** button. This brings up the ***Choose User Token for Login** window. 4. Select the **Keyfile Token** radio button. 5. Enter the location of the default keyfile **ADMIN.key**. This can be found on the product CD at *Software/<your OS>/<your system version>/Administration/key*. 6. Enter the admin password into the **Password** field. The default password is "123456". 7. Click on the **OK** button.

8. Verify that there is a green tick under the **Login Status** column on the **Login/Logoff User** window.
9. Click on **Manage User** in the toolbar. This brings up the User Management window.
10. Click on the **Change Token/Password...** button. This brings up the **Choose User Token for Login** window.
11. Repeat steps 4-7. Completing these steps brings up the **Choose new User Token** window.
12. Select the **Keyfile Token** radio button.
13. Enter the location of the of the newly-generated Keyfile.
14. Click on the **OK** button.
15. Click on **Login/Logoff** in the toolbar.
16. Click on the **Logoff All...** button.
17. Repeat steps 2-7, this time using the newly-generated keyfile to log in.

Create New User

Perform the following steps to create a new user.

1. Launch CAT.
2. Click on **Manage User** in the toolbar.
3. Click on the **Add User...** button. This brings up the the **Add User** window.
4. Fill in the **Name of New User** field with the desired username.
5. Select the appropriate radio button under **Authentication Mechanism** (e.g. **Keyfile (RSA Signature)**).
6. Click on the **OK** button. This brings up the **Choose User Token to Add a New User** window.
7. Select the **Keyfile Token** radio button.
8. Enter the location of the relevant keyfile.
9. Click on the **OK** button.
10. Perform a test login for the new user by clicking **Login/Logoff** in the toolbar and logging in as the newly-created user.

Create Master Backup Key (MBK)

Perform the following steps to create an MBK. An MBK is a 256-bit AES key used for backup, the encryption of external key storage, or the synchronization of CryptoServer in a server cluster.

1. Launch CAT.
2. Click on **Manage MBK** in the toolbar.
3. Enter a name in the **MBK Name** field.
4. Select the **m out of n** radio button.
5. Select **2** for **m (Shares)** and select **3** for **n (Shares)**.
6. Tick the **Automatic MBK Import** checkbox.
7. Click on the **Generate...** button. This brings up the **Master Backup Key (MBK) Share Storage** window.

	8. Select the **MBK File Token** radio button. 9. Click on the **OK** button. 10. Enter the location for the MBK keyfiles to be generated in within the **Key Path** field. Alternatively, use the **(...)** search button. 11. (Optional) Specify a password in the **Password** field to protect the MBK keyfile from unauthorized access. 12. Click on the **OK** button. This brings up a new window where the generation of the MBK is initiated. 13. Click the **OK** button on the confirmation message. 14. Repeat steps 8-13 for the next 2 MBK keyfiles. 15. Click on **Manage MBK** in the toolbar and navigate to the **Info** tab to check the details of the MBK stored in the CryptoServer.

Enable PKCS#11

This section introduces the steps needed for PKCS#11 implementation.

Modify Utimaco PKCS#11 Configuration File

Perform the following steps to complete the necessary configurations for PKCS#11 implementation.

1. Navigate to the folder containing the **cs_pkcs11_R3.cfg** file. By default, this is located at *C:/ProgramData/Utimaco/PKCS11_R3*.
2. Open the file.
3. Search for the CryptoServer keyword and locate the following lines:

```
Code

[CryptoServer]
# Device specifier (here: CryptoServer is CSLAN with IP
address 192.168.0.1)
Device = 192.168.0.1
```

4. Replace Device = 192.168.0.1 with Device = <port>@<ip address>, where <port> is the TCP/IP port for the HSM and <ip address> is the device's IP Address. By default, the TCP/IP port for the HSM is 288.
5. To enable logging for all PKCS#11 activities, change the following lines:

```
Code

[Global]
# Path to the logfile (name of logfile is attached by
```

```
the API)
# For unix:
#Logpath = /tmp
# For windows:
#Logpath = C:/ProgramData/Utimaco/PKCS11_R3

# Loglevel (0 = NONE; 1 = ERROR; 2 = WARNING; 3 = INFO;
4 = TRACE)
Logging = 0
# Maximum size of the logfile in bytes (file is rotated
with a backup file if full)
Logsize = 10mb
```

6. For Windows, uncomment #Logpath = C:/ProgramData/Utimaco/PKCS11_R3. For Linux, uncomment #Logpath = /tmp.

> **Note**: Ensure that the */tmp* directory exists. If it does not, create it.

7. Change Logging = 0 to the desired log level (e.g. Logging = 3).
8. Save the changes made.

Perform Generic Login

Perform the following steps to complete a generic login. This is necessary as an administrator must first log in to the CryptoServer with an authentication status of at least 20000000 before they can initialize a slot or token.

1. Launch PKCS#11 CAT.
2. Select a slot from **Slot List**.
3. Click on **Login/Logout** in the toolbar.
4. Select **Login Generic**.
5. Enter the username of the default admin user (i.e. "ADMIN") into the **User Name** field.
6. Select the desired authentication mechanism under **Authentication Token**.
7. Click on the **Login** button.
8. Follow the on-screen instructions for password or keyfile authentication; follow the instructions displayed on the PIN pad for smart card authentication.

Set Up a Security Officer

Perform the following steps to create a Security Officer (SO). The SO's main role is to initialize tokens and set normal users' access PINs.

1. Launch PKCS#11 CAT.
2. Select the relevant slot under **Slot List**.
3. Click on **Slot Management** in the toolbar.
4. Select **Init Token**.
5. (Optional) Enter a unique **Token Label** for the selected PKCS#11 slot.
6. Enter a password for the SO in the **SO PIN** field.
7. Repeat this for the **Confirm SO PIN** field.
8. Click on the **Init Token** button. Upon successfully initializing the token, a green tick appears in the **Token Init** column of the corresponding slot in the **Slot List**.
9. Click on **Login/Logout** in the toolbar.
10. Click on the **Logout All button**.

Set Up a User

Perform the following steps to create a new user.

1. Launch PKCS#11 CAT.
2. Select the relevant slot under **Slot List**.
3. Click **Login/Logout** in the toolbar.
4. Select **Login SO**.
5. Enter the password configured for the SO in the **SO PIN** field.
6. Click on the **Login** button.
7. Click **Slot Management** in the toolbar.
8. Select **Init PIN**.
9. Enter a password the for the slot's User in the **Normal User PIN** field.
10. Repeat this for the **Confirm Normal User PIN** field.
11. Click on the **Init PIN** button.

12. Repeat steps 2-11 to set up other slots as required.

Deploy E2EEA FM to Utimaco CryptoServer HSM

> **Note**: Skip this section if the E2EEA Functionality Module (FM) is already deployed to the Utimaco CryptoServer HSM.

1. Secure the compiled E2EEA FM file (this will be a **.out** file) provided by i-Sprint and copy this to a local folder.
2. Open the command prompt and enter the following to create the .key file used to sign the FM:
   `csadm GenKey=SDKSignKey.key,user`
3. Enter the following to load the generated key into the HSM:
   `csadm LogonSign=ADMIN,<keyspec> LoadAltMdlSigKey=SDKSignKey.key`
4. Enter the following to sign the compiled FM file with the key:
   `csadm Model=c50 MMCSignKey=SDKSignKey.key MakeMTC=<.out file>`
5. Enter the following to load the signed FM into the HSM:
   `csadm LogonSign=ADMIN,<keyspec> LoadFile=<.mtc file from previous step>`
6. Restart the server:
   `csadm restart`
7. Verify that the firmware is loaded:
   `csadm ListFirmware`

3.5.2. Configuration

This section details the configuration steps relating to the implementation of UAS E2EEA FM with a Utimaco CryptoServer HSM.

Setup Steps

Before you start the implementation, it is assumed that you have completed the planning and implementation checklists, and you have understood the concept of UAS E2EEA FM with a Utimaco CryptoServer HSM.

- **Create E2EEA Keys**
  A hardware security module (HSM) is a physical computing device that safeguards and manages digital keys for strong authentication and provides cryptoprocessing. Refer to Create E2EEA Keys to know the required keys for this UAS E2EEA solution.
- **Upload E2EEA License**
  Server license is used to enable general and product specific features. For E2EEA license, refer to Upload E2EEA License for details.
- **Configure E2EEA Login Policy**
  E2EEA Login Policy is a basic login policy that defines the behavior settings, precondition checks, post-event actions of login, reset password and change password operations. Refer to Configure E2EEA Login Policy for details.
- **Configure Utimaco CryptoServer E2EEA Login Module**
  Utimaco CryptoServer E2EEA Login Module provides a Utimaco CryptoServer-based E2EEA login method for the user who may reside in either an internal or external user store. Refer to Configure Utimaco CryptoServer E2EEA Login Module for details.
- **Create Authentication Realm**
  Authentication Realm associates user credentials for authentication based on a lookup module and login module(s) where user entries can be created into local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). Refer to Create Authentication Realm for details.
- **Assign Authentication Realm to User**
  User is an entity that accesses the system and may belong to a default store (local repository) or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to

the user by adding it to "Login Accounts" tab. Refer to Assign Authentication Realm to User for details.

Create E2EEA Keys

The following keys are required to be generated for E2EEA FM with SafeNet Protect Server HSM solution:

- **Public-Private Key Pair**

  The key used to encrypt the client-side PIN.

- **Secret Key**

  The key used to encrypt the stored PIN.

> **Note**: Ensure that the MBK has been generated before proceeding with the key generation steps below.

Generate Public/Private Key Pair

1. Perform the following steps to generate the public/private key pair for the initialized E2EEA token slot.
2. Click **Generate > Generate Key Pair**.
3. Click **Create Attribute List**.
4. Select "CKA_Label" from the drop-down list and click **Add**.
5. Enter the name of the public key (this will be used during login module configuration).
6. Click on the **OK** button.
7. Click **Generate**.
8. Repeat the above steps for the private key.

   > **Note**: The value of **CKA_Label** must be the same for both keys.

Generate Secret Key

Perform the following steps to generate the secret key for the initialized E2EEA token slot.

1. Launch PKCS#11 CAT.
2. Select the relevant slot under **Slot List**.
3. Click **Login/Logout** in the toolbar.

4. Select "Login User" and log in.
5. Click **Object Management*** in the toolbar. This brings up the **Object Management** page.
6. Click **Generate > Generate Key**.
7. Click **Create Attribute List**.
8. Select "CKA_Label" from the drop-down list and click **Add**.
9. Enter the name of the key (this will be used during login module configuration).
10. Click on the **OK** button.
11. Click **Generate**.

Upload E2EEA License

Before doing any configuration in the Admin Console, secure the server license required to implement UAS E2EEA FM with Utimaco CryptoServer HSM. Ensure that **E2EEA Base** license feature is set to **Yes** to enable E2EEA FM support and to display "Utimaco CryptoServer E2EEA Login Module" in the login module implementation list.

To load the server license:

1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
2. Select the Server Id and click **Edit**.
3. Select **Licensed Features** tab and select the first radio button. Browse for the license file.
4. Click **Upload Licence File** to upload the license file to the server.
5. Save the changes.
6. Restart the AccessMatrix server for the new license to take effect.

Configure E2EEA Login Policy

To create a new basic login policy, navigate to **Administration Dashboard > Security Policy > Authentication** and click **Login Policy** on the left pane. Click **Create** button and select "Basic Login Policy" implementation from the **Choose Login Policy Implementation** page. Then, click **Next** and provide the details for the login policy attributes before saving the changes.

Attribute Name	Description
*Login Policy Id	Unique login policy Id, e.g "E2EEALoginPolicy".
*Login Policy Name	Name of login policy, e.g. "E2EEALoginPolicy".
Description	Description of login policy.
Version	Login policy entry version.
Implementation	Type of login policy implementation, i.e. "Basic Login Policy".
Attributes Tab	
Login Account	
Check login account status	If set to true then login account status will be checked and will display corresponding error message if counter already reached the set bad login count. Otherwise, user will be allowed to login if correct password is entered regardless if the login account status was already disabled. Default: true
Disable login account if bad login count reaches	Sets the maximum bad login count. If bad login count reaches the specified number, then login account will be disabled. Default: 3
Allow resetting password without force password change	Allows password reset without forcing a password change. The behavior will depend on the selected option: • "Disabled" - Disallow password reset. • "Enabled in API only" - Allows password reset in API only. • "Enabled in API and Admin Console" - Allows password reset in both API and Admin Console. Default: Disabled
Allow enabling of login account without resetting password	Allows login account activation without resetting the password. Default: false **Note:** This attribute is applicable only for the **Modify Account** button which is hidden by default. This button can change the login account's status. Uncomment the Modify Account button tag in **field-definitions.xml** to display the button.
Increment bad login count for the wrong password used during change password	Specifies the action to be done if change password failed due to an incorrect password. • "Do not increment" - No action to be taken. • "Increment same bad login counter as login" - Increment the bad login count if change password failed due to an incorrect password.

Attribute Name	Description
	• "Increment separate bad login counter" - This will create or increment a separate counter if a change password failed due to an incorrect password. Default: Do not increment
Auto Unlock	
Enable Auto Unlock By Time	If enabled, then locked accounts will be unlocked after the specified time in **Auto Unlock Time Duration** attribute. Default: false
Auto Unlock Time Duration (in minutes)	Sets the time duration when auto unlock will be executed. Value should be from 1 to 1440. Default: 30
Others	
Include new password in change and reset password event	Setting this to true will include the new password in event object for successful change and reset password events. **Note:** Only enable this if required, e.g. for provisioning. Default: false

Configure Utimaco Crypto Server E2EEA Login Module

To create a new login module, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane. Click Create button and select "Utimaco Crypto Server E2EEA Login Module" implementation from the **Choose Login Module Implementation** page. Then, click **Next** and provide the details for the login module attributes before saving the changes.

Attribute	Description
*Login Module Id	Unique Id of login module, e.g. "E2EECSLM".
*Login Module Name	Name of login module, .e.g "E2EE CS Login Module".
Description	Description of login module.
Version	Login module entry version.
Implementation	Type of login module implementation, i.e. "Utimaco Crypto Server E2EEA Login Module".
Handler	Handler of the login module, i.e. "UtimacoCSE2EELoginModule".

Attribute	Description
Configuration Tab	
Login Configuration	
Login Policy	The login policy specifies if bad login count should be incremented and if login account should be locked upon exceeding maximum bad login count, as well as other policy controls. If no login policy is attached, then the corresponding policy controls like incrementing bad login count will not take effect. Example: E2EEALoginPolicy
Authentication Level	(Unused) This is an attribute of login module used for UAM product. Default: 1
HSM Basic Configuration - Require restart to take effect	
*Device	The CryptoServer HSM device address. Example: 3001@192.168.1.49 where 192.168.1.49 is the IP address and 3001 is the port of CryptoServer LAN address.
Connection timeout (seconds)	Specify the maximum time in seconds to wait before the connection establishment is aborted if the device is not responding. Default: 3
Command Timeout (seconds)	Specify the maximum time in seconds to wait for the answer from CryptoServer after sending a command. Default: 60
*HSM User PIN	This is the same user PIN you used to login to p11tool.
HSM Slot ID	The slot containing your keys for this module. Default: 0
Maximum Connections Total	Default: 2
*Public Key Label(s)	List of public key names or IDs to use to encrypt the client-side PIN. **Note:** The label of RSA private and public keys created in HSM must be equal because AccessMatrix will use the same key label for both public and private keys.
Hash Algorithm	Default: SHA256
*TPIN Key Label(s)	List of PIN encryption key names or IDs to use for encrypting the stored PIN.
Password Policies	

Attribute	Description

Password Quality Policy	The password quality policy to use.
	
	Note: If **Check password quality policy in HSM** is set to "true":
	
	• For AccessMatrix versions below 5.3.3.3304, only Password Age and Password History of the policy are checked by AccessMatrix server.
	• For AccessMatrix versions 5.3.3.3304 onwards, besides Password Age and History, the following password policy checks are also performed in the HSM:
	  ○ Min length
	  ○ Max length
	  ○ Min upper case
	  ○ Min lower case
	  ○ Min numeric
	  ○ Min alphabetic
	  ○ Min non-alphanumeric
	  ○ Allow Repeat character
	  ○ Number of consecutive repeating char allowed
	  ○ Allow sequence of alphabetic char
	  ○ Allow sequence of numeric

Check password quality policy in HSM (if supported by HSM)	Setting to "true" will check the password quality policy in HSM. Default: false
Password Expiry Policy	The password expiry policy to use, e.g. "DefaultPasswordExpiryPolicy".

Create Authentication Realm

To create a new Authentication Realm, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane. Click **Create** and provide the details for the authentication realm attributes before saving the changes.

Attribute Name	Description
*Authentication Realm Id	Unique Id of authentication realm, e.g. "E2EEACR".
*Authentication Realm Name	Name of authentication realm, e.g. "E2EEA CryptoServer Realm".
Description	Description of authentication realm.

Attribute Name	Description
Version	Authentication realm entry version.
Configuration Tab	
Automatically assigned to users	Allow this authentication realm to be automatically assigned to all users.
User Lookup Module	Lookup module is used to validate user identity (i.e. user search in user store(s) or other validation methods). Example: "SystemLookup"
First(1st) Login Module	The first login method to use for user authentication, e.g. "E2EE CS Login Module (E2EECSLM)".

Assign Authentication Realm to User

A user may belong to a default store or external user store. You may refer to *AccessMatrix Common Administration Guide - Manager User Stores* to know about the different types of user stores and how to configure it.

To create a new user in default store, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Provide values to the following attributes that are necessary for E2EEA implementation before saving the changes.

Attribute Name	Description
*User Id	Unique user Id.
*User Name	Name of user.
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of user store the user belongs to, i.e. "Default Store".
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to user. Click **Add** to display the **Choose Authentication Realm** dialog box. Select the authentication realm created in previous section (e.g. "E2EEA CryptoServer Realm (E2EECR)") and click **OK**. Both authentication realm and the login module(s) associated to it will be assigned to the user. Save the changes.	

Refer to *AccessMatrix Common Administration Guide - Manager Users* for detailed information about users.

3.6. UAS Pseudo E2EEA Implementation

The following page provides detailed instructions regarding the implementation of the UAS Pseudo-E2EEA solution:

- Configuration

3.6.1. Configuration

The following sections provide the steps to configure the AccessMatrix Universal Authentication Server (UAS) Pseudo End-to-End Encryption Authentication (E2EEA) solution.

Setup Steps

This section defines the steps relating to the implementation of UAS Pseudo E2EEA. Before you start the implementation, it is assumed that you have completed the planning and implementation checklists, and you have understood the concept of UAS Pseudo E2EEA solution.

UAS Pseudo E2EEA solution implements an end-to-end application layer encryption either without an HSM or with an HSM.

- **Upload E2EEA License**
  Server license is used to enable general and product specific features. For E2EEA license, refer to Upload E2EEA License for details.
- **Configure Pseudo E2EEA Login Policy**
  Pseudo E2EEA Login Policy is a basic login policy that defines the behavior settings, precondition checks, post-event actions of login, reset password and change password operations. Refer to Configure Pseudo E2EEA Login Policy for details.
- **Configure Pseudo E2EEA Password Expiry Policy**
  Pseudo E2EEA Password Expiry Policy governs the user's login password expiration. Refer to Configure Pseudo E2EEA Password Expiry Policy for details.
- **Configure Pseudo E2EEA Password Quality Policy**
  Pseudo E2EEA Password Quality Policy defines the user's login password quality based on its password length, password characters, etc. Refer to Configure Pseudo E2EEA Password Quality Policy for details.
- **Configure Key Provider**
  Key Provider is a key manager that defines different implementations of cryptography key provisioning. It indicates where and how the key will be created, it's either in hardware such as the Hardware Security Module (HSM), in software, or both. Refer to Configure Key Provider for details.

- **Configure Encryption Key**

  Encryption Key is used to represent the key itself, which may be software-based or HSM-based, depending on the Key Provider used. Refer to Configure Encryption Key for details.

- **Configure Pseudo E2EEA Login Module**

  Pseudo E2EEA Login Module provides a secure end-to-end encrypted authentication method, where the password is encrypted at the client's side using a public key and decrypted at AccessMatrix server with a private key. The password is verified in AccessMatrix Server. Refer to Configure Pseudo E2EEA Login Module for details.

- **Create Authentication Realm**

  Authentication Realm associates user credentials for authentication based on a lookup module and login module(s) where user entries can be created into a local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). Refer to Create Authentication Realm for details.

- **Assign Login Account to User**

  User is an entity that accesses the system and may belong to a default store (local repository) or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to the user by adding it to the "Login Accounts" tab. Refer to Assign Authentication Realm to User for details.

Upload E2EEA License

Before doing any configuration in the Admin Console, secure the server license required to implement Pseudo E2EEA. Ensure that **E2EEA Base** license feature is set to "Yes" to enable Pseudo E2EEA support and to display "Pseudo E2EEA Login Module" in the login module implementation list.

> **Note:** To enable post-quantum cryptography support within the login module, ensure that **Post-Quantum Cryptography (PQC)** license feature is set to "Yes".

To load the server license:

1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
2. Select the relevant Server Id and click **Edit**.
3. Select the **Licensed Features** tab and select the first radio button. Browse for the license file.
4. Click **Upload Licence File** to upload the license file to the server.

5. Save the changes.
6. Restart the AccessMatrix server for the new license to take effect.

Configure Pseudo E2EEA Login Policy

Perform the following steps to configure the Pseudo E2EEA login policy:

1. Navigate to **Administration Dashboard > Security Policy > Authentication > Login Policy**.
2. Click the **Create** button and select "Basic Login Policy" implementation from the **Choose Login Policy Implementation** page.
3. Click **Next**.
4. Enter a **Login Policy Id** (e.g. "PseudoE2EEALoginPolicy") and a **Login Policy Name** (e.g. "PseudoE2EEALoginPolicy").
5. Specify the values for the login policy attributes.
6. Click **Save**. A new login policy is created.

Refer to *AccessMatrix Common Administration Guide - Configure Login Policy* for detailed information about the login policy.

Configure Pseudo E2EEA Password Expiry Policy

Perform the following steps to configure the Pseudo E2EEA password expiry policy:

1. Navigate to **Administration Dashboard > Security Policy > Authentication > Password Expiry Policy**.
2. Click the **Create** button and select the "Password Expiry Policy Handler" implementation from the **Choose Password Expiry Policy Implementation** page.
3. Click **Next**.
4. Enter a **Password Expiry Policy Id** (e.g. "PseudoE2EEAExpPolicy") and a **Password Expiry Policy Name** (e.g. "PseudoE2EEAExpPolicy").
5. Specify the values for the password expiry policy attributes.
6. Click **Save**. A new password expiry policy is created.

Refer to *AccessMatrix Common Administration Guide - Configure Password Expiry Policy* for detailed information about the password expiry policy.

Configure Pseudo E2EEA Password Quality Policy

Perform the following steps to configure the Pseudo E2EEA password quality policy:

1. Navigate to **Administration Dashboard > Security Policy > Authentication > Password Quality Policy**.
2. Click **Create**.
3. Enter a **Password Quality Policy Id** (e.g. "PseudoE2EEAPwdQltyPolicy") and a **Password Quality Policy Name** (e.g. "PseudoE2EEAPwdQltyPolicy").
4. Specify the values for the password quality policy attributes.
5. Click **Save**. A new password quality policy is created.

Refer to *AccessMatrix Common Administration Guide - Configure Password Quality Policy* for detailed information about the password quality policy.

Configure Key Provider

Upon the successful setup of the AccessMatrix system, a System Key Provider called "SystemKeyProvider" will be created by default. This key provider must be used for implementing Pseudo E2EEA without HSM as it does not rely on external software or hardware like HSM. To configure this default key provider, navigate to **Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider** and select **SystemKeyProvider** from the **Key Provider Result** list.

To configure an HSM-based key provider, create a new key provider via the AccessMatrix Admin Console by following the steps below:

1. **Navigate to Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider**.
2. Click **Create** and select the "Key Provider" implementation (e.g. "PKCS11 Key Provider").
3. Click **Next**.
4. Enter a **Key Provider Id** (e.g. "PKCS11KeyProv") and a **Key Provider Name** (e.g. "PKCS11KeyProv.').
5. Specify the values for the key provider attributes.
6. Click **Save**. A new key provider is created.

Refer to *AccessMatrix Common Administration Guide - Configure Key Provider* for detailed information about the key providers and refer to *AccessMatrix Supported Platforms - HSM For Key Management* for the list of Pseudo E2EEA supported HSMs.

Configure Encryption Key

Perform the following steps to configure an encryption key:

1. Navigate to **Administration Dashboard > Security Policy > Key Management > Encryption Key**.
2. Click **Create** and select the Key Provider implementation, e.g. "SystemKeyProvider".
3. Select **Symmetric key (secret key)** as the Encryption key template.

   > **Note**: i-Sprint's Pseudo E2EEA login module requires a symmetric key.

4. Click **Next**.
5. Enter an **Encryption Key Id** (e.g. "WrapKey") and an **Encryption Key Name** (e.g. "WrapKey").
6. Under **Attributes** tab, select the "Key Wrapping**"** value for the **Purpose** attribute.
7. Specify the values for the rest of the encryption key attributes, if necessary.
8. Click **Save**. A new encryption key is created.

Refer to *AccessMatrix Common Administration Guide - Configure Encryption Key* for detailed information about the encryption key.

Configure Pseudo E2EEA Login Module

To create a Pseudo E2EEA Login Module, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane. Click the **Create** button and select the "Pseudo E2EEA Login Module" implementation from the **Choose Login Module Implementation** page. Then, click **Next** and provide the details for the login module attributes before saving the changes.

Attribute Name	Description
*Login Module Id	Unique Id of login module, e.g. "PseudoLM".
*Login Module Name	Name of login module, e.g. "Pseudo E2EEA Login Module".
Description	Description of login module.

Attribute Name	Description
Version	Login module entry version.
Implementation	Type of login module implementation, i.e. "Pseudo E2EEA Login Module".
Handler	Handler of the login module, i.e. "PseudoE2EEALoginModule".
Configuration Tab	
Login Configuration	
Login Policy	The login policy specifies if bad login count should be incremented and if login account should be locked upon exceeding maximum bad login count, as well as other policy controls. If no login policy is attached, then the corresponding policy controls like incrementing bad login count will not take effect. Example: PseudoE2EEALoginPolicy
Authentication Level	Unused. **Note**: This is an attribute of login module used for UAM product. Default: 1
Asymmetric Cipher (affects generated key pairs in future)	
Algorithm	Algorithm that will be used to encrypt data on the client-side with the public key and to decrypt it on pseudo-HSM inside AccessMatrix server with the private key. Default: RSA
Post Quantum Cryptography (PQC) Option	Set this to "true" to enable PQC. Adversaries with access to quantum computing will theoretically be able to break conventional methods of encryption that are currently used (such as RSA). By using PQC algorithms that are designed to provide encryption that is resistant to attacks by quantum computers, this threat is mitigated. Default: false
Post Quantum Cryptography (PQC) Algorithm	Specifies the PQC algorithm that will be used, if **Post Quantum Cryptography (PQC) Option** has been set to "true". These algorithms are based on the Module-Lattice-Based Key-Encapsulation Mechanism Standard. They use a key encapsulation mechanism (KEM), which is similar to asymmetric encryption, where a public key is shared between the sender and the recipient. In KEM however, the public key further undergoes the encapsulation algorithm to generate a symmetric key and a ciphertext of the symmetric key that are used as part of the encryption and decryption process. Options:

Attribute Name	Description
	• "ML_KEM_512" • "ML_KEM_768" • "ML_KEM_1024" Default: ML_KEM_512 **Note:** During preauthentication, you may include the parameter PQC to have it return PQC keys instead of RSA keys.
Number Of Key Pairs	Sets the number of key pairs to be generated. Value should be from 1 to 16. Default: 4
Key size	Size of key in bytes. Options: • "2048" • "4096" Default: 2048
*Wrapping Key	This key will be used to encrypt (wrap) asymmetric keys. It can be created at **Administration Dashboard > Security Policy > Key Management > Encryption Key**, with "Key Wrapping" selected as its purpose.
Storage Configuration	
*Storage Key	This key will be used to encrypt the PIN\password in the storage. **Note**: Use the same steps as described in Configure Encryption Key section to create an encryption key, except that "PIN Encryption", e.g. PEK1, should be selected as default for its **Purpose** attribute.
Hash Scheme	The hash algorithm that will be used to create the password hash before it is encrypted with the storage key. In addition to the salted SHA-256 and salted SHA-512 algorithms, three PBKDF2 hashing algorithms are also supported. These are NIST FIPS-140-compliant slow hashing algorithms designed to reduce the vulnerability of passwords to brute-force attacks. The work factor for PBKDF2 is implemented through an iteration count, which differs depending on the internal hashing algorithm it is used with. Options: • "Salted SHA-256" • "Salted SHA-512" • "PBKDF2 with SHA-1 and 1,300,000 iterations" • "PBKDF2 with SHA-256 and 600,000 iterations" • "PBKDF2 with SHA-512 and 210,000 iterations" Default: Salted SHA-256 **Note:** Selecting a PBKDF2 hashing algorithm will lead to slower

Attribute Name	Description
	performance. This is the intended effect, as such algorithms are meant to deter brute-force attackers by reducing the speed/ increasing the cost of cracking the hash. Options with less iterations are expected to provide better performance, with the "PBKDF2 with SHA-512 and 210,000 iterations" option being the fastest of the PBKDF2 options.
Password Policies	
Password Quality Policy	The password quality policy of this login module, e.g. "PseudoE2EEAQltyPolicy".
Password Expiry Policy	The password expiry policy of this login module, e.g. "PseudoE2EEAExpPolicy".
Reset Password Options	
Allow Reset By System Assigned	Sets this to "true" will enable password generation by the system. Default: false
Reset Password Specific Configuration	
Event Notification Policy	It is an event-based trigger for sending Email or SMS. For example, sending the generated password to log in users when Reset Password event is triggered. **Important Note:** Leaves event types to be blank in the event notification policy configuration to avoid sending an unnecessary message to the user.
Allow password to be delivered via Message Delivery without encrypted PDF	Sets this to true will allow sending clear password through message delivery channel. **Warning:** Be cautious that this practice could lead to security issues if your user's SMS or Email are compromised. Default: false
Password Generation Policy (The settings below must comply with the password policies)	
Password Length	Length of the generated password. Value should be from 6 to 30. Default: 8
Password Type	Type of characters in a generated password. Default: Uppercase Alphanumeric.

Important Note: Once the Pseudo E2EEA Login Module has been successfully created, click on the **Generate Key Pairs** button. If PQC has been enabled, the user may select the algorithm for the keys to be generated. Upon completion, the **Generate Key Pairs** dialog box will display the list of generated keys. You may click on the **View Keys**

	Status button to view the list of generated keys and their status within this login module.

Create Authentication Realm

To create a new Authentication Realm, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane. Click **Create** and provide the details for the authentication realm attributes before saving the changes.

Attribute Name	Description
*Authentication Realm Id	Unique Id of authentication realm, e.g. "PE2EEAR".
*Authentication Realm Name	Name of authentication realm, e.g. "Pseudo E2EEA Realm".
Description	Description of authentication realm.
Version	Authentication realm entry version.
Configuration Tab	
Automatically assigned to users	Allow this authentication realm to be automatically assigned to all users.
User Lookup Module	Lookup module is used to validate user identity (i.e. user search in user store(s) or other validation methods), e.g. "SystemLookup".
First(1st) Login Module	The first login method to use for user authentication, e.g. "Pseudo E2EEA Login Module (PseudoLM)".

Assign Authentication Realm to User

A user may belong to a default store or external user store. You may refer to *AccessMatrix Common Administration Guide - Manager User Stores* to know about the different types of user stores and how to configure it.

To create a new user in default store, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Provide values to the following attributes that are necessary for Pseudo E2EEA implementation before saving the changes.

Attribute Name	Description
*User Id	Unique user Id.
*User Name	Name of user.
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of user sStore the user belongs to, i.e. "Default Store".
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to the user. Click **Add** to display the **Choose Authentication Realm** dialog box. Select the authentication realm created in the previous section (e.g. "Pseudo E2EEA Realm (PE2EEAR)") and click **OK**. Both the authentication realm and the login module(s) associated with it will be assigned to the user. Save the changes.	

You may refer to *AccessMatrix Common Administration Guide - Manage Users* for detailed information about users.

3.7. UAS E2EEA Password Migration

When switching from one HSM to another, it is necessary to migrate user passwords from the E2EEA login module of the existing HSM to the E2EEA login module of the new HSM.

To facilitate the easy migration of users, you can make use of the E2EEA migrator login module. This login module seamlessly migrates the user password associated with the specified source E2EEA login module to the specified target E2EEA login module.

After performing the setup steps detailed below, when the user next attempts to log in via the authentication realm configured with the E2EEA migrator login module, their password will be migrated to the target E2EEA login module.

> **Important Note**: To use the E2EEA migrator login module, it is necessary to first obtain the latest version of the E2EE RPIN client.

Setup Steps

It is assumed that the respective source and target E2EEA login modules have already been created and configured. The user whose password is being migrated should already have an existing password associated with the source login module of the existing HSM.

- **Create E2EEA Migrator Login Module**
  The E2EEA migrator login module is used to migrate a user's password from one E2EEA login module to another E2EEA login module. Refer to Create E2EEA Migrator Login Module for details.
- **Create Authentication Realm**
  An authentication realm associates user credentials for authentication based on a lookup module and login module(s) where user entries can be created into a local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). Refer to Create Authentication Realm for details.
- **Assign Authentication Realm to User**
  A user is an entity that accesses the system and may belong to a default store (local repository) or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to the user by adding it to the **Login Accounts** tab. Refer to Assign Authentication Realm to User for details.

Create E2EEA Migrator Login Module

To create a new E2EEA migrator login module, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane. Click the **Create** button and select the "E2EEA Migrator Login Module" implementation from the **Choose Login Module Implementation** page. Then, click **Next** and perform the following steps:

1. Configure the **Login Policy** and **Authentication Level** attributes as appropriate.
2. Ensure that the **Activated** attribute is set to "true".
3. Select the relevant E2EEA login modules under the **From** and **To** attributes.
4. Save the changes.

Attribute Name	Description
Login Configuration	
Login Policy	The login policy specifies if bad login count should be incremented and if login account should be locked upon exceeding maximum bad login count, as well as other policy controls. If no login policy is attached, then the corresponding policy controls like incrementing bad login count will not take effect. Default: DefaultBasicLoginPolicy
Authentication Level	This is an attribute of the login module used for the UAM product. During authorization, a resource may require permission with a higher authentication level. For example, given a URL which requires authentication level 2, if a user is logged in with authentication level 1 and tries to access it, WSA will challenge the user to re-authenticate with a login module of a higher authentication level (at least authentication level 2 and above). Strength of authentication is from 1 (lowest) to 10 (highest). Default: 1
Activated	Setting this to "true" will enable the migration of the user password from the source login module to the target login module. Upon the completion of user password migration, set this to "false to disable further migration. Default: true
Login Module	
*From	The source E2EEA login module that the user password will be migrated from.
*To	The target E2EEA login module that the user password will be migrated to.

Create Authentication Realm

To create a new Authentication Realm, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane. Click **Create** and perform the following steps.

1. Enter an **Authentication Realm Id** (e.g. "ME2EEAR") and **Authentication Realm Name** (e.g. "Migrator E2EEA Realm").
2. Fill in the **Automatically assigned to users** and **User Lookup Module** attributes as necessary.
3. Select the E2EEA migrator login module created in the previous section under **First(1st) Login Module**.
4. Save the changes.

Attribute Name	Description
*Authentication Realm Id	Unique Id of authentication realm, e.g. "ME2EEAR".
*Authentication Realm Name	Name of authentication realm, e.g. "Migrator E2EEA Realm".
Description	Description of authentication realm.
Version	Authentication realm entry version.
Configuration Tab	
Automatically assigned to users	Allow this authentication realm to be automatically assigned to all users. Default: false
User Lookup Module	Lookup module is used to validate user identity (i.e. user search in user store(s) or other validation methods), e.g. "SystemLookup".
First(1st) Login Module	The first login method to use for user authentication, e.g. "E2EEA Migrator Login Module".

Assign Authentication Realm to User

To locate the user whose password needs to be migrated, navigate to **Operation Dashboard > User & Segment > User**. Select the relevant user and perform the following:

1. Click the **Edit** button.

2. Navigate to the **Login Accounts** tab and click the **Add** button.
3. Select the authentication realm created in the previous section (e.g. "Migrator E2EEA Realm (ME2EEAR)") and click **OK**.
4. Remove the existing authentication realm configured with the old E2EEA login module by clicking **Delete** as necessary.
5. Save the changes.

Attribute Name	Description
*User Id	Unique user Id.
*User Name	Name of user.
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of user store the user belongs to, i.e. "Default Store".
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to the user. Click **Add** to display the **Choose Authentication Realm** dialog box. Select the authentication realm created in the previous section (e.g. "Migrator E2EEA Realm (ME2EEAR)") and click **OK**. Both the authentication realm and the login module(s) associated with it will be assigned to the user. Save the changes.	

You may refer to *AccessMatrix Common Administration Guide - Manage Users* for detailed information about users.

4. UAS E2EEA Implementation Using API

The following pages provide detailed information on the various UAS E2EEA APIs:

- Pre-Authentication
- Password Stores in UAS
- Password Stored in Application
- Add-on Module: Device-based Password Login Module (Biometric Authentication)

4.1. Pre-Authentication

Pre-Authentication

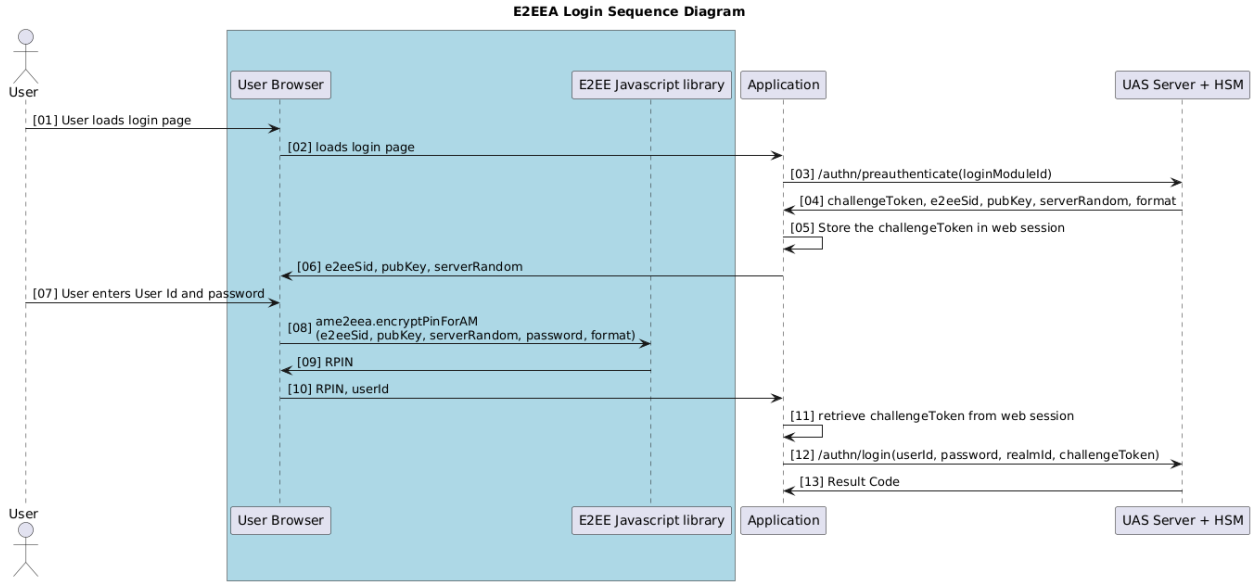
- Use /authn/preauthenticate and /authn/login to log in as a user.
- Refer to the *AccessMatrix REST API Specification* for more details.

Sequence Diagram

1. Call /authn/preauthenticate(loginModuleId) to obtain the challenge token, public key, server random number and e2ee session id from the server before login. The loginModuleId is the E2EE login module Id that the user is tagged with. Preset the E2EE login module Id and E2EE authentication realm Id in the application, if possible.
2. The application's login page checks the user password quality as needed.
3. The application's login page needs to call JavaScript function ame2eea.encryptPinForAM(e2eeSid, pubKey, serverRandom, userPassword, oaepHashAlgo) after the user has entered the password and this function will generate encrypted password RPIN for login. The clear password should be deleted after the encryption is done.

   > **Note**: For Thales payShield login module, the pin format must be specified in ame2eea.encryptPinForAM. Refer to UAS E2EEA Client Integration.

4. Retrieve the challenge token obtained from /authn/preauthenticate for login.
5. The RPIN generated is passed in as a password, and E2EE authentication realm Id is passed in as the realmId for /authn/login.
6. The /authn/login(userId, password, realmId, challengeToken) API is called to invoke the login process.



REST Sample Code

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
   "loginModuleId": "SafeNetPHEFTE2EE"
}
```

Response

JSON

```
{
  "result": {
      "challengeToken":
"00017g5fb5k8lTTNKS2T3Cpdym...DylnOn8GrmtWsZEmL7LBIFQFiBrLtAZX9t1eAs",
      "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
"00017g5fb5k8lTTNKS2T3Cpdym...DylnOn8GrmtWsZEmL7LBIFQFiBrLtAZX9t1eAs",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
        ...
      }
    ...
```

E2EE Preauthenticate API Call

```
  }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as the password value for the /authn/login API.

E2EE Login API Call

URL: /authn/login

Request

JSON

```
{
  "userId": "E2EEuser",
  "password":
"00017g5fb5k8lTTNKS2T3Cpdym...82BDB77C269F6C53590681E9FD5C72842D714E3",
  "realmId": "SafeNetE2EE",
  "challengeToken":
"00017g5fb5k8lTTNKS2T3Cpdym...DylnOn8GrmtWsZEmL7LBIFQFiBrLtAZX9t1eAs"
}
```

Response

JSON

```
{
  "result": {
    "authenticated": true,
    "sessionToken": "10001RAcj6lqgVa_RbTb-
Z4sLuFfxQ1y...jPB1fbRpwZTDBfDPg_cPD5NHSr_xves8ayVmVb-",
    ...
  }
}
```

4.2. Password Stored in UAS

Reset Password

- Use /password/reset to reset the password.
- Refer to the *AccessMatrix REST API Specification* for more details.

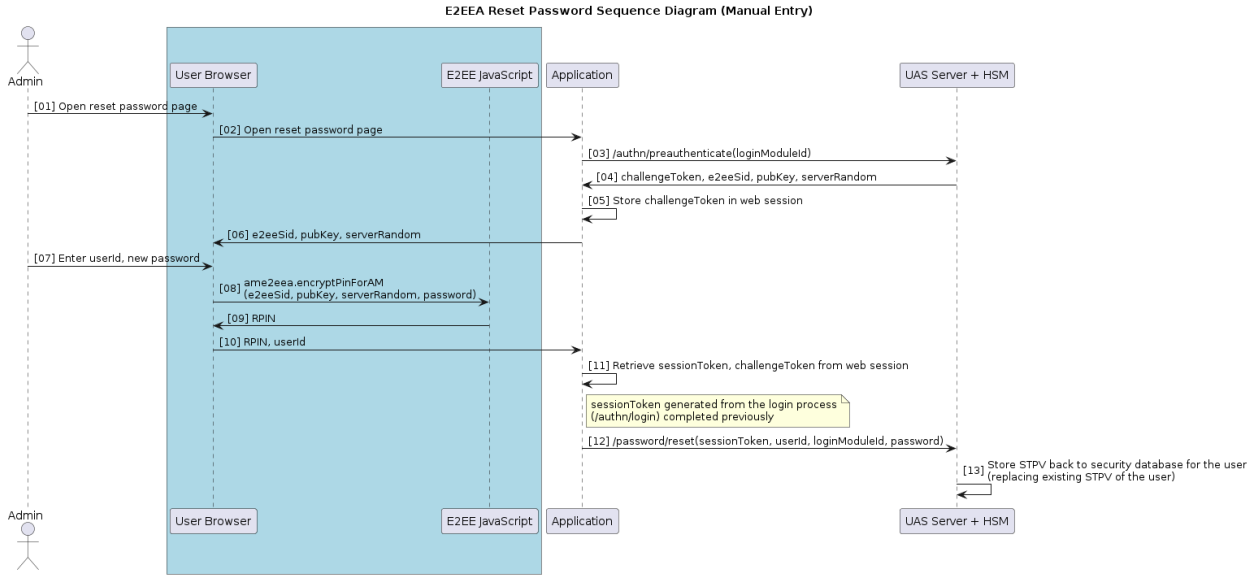
Password Generation Option For Reset Password:

- **Manual Entry**
  - It is for a manual reset password.
  - For the Manual Entry use case, it is important to check the password quality in the reset password page as UAS is unable to check the password quality due to the password being encrypted.
- **System Assigned**
  - It is for immediate password generation and printing via HSM.
- **System Assigned Pending**
  - It is for password generation and printing via HSM using pin mailer workflow.
- **System Assigned PDF**
  - It is for immediate PDF Mailer generation via HSM PDF Mailer, which leverages the existing Safenet Payment HSM for E2EEA.
- **System Assigned Pending PDF**
  - It is for PDF Mailer generation with pin mailer workflow via HSM PDF Mailer, which leverages the existing Safenet Payment HSM for E2EEA.

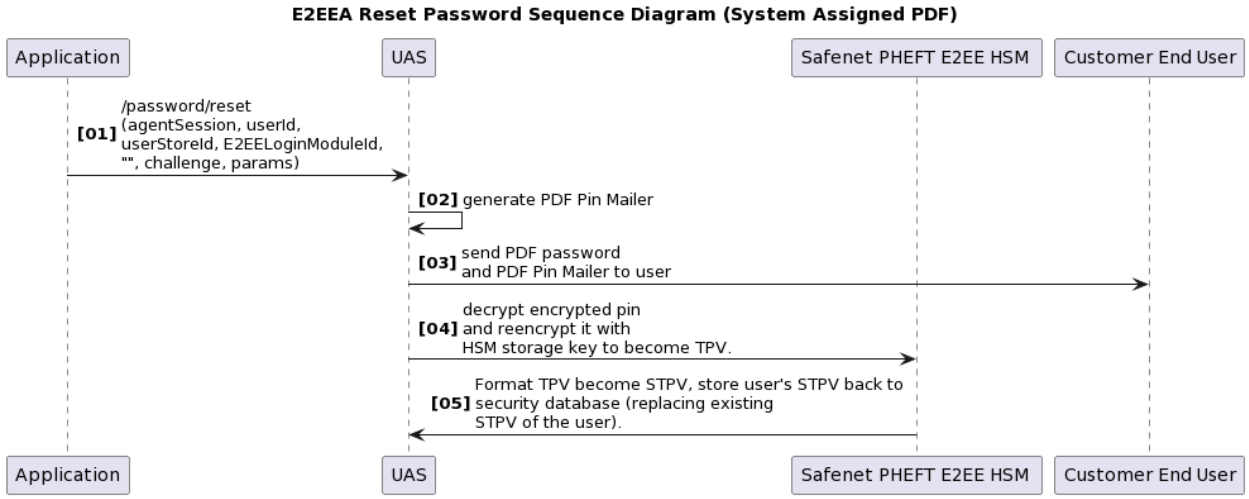
i	**Notes**: • Pseudo E2EEA only supports **Manual Entry**, **System Assigned** and **System Assigned PDF**. • Thales payShield only supports **System Assigned PDF**.

Sequence Diagram

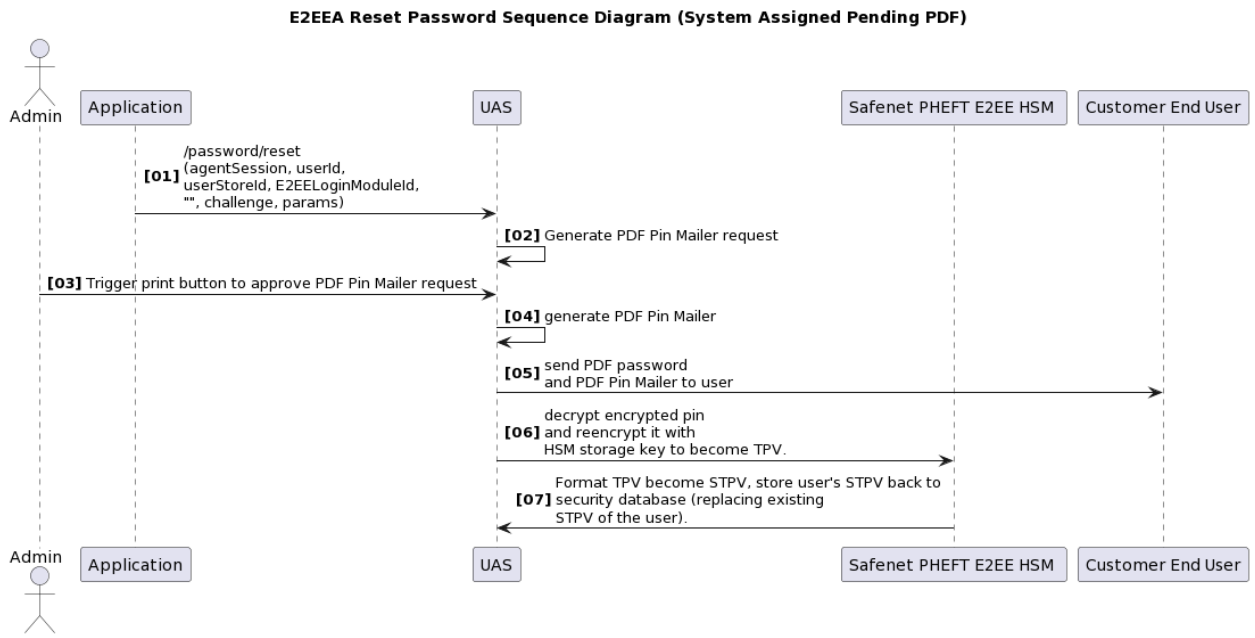
E2EEA Reset Password Sequence Diagram (Manual Entry)



E2EEA Reset Password Sequence Diagram (System Assigned PDF)



E2EEA Reset Password Sequence Diagram (System Assigned Pending PDF)



REST Sample Code

Below are the sample codes of E2EE reset password using API calls.

E2EEA Reset Password REST Sample Code (ManualEntry)

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
"loginModuleId": "SafeNetPHEFTE2EE"
  }
```

Response

JSON

```
{
"result": {
     "challengeToken":
"0001f5fb5k8lOONKS1T3Cpbyn...DylnOn8GrmtXsZKmL7LBZFQFiBrLtAZX9t1eAs",
     "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
```

E2EE Preauthenticate API Call

```
"0001f5fb5k8lOONKS1T3Cpbyn...DylnOn8GrmtXsZKmL7LBZFQFiBrLtAZX9t1eAs",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
        ...
      }
    ...
 }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as password value for the /password/reset API.

E2EEA Reset Password API Call (Manual Entry)

URL: /password/reset

Request

JSON

```
{
   "sessionToken":
"10001hBzoaQhx_ltXdxaf1mqOBCsSW5U...bZWWfs0wsTO6WozUgaynu3tCgTB1K7KTV5JN7Dp"
,
   "userId": "E2EEuser",
   "userStoreId": "DefaultStore",
   "password": "0001f5fb5k8lOONKS1T3Cpbyn...CB53C410AD2C34B857F4F44D",
   "loginModuleId": "SafeNetPHEFTE2EE",
   "challengeToken":
"0001f5fb5k8lOONKS1T3Cpbyn...DylnOn8GrmtXsZKmL7LBZFQFiBrLtAZX9t1eAs"
   "params": {
     "passwordGenerationOption": "ManualEntry",
     "remoteAddr": "1.32.181.15"
 }
}
```

Response

JSON

```
{
   "result": {
     "authenticated": true,
     "sessionToken": "10001ACY6eNCkU0-
LRt65Y0r7vx35TZE...wcxO_nzE9Tkti6DPDQwwrSsD53jpgt6X2oZlePo",
     ...
  }
}
```

E2EEA Reset Password REST Sample Code (System Assigned)

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
   "loginModuleId": "SafeNetPHEFTE2EE"
}
```

Response

JSON

```
{
   "result": {
       "challengeToken":
"00018b5fb5k8lTTNKS2T3Cpdym...DylnOn3GrmtWsZEmL7LBIFQFiBrLtAZX8t1eAs",
       "params": {
       "oaepHashAlgo": "SHA256",
       "e2eeSid":
"00018b5fb5k8lTTNKS2T3Cpdym...DylnOn3GrmtWsZEmL7LBIFQFiBrLtAZX8t1eAs",
       "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
       "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
       ...
    }
   ...
  }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as password value for the /password/reset API.

E2EEA Reset Password API Call (System Assigned)

URL: /password/reset

Request

JSON

```
{
    "sessionToken":
"10001hBzoaQhx_ltXdxaf1mqOBCsSW5U...bZWWfs0wsTO6WozUgaynu3tCgTB1K7KTV5JN7Dp"
,
    "userId": "E2EEuser",
    "userStoreId": "DefaultStore",
```

E2EE Preauthenticate API Call


```
    "password": "",
    "loginModuleId": "SafeNetPHEFTE2EE",
    "challengeToken":
"00018b5fb5k8lTTNKS2T3Cpdym...DylnOn3GrmtWsZEmL7LBIFQFiBrLtAZX8t1eAs",
    "params": {
       "passwordGenerationOption": "SystemAssigned",
       "remoteAddr": "1.32.181.15",
       "pinmailer.attr.mobile": "+6527162517",
       "pinmailer.attr.email": "e2ee.user@i-sprint.com"
   }
}
```


Response

JSON


```
{
     "result": {
       "authenticated": true,
       "sessionToken": "10001ACY6eNCkU0-
LRt65Y0r7vx35TZE...wcxO_nzE9Tkti6DPDQwwrSsD53jpgt6X2oZ1ePo",
       ...
  }
}
```


E2EEA Reset Password REST Sample Code (System Assigned Pending)

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON


```
{
     "loginModuleId": "SafeNetPHEFTE2EE"
}
```


Response

JSON


```
{
     "result": {
        "challengeToken":
"00013y5fb5k8lTTNKS2T3Cpdym...DylnOn5GrmtWsZEmL7LBIFZFiBrLtAZX9t1eCb",
        "params": {
          "oaepHashAlgo": "SHA256",
          "e2eeSid":
"00013y5fb5k8lTTNKS2T3Cpdym...DylnOn5GrmtWsZEmL7LBIFZFiBrLtAZX9t1eCb",
```

E2EE Preauthenticate API Call

```
          "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
          "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
          ...
       }
     ...
     }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as password value for the /password/reset API.

E2EEA Reset Password API Call (System Assigned Pending)

URL: /password/reset

Request

JSON

```
{
    "sessionToken":
"10001hBzoaQhx_ltXdxaf1mqOBCsSW5U...bZWWfs0wsTO6WozUgaynu3tCgTB1K7KTV5JN7Dp"
,
     "userId": "E2EEuser",
     "userStoreId": "DefaultStore",
     "password": "",
     "loginModuleId": "SafeNetPHEFTE2EE",
     "challengeToken":
"00013y5fb5k8lTTNKS2T3Cpdym...DylnOn5GrmtWsZEmL7LBIFZFiBrLtAZX9t1eCb",
     "params": {
        "passwordGenerationOption": "SystemAssignedPending",
        "remoteAddr": "1.32.181.15",
        "pinmailer.attr.mobile": "+6527162517",
        "pinmailer.attr.email": "e2ee.user@i-sprint.com"
   }
}
```

Response

JSON

```
{
      "result": {
        "authenticated": true,
        "sessionToken": "10001ACY6eNCkU0-
LRt65Y0r7vx35TZE...wcxO_nzE9Tkti6DPDQwwrSsD53jpgt6X2oZlePo"
        ...
    }
}
```

E2EEA Reset Password REST Sample Code (System Assigned PDF)

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
    "loginModuleId": "SafeNetPHEFTE2EE"
}
```

Response

JSON

```
{
    "result": {
       "challengeToken":
"00019g5fb7k8lXXNKS2T3Cpdym...DylnOn8HrmtWsZEmL7LBIFQFiBrLtAZX9t1eAv",
       "params": {
          "oaepHashAlgo": "SHA256",
          "e2eeSid":
"00019g5fb7k8lXXNKS2T3Cpdym...DylnOn8HrmtWsZEmL7LBIFQFiBrLtAZX9t1eAv",
          "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
          "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
          ...
       }
      ...
   }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as password value for the /password/reset API.

E2EEA Reset Password API Call (System Assigned PDF)

URL: /password/reset

Request

JSON

```
{
    "sessionToken":
"10001hBzoaQhx_ltXdxaf1mqOBCsSW5U...bZWWfs0wsTO6WozUgaynu3tCgTB1K7KTV5JN7Dp"
,
    "userId": "E2EEuser",
    "userStoreId": "DefaultStore",
```

E2EE Preauthenticate API Call

```
    "password": "",
    "loginModuleId": "SafeNetPHEFTE2EE",
    "challengeToken":
"00019g5fb7k8lXXNKS2T3Cpdym...DylnOn8HrmtWsZEmL7LBIFQFiBrLtAZX9t1eAv",
    "params": {
        "passwordGenerationOption": "SystemAssignedPDF",
        "remoteAddr": "1.32.181.15",
        "pinmailer.attr.mobile": "+6527162517",
        "pinmailer.attr.email": "e2ee.user@i-sprint.com"
            ...
     }
}
```

Response

JSON

```
{
    "result": {
        "authenticated": true,
        "sessionToken": "10001ACY6eNCkU0-
LRt65Y0r7vx35TZE...wcxO_nzE9Tkti6DPDQwwrSsD53jpgt6X2oZlePo",
        ...
    }
}
```

E2EEA Reset Password REST Sample Code (System Assigned Pending PDF)

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
    "loginModuleId": "SafeNetPHEFTE2EE"
}
```

Response

JSON

```
{
    "result": {
        "challengeToken":
"00017g5fb5z8lTTNUS2T3Cpgyh...DylnOm8GrmtWsZEmL7LKIFQFiBrLtAIX9t1eIa",
        "params": {
            "oaepHashAlgo": "SHA256",
            "e2eeSid":
```

E2EE Preauthenticate API Call

```
"00017g5fb5z8lTTNUS2T3Cpgyh...DylnOm8GrmtWsZEmL7LKIFQFiBrLtAIX9t1eIa",
            "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
            "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
            ...
        }
        ...
    }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as password value for the /password/reset API.

E2EEA Reset Password API Call (System Assigned PDF Pending)

URL: /password/reset

Request

JSON

```
{
    "sessionToken":
"10001hBzoaQhx_ltXdxaf1mqOBCsSW5U...bZWWfs0wsTO6WozUgaynu3tCgTB1K7KTV5JN7Dp"
,
    "userId": "E2EEuser",
    "userStoreId": "DefaultStore",
    "password": "",
    "loginModuleId": "SafeNetPHEFTE2EE",
    "challengeToken":
"00017g5fb5z8lTTNUS2T3Cpgyh...DylnOm8GrmtWsZEmL7LKIFQFiBrLtAIX9t1eIa",
    "params": {
        "passwordGenerationOption": "SystemAssignedPendingPDF",
        "remoteAddr": "1.32.181.15",
        "pinmailer.attr.mobile": "+6527162517",
        "pinmailer.attr.email": "e2ee.user@i-sprint.com"
        ...
  }
}
```

Response

JSON

```
{
    "result": {
        "authenticated": true,
        "sessionToken": "10001ACY6eNCkU0-
LRt65Y0r7vx35TZE...wcxO_nzE9Tkti6DPDQwwrSsD53jpgt6X2oZlePo"
    ...
```

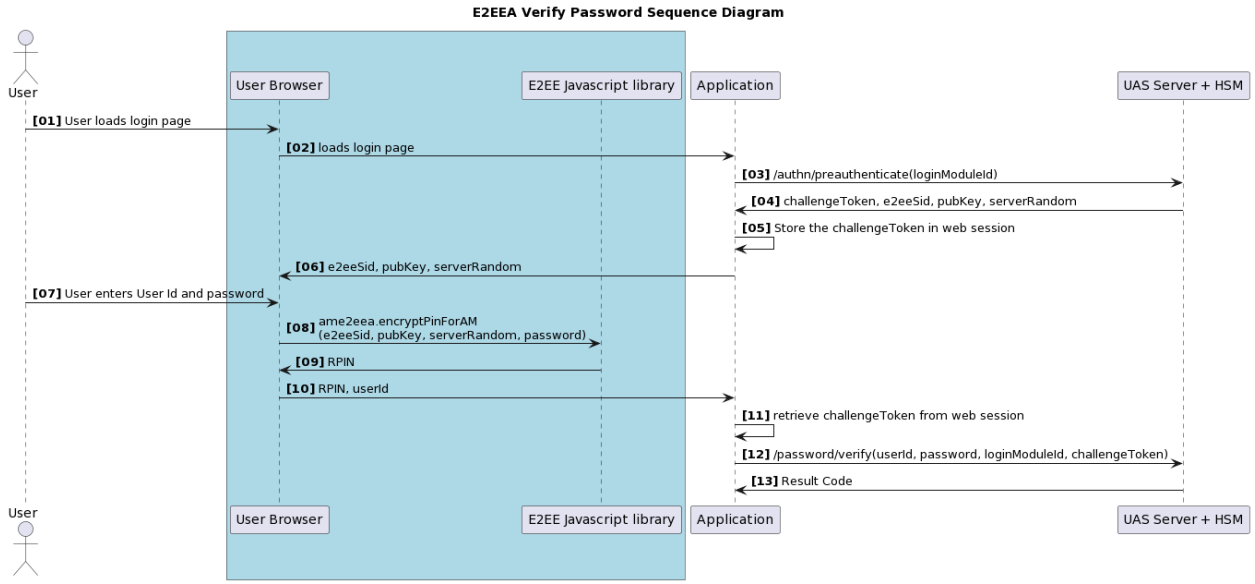
E2EE Preauthenticate API Call

```
    }
}
```

Verify Password

- Use /password/verify to verify the password.
- Refer to the *AccessMatrix REST API Specification* for more details.

Sequence Diagram



REST Sample Code

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
    "loginModuleId": "SafeNetPHEFTE2EE"
}
```

Response

E2EE Preauthenticate API Call

JSON


```
{
  "result": {
      "challengeToken":
"00014g5fc5k8lTTMJS2T3Cpasd...DylnOn8GrmfWsZEmL7LBIFQFiBrLtAZX9t1gYj",
      "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
"00014g5fc5k8lTTMJS2T3Cpasd...DylnOn8GrmfWsZEmL7LBIFQFiBrLtAZX9t1gYj",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
        ...
      }
    ...
  }
}
```


RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as password value for the /password/verify API.

E2EEA Verify Password API Call

URL: /password/verify

Request

JSON


```
{
  "sessionToken": "10001RAcj6lqgVaRbTb-
Z4sLuFfxQ1y...jPB1fbRpwZTDBfDPgcPD5NHSr_xves8ayVmVb-",
  "userId": "E2EEuser",
  "password":
"00014g5fc5k8lTTMJS2T3Cpasd...82BDB77C269F6C53590681E9FD5C72842D714E3",
  "loginModuleId": "SafeNetPHEFTE2EE",
  "challengeToken":
"00014g5fc5k8lTTMJS2T3Cpasd...DylnOn8GrmfWsZEmL7LBIFQFiBrLtAZX9t1gYj"
}
```


Response

JSON


```
{
  "result": {
    "responses": {},
    "loginModuleStates": [
      {
```

E2EE Preauthenticate API Call

```
        "authenticated": true,
        "canChangePassword": true,
        "id": "SafeNetPHEFTE2EE",
        "loginModuleHandlerClass":
"SafenetPHEFTLoginModuleHandler"
        ...
      }
    ]
    ...
  }
}
```

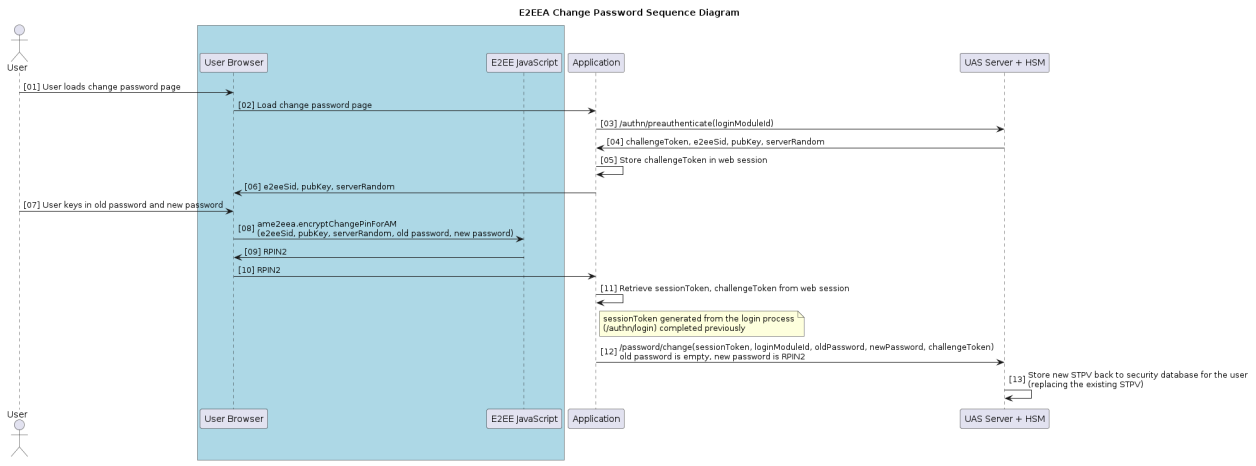
Change Password

- Use /password/change to change the password.
- Similar to the login scenario, except ame2eea.encryptChangePinForAM is called to encrypt both old password and new password. The RPIN2 value should be used as the new password when calling /password/change. The old password parameter of /password/change should be set to empty because the user's old password is encrypted as part of RPIN2. The user's actual old password will still be verified.

  > **Note**: For Thales payShield login module, the pin format and password quality policy must be specified in ame2eea.encryptChangePinForAM. Refer to UAS E2EEA Client Integration.

- Refer to the *AccessMatrix REST API Specification* for more details.

Sequence Diagram



REST Sample Code

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
   "loginModuleId": "SafeNetPHEFTE2EE"
}
```

Response

JSON

```
{
  "result": {
      "challengeToken":
"00013g6fb6k7lTTNKS2T3Clkhy...DylnOn8GrmtKnTEmL7LBIFQFiBrLtAZX9kajYh",
      "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
"00013g6fb6k7lTTNKS2T3Clkhy...DylnOn8GrmtKnTEmL7LBIFQFiBrLtAZX9kajYh",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
        ...
      }
    ...
  }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as the newPassword value for the /password/change API.

E2EEA Change Password API Call

URL: /password/change

Request

JSON

```
{
  "sessionToken": "10001RAcj6lqgVaRbTb-
Z4sLuFfxQ1y...jPB1fbRpwZTDBfDPgcPD5NHSr_xves8ayVmVb-",
  "loginModuleId": "SafeNetPHEFTE2EE",
  "oldPassword": "",
```

E2EE Preauthenticate API Call

```
  "newPassword":
"00013g6fb6k7lTTNKS2T3Clkhy...DEC443B7EF1A67CD6E624D65",
  "challengeToken":
"00013g6fb6k7lTTNKS2T3Clkhy...DylnOn8GrmtKnTEmL7LBIFQFiBrLtAZX9kajYh"
}
```

Response

JSON


```
{
  "result": {
    "authenticated": true,
    "sessionToken":
"100013f0N9IpUKcQU5nM4QeypFssQmY...SLTYn7lP2T3sXeryK6Ppp_n66csdMZVHibHzR2VCf
d",
    "loginModuleStates": [
      {
        "authenticated": true,
        "canChangePassword": true,
        "id": "SafeNetPHEFTE2EE",
        "loginModuleHandlerClass": "SafenetPHEFTLoginModuleHandler"
        ...
      }
    ],
    ...
  }
}
```


Note:

- oldPassword - this is empty as the user's old and new password are combined and encrypted as the RPIN value in the newPassword field.

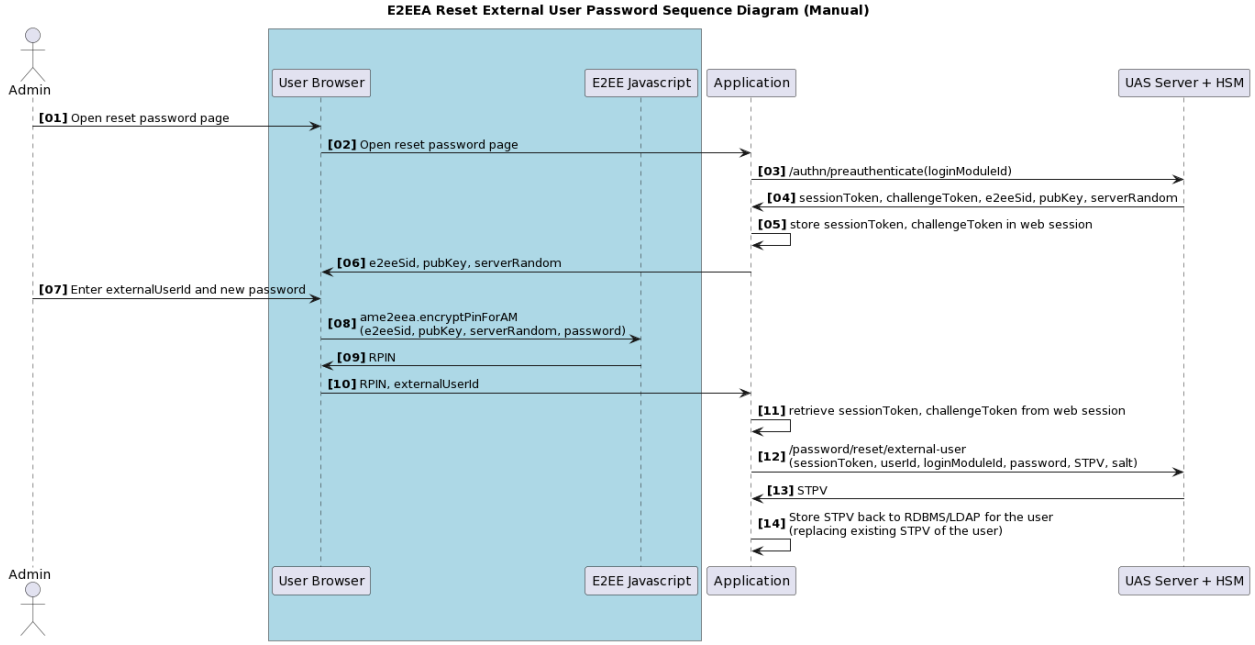
4.3. Password Stored in Application

Reset External User Password

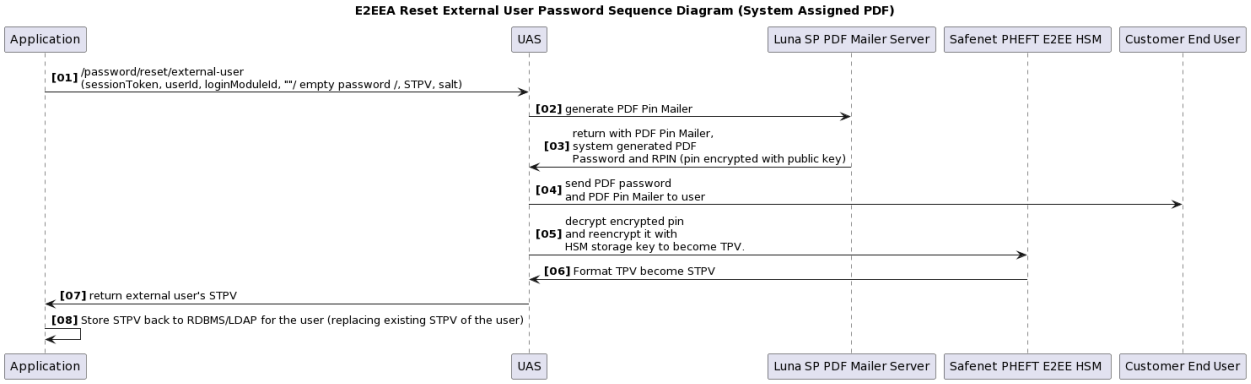
- Use /password/reset/external-user to reset an external user's password.
- The reset password flow is similar to a UAS-managed password. The only difference is that the STPV returned by UAS needs to be stored by the application.
- Refer to the *AccessMatrix REST API Specification* for more details.

Sequence Diagram

E2EEA Reset External User Password Sequence Diagram (Manual Entry)



E2EEA Reset External User Password Sequence Diagram (System Assigned PDF)



REST Sample Code

Below are the sample codes of E2EE reset external user passwords using API calls.

E2EEA Reset External User Password REST Sample Code (Manual Entry)

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
   "loginModuleId": "SafeNetPHEFTE2EE"
}
```

Response

JSON

```
{
  "result": {
      "challengeToken":
"00015g9fb5k8ljjNKS3T3Coiuy...jylnOn8GrmtWsZEmL7LBIFQFiBrLtAZX9tAeAs",
      "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
"00015g9fb5k8ljjNKS3T3Coiuy...jylnOn8GrmtWsZEmL7LBIFQFiBrLtAZX9tAeAs",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
        ...
      }
    ...
```

E2EE Preauthenticate API Call

```
  }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as the password value for the /password/reset/external-user API.

E2EEA Reset External User Password API Call (Manual Entry)

URL: /password/reset/external-user

Request

JSON

```
{
  "sessionToken":
"10001hBzoaQhx_ltXdxaf1mqOBCsSW5U...bZWWfs0wsTO6WozUgaynu3tCgTB1K7KTV5JN7Dp"
,
  "userId": "E2EEExternalUser",
  "password": "00015g9fb5k8ljjNKS3T3Coiuy...CB53C410AD2C34B857F4F44D",
  "loginModuleId": "SafeNetPHEFTE2EE",
  "challengeToken":
"00015g9fb5k8ljjNKS3T3Coiuy...jylnOn8GrmtWsZEmL7LBIFQFiBrLtAZX9tAeAs"
  "salt": "Shq6$1Op",
  "params": {
    "passwordGenerationOption": "ManualEntry",
    "remoteAddr": "1.32.181.15"
  }
}
```

Response

JSON

```
{
  "result": {
    "authenticated": true,
    "sessionToken": "10001ACY6eNCkU0-
LRt65Y0r7vx35TZE...wcxO_nzE9Tkti6DPDQwwrSsD53jpgt6X2oZlePo",
    "challenge": {
      "newSTPV":
"3:S:B6B9E9E02922C6B99F4407457EF8D9D7BC30D5A022063191E1DF506C4054230C:SHA256
"
      ...
    }
   ...
  }
}
```

E2EEA Reset External User Password REST Sample Code (System Assigned PDF)

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
   "loginModuleId": "SafeNetPHEFTE2EE"
}
```

Response

JSON

```
{
  "result": {
      "challengeToken":
"00017g5fb5n8lKKUKS2T3Jansy...DylnNn8GrmtWsZEmL7LBIFQFiBrLtAZX9tlhaA",
      "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
"00017g5fb5n8lKKUKS2T3Jansy...DylnNn8GrmtWsZEmL7LBIFQFiBrLtAZX9tlhaA",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
        ...
      }
    ...
  }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as password value for the /password/reset/external-user API.

E2EEA Reset External User Password API Call (System Assigned PDF)

URL: /password/reset/external-user

Request

JSON

```
{
  "sessionToken":
"10001hBzoaQhx_ltXdxaf1mqOBCsSW5U...bZWWfs0wsTO6WozUgaynu3tCgTB1K7KTV5JN7Dp"
,
  "userId": "E2EEExternalUser",
  "password": "",
```

E2EE Preauthenticate API Call


```
  "loginModuleId": "SafeNetPHEFTE2EE",
  "challengeToken":
"00017g5fb5n8lKKUKS2T3Jansy...DylnNn8GrmtWsZEmL7LBIFQFiBrLtAZX9tlhaA",
  "salt": "Shq6$1Op",
  "params": {
    "passwordGenerationOption": "SystemAssignedPDF",
    "remoteAddr": "1.32.181.15",
    "pinmailer.attr.userid": "E2EEExternalUser",
    "mobile": "+6527162517",
    "email": "e2ee.user@i-sprint.com"
    }
}
```


Response

JSON


```
{
  "result": {
    "authenticated": true,
    "sessionToken": "10001ACY6eNCkU0-
LRt65Y0r7vx35TZE...wcxO_nzE9Tkti6DPDQwwrSsD53jpgt6X2oZlePo",
    "challenge": {
      "newSTPV":
"3:S:B6B9E9E02922C6B99F4407457EF8D9D7BC30D5A022063191E1DF506C4054230C:SHA256
"
      ...
    }
    ...
  }
}
```

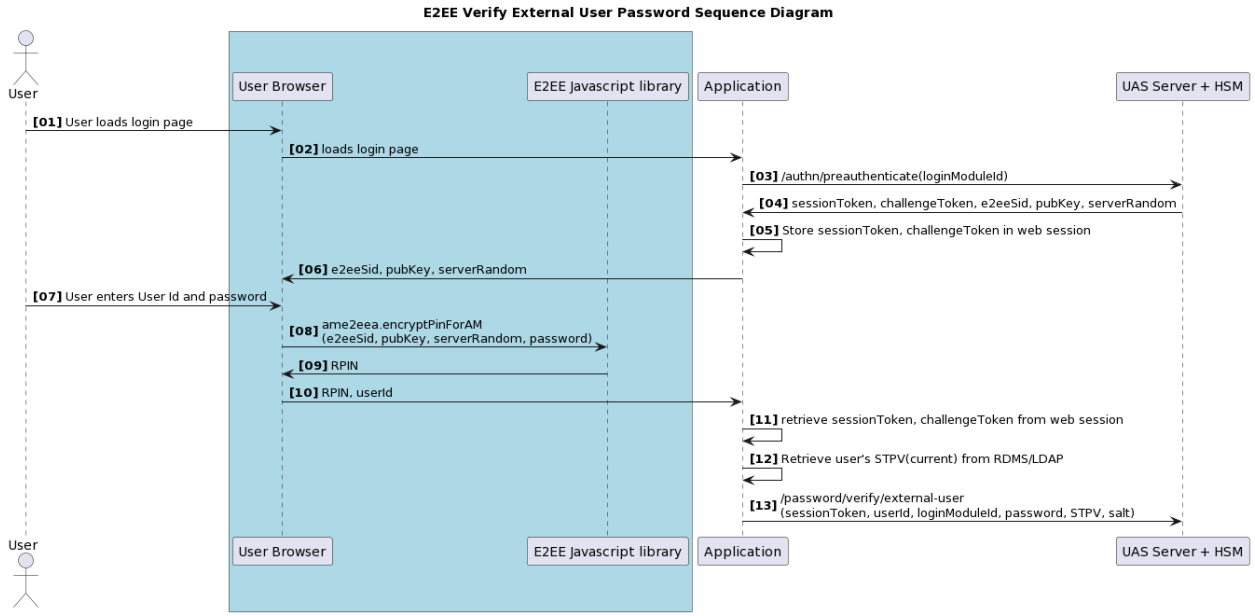

Note:

- password - empty RPIN
- pinmailer.attr.userid - used as userid in PDF Pin mailer where ${pinmailer.userid} is set in the template
- mobile - used as user email for PDF Password Template where ${event.mobile} is set in the template
- email - used as user email for PDF Delivery Template where ${event.email} is set in the template

Verify External User Password

- Use /password/verify/external-user to verify an external user's password.
- The verification flow is similar to a UAS-managed password. The only difference is that the STPV needs to be passed to UAS.
- Refer to the *AccessMatrix REST API Specification* for more details.

Sequence Diagram



REST Sample Code

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
   "loginModuleId": "SafeNetPHEFTE2EE"
}
```

Response

JSON

```
{
  "result": {
      "challengeToken":
"00017h5fb5k8lTTNKS8H6Cmwby...KylnOn8GrmtWsZEmL7LJ6JHFiBrLtAZX9t1eIs",
      "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
"00017h5fb5k8lTTNKS8H6Cmwby...KylnOn8GrmtWsZEmL7LJ6JHFiBrLtAZX9t1eIs",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
        ...
      }
```

E2EE Preauthenticate API Call

```
    ...
  }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as the password value for the /password/verify/external-user API.

E2EEA Verify External User Password API Call

URL: /password/verify/external-user

Request

JSON


```
{
  "sessionToken":
"10001hBzoaQhx_ltXdxaf1mqOBCsSW5U...bZWWfs0wsTO6WozUgaynu3tCgTB1K7KTV5JN7Dp"
,
  "userId": "E2EEExternalUser",
  "password":
"00017h5fb5k8lTTNKS8H6Cmwby...82BDB77C269F6C53590681E9FD5C72842D714E3",
  "loginModuleId": "SafeNetPHEFTE2EE",
  "challengeToken":
"00017h5fb5k8lTTNKS8H6Cmwby...KylnOn8GrmtWsZEmL7LJ6JHFiBrLtAZX9t1eIs",
  "salt": "Shq6$1Op",
  "STPV":
"3:S:8527190A254F691919EB11AF4EE5A71BEE80182D25046C1A903B219BDCA29A43:SHA256
",
  "params": {
    "remoteAddr": "1.32.181.15"
  }
}
```


Response

JSON


```
{
  "result": {
    "challenge": {...},
    "responses": {...}
    ...
  }
}
```


Change External User Password

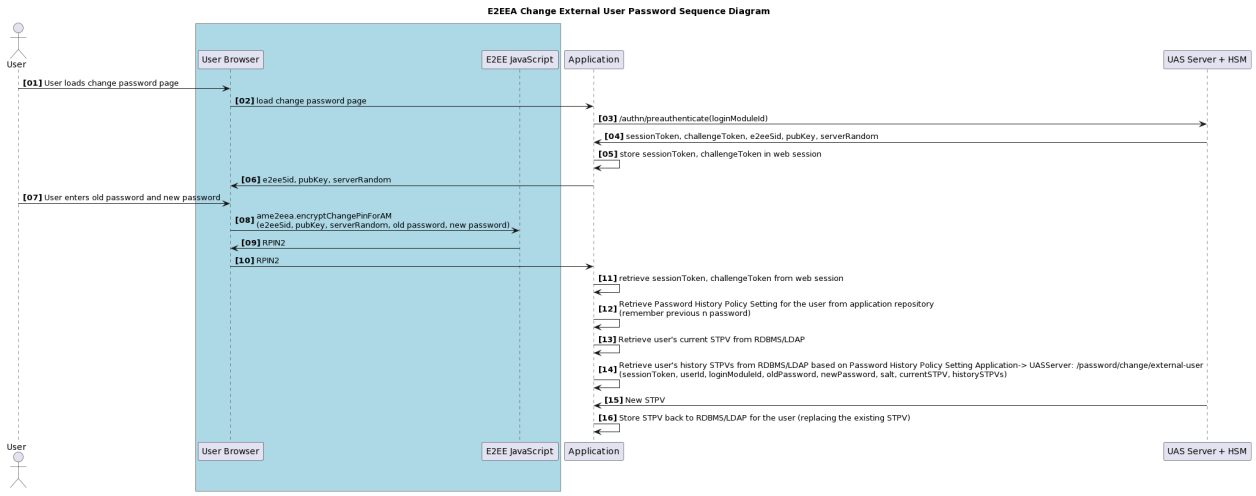
- Use /password/change/external-user to change an external user's password.

- Similar to UAS-managed password. The difference is that the current STPV and historical STPVs need to be submitted to the security server. The security server will perform the following:

1. First, verify that the old password is correct with the HSM.
2. Then, go through password histories checking by verifying the new password against all the historical TPVs (provided by the application) with the HSM (the same HSM API used in the password verification operation will be called once, against each password history).
3. Call the HSM box to perform the actual change password using the given encryption salt. The HSM box will check the new password against its password policy.
4. Return STPV, the new TPV generated ready to be stored, to the application to replace the existing STPV of the user.

- Refer to the *AccessMatrix REST API Specification* for more details.

Sequence Diagram



REST Sample Code

E2EE Preauthenticate API Call
URL: /authn/preauthenticate

Request
JSON
{
"loginModuleId": "SafeNetPHEFTE2EE"
}

E2EE Preauthenticate API Call

Response

JSON


```
{
  "result": {
      "challengeToken":
"00017g5fb5k8lKKJKS2T7Hjahs...DylnOn8GnmhWsZEmL7LBIAHFiBrLtAZX9t1eKs",
      "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
"00017g5fb5k8lKKJKS2T7Hjahs...DylnOn8GnmhWsZEmL7LBIAHFiBrLtAZX9t1eKs",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
        ...
      }
    ...
  }
}
```


RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as the newPassword value for the /password/change/external-user API.

E2EEA Change External User Password API Call

URL: /password/change/external-user

Request

JSON


```
{
  "sessionToken":
"10001hBzoaQhx_ltXdxaf1mqOBCsSW5U...bZWWfs0wsTO6WozUgaynu3tCgTB1K7KTV5JN7Dp"
,
  "userId": "E2EEExternalUser",
  "oldPassword": "",
  "newPassword": "00017g5fb5k8lKKJKS2T7Hjahs...DEC443B7EF1A67CD6E624D65",
  "loginModuleId": "SafeNetPHEFTE2EE",
  "challengeToken":
"00017g5fb5k8lKKJKS2T7Hjahs...DylnOn8GnmhWsZEmL7LBIAHFiBrLtAZX9t1eKs",
  "salt": "Shq6$1Op",
  "currentSTPV":
"3:S:B6B9E9E02922C6B99F4407457EF8D9D7BC30D5A022063191E1DF506C4054230C:SHA256
",
  "historicalSTPVs":
["3:S:B6B9E9E02922C6B99F4407457EF8D9D7BC30D5A022063191E1DF506C4054230C:SHA25
6","..."],
  "params": {
    "remoteAddr": "1.32.181.15"
```

E2EE Preauthenticate API Call

```
   }
}
```

Response

JSON


```
{
  "result": {
    "challenge": {
      "params": {
        "newSTPV":
"3:S:1B1CC2C0A9A6D5602D9445E9A402359BC4E6DB30548AC2B7B87FE2658C34518B:SHA256
"
      }
    }
    ...
  }
}
```


Note:

- oldPassword - this is empty as the user's old and new password are combined and encrypted as RPIN value in the newPassword field

4.4. Add-on Module: Device-based Password Login Module (Biometric Authentication)

Nowadays, smartphone vendors commonly provide local device biometric verification. Applications can use this feature to store/derive a password from the device secure storage and authenticate to UAS by using the (E2EEA) Device-Based Password Login Module.

The following examples assume the following:

1. A Device-based Password Login Module is created in the Admin Console and assigned to the users.
   Refer to *AccessMatrix Common Administration Guide - Device-based Password Login Module* for more information on the login module.
2. E2EEA Client Library is used on the mobile side.

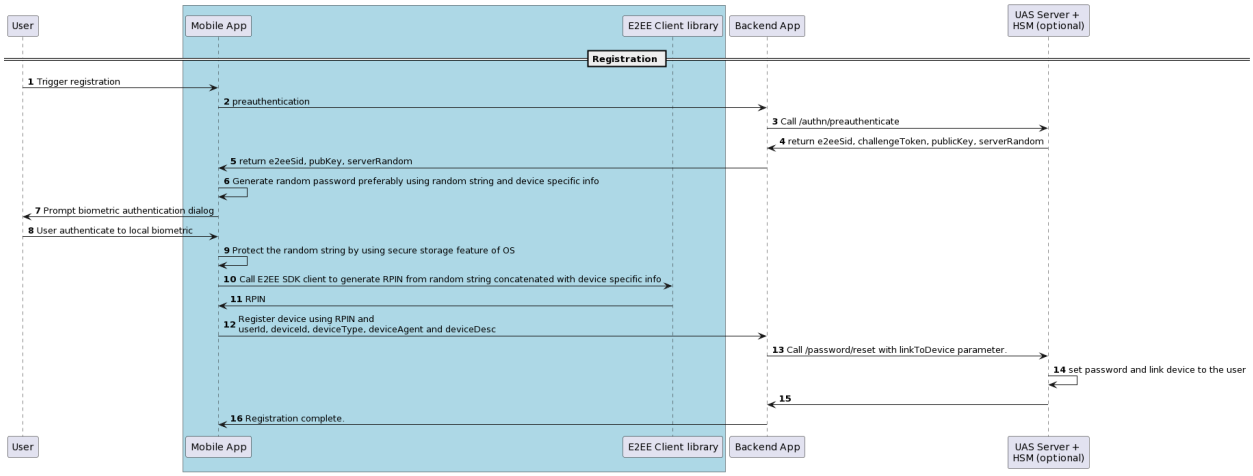
Device Registration and Password Provisioning

- The user triggers the registration process.
- The backend application calls the API **/authn/preauthenticate** passing in the **loginModuleId** to obtain the challenge token, public key, server random number and e2ee session id from the server.
- Upon receiving **e2eeSid**, **public key**, and **serverRandom**, the mobile app generates a random password.
- The mobile app uses i-Sprint E2EE Client Library to encrypt the random password with the public key and form an RPIN (public key encrypted password with special format).
- Backend application calls the API **/password/reset** with an additional parameter **linkToDevice**, specifying the device information.

> **Notes**: Some best practices for Android:
>
> - Use secure random generator, e.g. SecureRandom class.
> - Optionally, the mobile app can combine the random password with the device id.
> - Store the password securely by:
>   - Use Android KeyStore instead of KeyChain to ensure that the stored credential is only accessible by your mobile app.
>   - Encrypt the password with AES algorithm in GCM mode, with AES key generated by using KeyGenerator from AndroidKeyStore.

- Depending on the Android API level (i.e. if it is greater than 28), you can set the key store to be backed by a hardware security module (HSM) in Android. According to the Android KeyStore documentation, this will make the AES key to be protected by the HSM master key. HSM is a special hardware environment with a special chip inside the Android phone that does cryptographic operations securely, where the master key never leaves the HSM. Therefore, the master key is unique for each device, protected in the HSM. Thus it will bind the password to the device securely.
- We recommend for a mobile developer follows: OWASP Mobile Security Testing Guide.

Sequence Diagram



REST Sample Code

Below is a sample code on how to register a device using API calls.

Device Registration and Password Provisioning API Call
URL: /authn/preauthenticate
Request JSON `{ "loginModuleId": "DevBasedPwdLoginModule" }`
Response

Device Registration and Password Provisioning API Call

JSON


```
{
  "result": {
    "challengeToken":
"00017g5hb8k8lLKAYS2T3Cahst...DlajOn8GrmtWsZEmL7LBIFQFiBrLtAZX9t1mAy",
      "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
"00017g5hb8k8lLKAYS2T3Cahst...DlajOn8GrmtWsZEmL7LBIFQFiBrLtAZX9t1mAy",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
        ...
      }
    ...
  }
}
```


RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as the password value for the /password/reset API.

Password Reset with parameters

URL: /password/reset

Request

JSON


```
{
  "sessionToken":
"10001hBzoaQhx_ltXdxaf1mqOBCsSW5U...bZWWfs0wsTO6WozUgaynu3tCgTB1K7KTV5JN7Dp"
,
  "userId": "E2EEuser",
  "userStoreId": "DefaultStore",
  "password": "00017g5hb8k8lLKAYS2T3Cahst...CB53C410AD2C34B857F4F44D",
  "loginModuleId": "DevBasedPwdLoginModule",
  "challengeToken":
"00017g5hb8k8lLKAYS2T3Cahst...DlajOn8GrmtWsZEmL7LBIFQFiBrLtAZX9t1mAy",
  "params": {
    "linkToDevice":{
      "id": "xiaomi-123456781",
      "type": "Android",
      "agent": "Andriod 8.0",
      "desc": "A description"
    }
  }
}
```

Device Registration and Password Provisioning API Call

Response

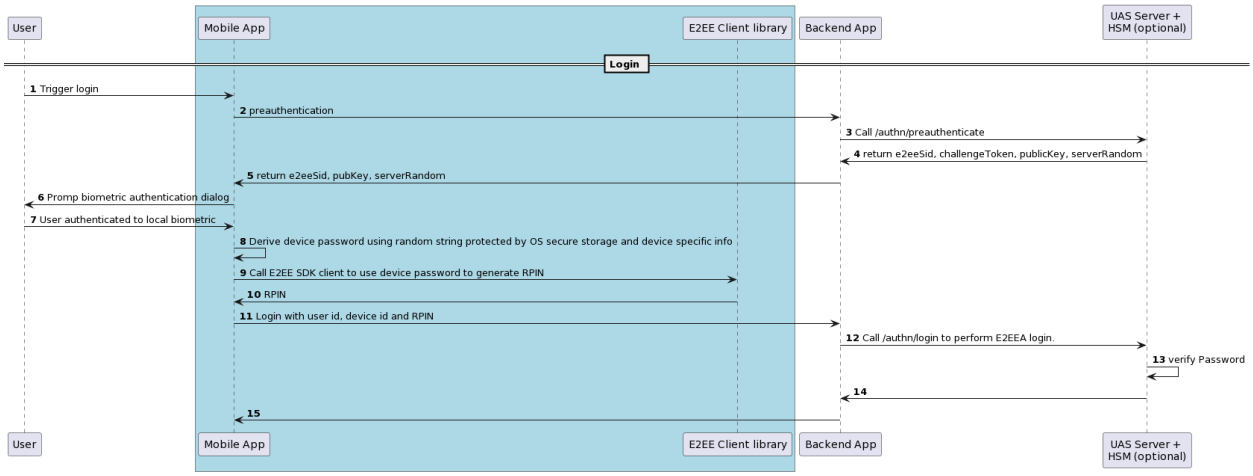
JSON

```
{
  "result": {
    "authenticated": true,
    "sessionToken": "10001ACY6eNCkU0-
LRt65Y0r7vx35TZE...wcxO_nzE9Tkti6DPDQwwrSsD53jpgt6X2oZlePo",
     ...
   }
}
```

Password Verification

- After verifying the user's local biometric, the mobile app decrypts the locally stored password.
- It also generates the device id and optionally combines it with the password.
- The mobile app forms the RPIN by calling the E2EE Client Library.
- The mobile app calls backend application API, passing in RPIN and device id.
- Backend application calls the AccessMatrix API /authn/login/.

Sequence Diagram



REST Sample Code

Password Verification API Call
URL: /authn/preauthenticate

Password Verification API Call

Request

JSON

```
{
   "loginModuleId": "DevBasedPwdLoginModule"
}
```

Response

JSON

```
{
  "result": {
    "challengeToken":
"00017g5aj6Y8lTTNKS2T3Caiwh...Dylaj78GrmtWsZEmL7LBIFAjstrLtAZX9t1eAs",
      "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
"00017g5aj6Y8lTTNKS2T3Caiwh...Dylaj78GrmtWsZEmL7LBIFAjstrLtAZX9t1eAs",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
        ...
      }
    ...
  }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration.
RPIN is passed in as the password value for the /authn/login API.

E2EE Login API Call

URL: /authn/login

Request

JSON

```
{
  "userId": "E2EEuser",
  "password":
"00017g5aj6Y8lTTNKS2T3Caiwh...82BDB77C269F6C53590681E9FD5C72842D714E3",
  "realmId": "DevBased",
  "challengeToken":
"00017g5aj6Y8lTTNKS2T3Caiwh...Dylaj78GrmtWsZEmL7LBIFAjstrLtAZX9t1eAs"
  "params" : {
    "deviceId": "xiaomi-123456781"
  }
}
```

Password Verification API Call

Response

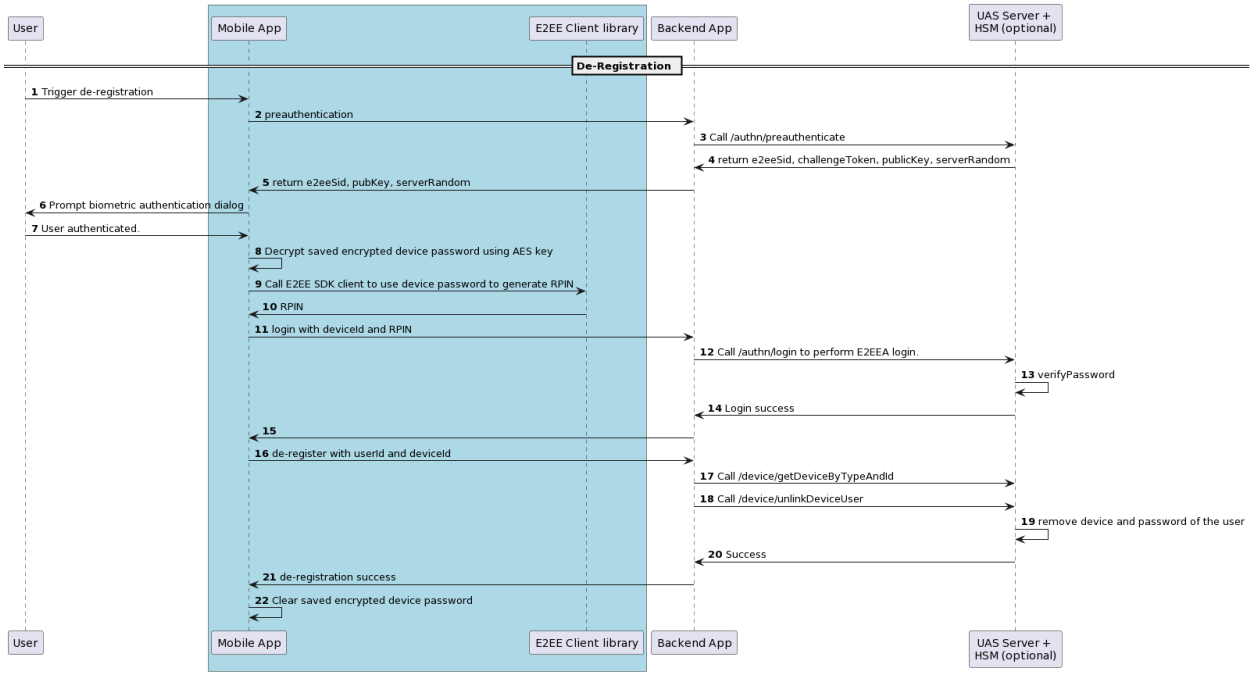
JSON


```
{
  "result": {
    "authenticated": true,
    "sessionToken": "10001ABhj6lqgVaRbTb-
Z4sLuFfxYjk...jPB1fbRpwZTDBfDPgcPD5NHSr_kaj7ayVmBh-",
    ...
  }
}
```


Device De-registration

1. After verifying the user's local biometric, the mobile app decrypts the locally stored password.
2. It also generates the device id and optionally combines it with the password.
3. The mobile app forms the RPIN by calling the E2EE Client Library.
4. The mobile app calls backend application API, passing in RPIN and device id.
5. The backend application verifies the password and device id first by calling AccessMatrix API /authn/login/.
6. The backend application resolves deviceUUID by calling AccessMatrix API /device/getDeviceByTypeAndId.
7. If verification is successful, the backend application calls AccessMatrix API /device/unlinkDeviceUser/.

Sequence Diagram



REST Sample Code

E2EE Preauthenticate API Call

URL: /authn/preauthenticate

Request

JSON

```
{
   "loginModuleId": "DevBasedPwdLoginModule"
}
```

Response

JSON

```
{
  "result": {
    "challengeToken":
"00017g8Jhtk8lTTNKS2T3Clqny...DylnOn8GrmtWsZEmU7GBIFQFiBrLtAZX9t1mYt",
      "params": {
        "oaepHashAlgo": "SHA256",
        "e2eeSid":
"00017g8Jhtk8lTTNKS2T3Clqny...DylnOn8GrmtWsZEmU7GBIFQFiBrLtAZX9t1mYt",
        "serverRandom": "CB8DB3DF6E0087774012EC79C4A9C6A8",
        "pubKey":
"9E41BB8486284B1A90CDAF669DA8A9100A8F...000000000000010001",
```

E2EE Preauthenticate API Call

```
        ...
      }
    ...
  }
}
```

RPIN Generation

Refer to UAS E2EEA Client Integration
RPIN is passed in as the password value for the /authn/login API.

E2EE Login API Call

URL: /authn/login

Request

JSON

```
{
  "userId": "E2EEuser",
  "password":
"00017g8Jhtk8lTTNKS2T3Clqny...82BDB77C269F6C53590681E9FD5C72842D714E3",
  "realmId": "DevBased",
  "challengeToken":
"00017g8Jhtk8lTTNKS2T3Clqny...DylnOn8GrmtWsZEmU7GBIFQFiBrLtAZX9t1mYt",
  "params" : {
    "deviceId": "xiaomi-123456781"
  }
}
```

Response

JSON

```
{
  "result": {
    "authenticated": true,
    "sessionToken": "10001JNyj6luyVbRbTb-
Z4sKhFfxPlt...jMNlfbRpwZTDBfLkucNJ7NHSr_kahs8ayVmKu-",
    ...
  }
}
```

Get deviceUUID used for Device de-registration

URL: /device/getDeviceByTypeAndId

Request

JSON

E2EE Preauthenticate API Call

```
{
  "sessionToken": "10001JNyj6luyVbRbTb-
Z4sKhFfxPlt...jMNlfbRpwZTDBfLkucNJ7NHSr_kahs8ayVmKu-",
  "type" : "Android",
  "id": "xiaomi-123456781",
}
```

Response

JSON

```
{
    "result": {
        "UUID": "eKhpocD_PAFW2MWk",
        "status": "Activated",
        "type": "Android",
        "id": "xiaomi-123456781",
        ...
    },
    "amProcessingTimeMillis": 386
}
```

Device De-registration API Call

URL: /device/unlinkDeviceUser

Request

JSON

```
{
  "sessionToken": "10001JNyj6luyVbRbTb-
Z4sKhFfxPlt...jMNlfbRpwZTDBfLkucNJ7NHSr_kahs8ayVmKu-",
  "userId" : "E2EEuser",
  "deviceUUID": "eKhpocD_PAFW2MWk",
  "params" : {
              "loginModuleId": "DevBasedPwdLoginModule"
        }
}
```

Response

JSON

```
{
    "result": "Success"
}
```

5. UAS E2EEA Client Integration

E2EEA client library can be loaded or integrated with different client channels (e.g. Web browser, iPhone application, Android application) to protect user's clear password. Having it encrypted using RSA public key to produce encrypted pin block (also known as RPIN (RSA encrypted PIN)).

Below are the required E2EEA client libraries for each application:

- **JavaScript**
  - **ame2eea.js**
- **Java**
  - **e2eeAndroid.jar**
  - Minimum API level is **android:minSdkVersion:3.0**
- **iOS**
  - **E2EEA.h**
  - **AxMxCrypto.h**
  - **libE2EEA.a**

Note: Pseudo E2EEA uses the same client library.

Javascript

Reset

Encrypt password for reset password (JavaScript) for Thales payShield login module

JavaScript

```
<script language="javascript" src="ame2eea.js"></script>
<script>
function onFormSubmit(form, userId, password, RPIN) {
  // Additional validation if required
  if (password.value.length < 4 || password.value.length >
14){
    alert("The PIN is too short or too long");
    return false;
  }

  // Encryption parameters
  var e2eeSid = '<%=_e2EESid%>';
  var pubKey = '<%=_e2EEPublicKey%>';
  var randomNumber = '<%=_e2EERandomNum%>';
  var oaepHashAlgo = 'SHA-1';
```

Encrypt password for reset password (JavaScript) for Thales payShield login module

```
  var pinFormat = 'ISO-0';

  // Encrypt (get RPIN)
  // ame2eea.encryptPinForAM(e2eeSid, publicKey,
randomNumber, pin, oaepHashAlgo, format)
  RPIN.value = ame2eea.encryptPinForAM(e2eeSid, pubKey,
randomNumber, password.value, oaepHashAlgo, pinFormat);
  password.value = ""; // clear unencrypted pin

  return true;
}
</script>
```

Encrypt password reset password (JavaScript) for other login module

JavaScript

```
<script language="javascript" src="ame2eea.js"></script>
<script>
function onFormSubmit(form, userId, password, RPIN) {
  // Additional validation if required
  if (password.value.length < 4 || password.value.length >
14){
    alert("The PIN is too short or too long");
    return false;
  }

  // Encryption parameters
  var e2eeSid = '<%=_e2EESid%>';
  var pubKey = '<%=_e2EEPublicKey%>';
  var randomNumber = '<%=_e2EERandomNum%>';
  var oaepHashAlgo = 'SHA-1';

  // Encrypt (get RPIN)
  // ame2eea.encryptPinForAM(e2eeSid, publicKey,
randomNumber, pin, oaepHashAlgo)
  RPIN.value = ame2eea.encryptPinForAM(e2eeSid, pubKey,
randomNumber, password.value, oaepHashAlgo);
  password.value = ""; // clear unencrypted pin

  return true;
}
</script>
```

Login

Encrypt password for login (JavaScript) for Thales payShield login module

Encrypt password for login (JavaScript) for Thales payShield login module

JavaScript

```
<script language="javascript" src="ame2eea.js"></script>
<script>
function onFormSubmit(form, userId, password, RPIN) {
  // Additional validation if required
  if (password.value.length < 4 || password.value.length >
14){
    alert("The PIN is too short or too long");
    return false;
  }

  // Encryption parameters
  var e2eeSid = '<%=_e2EESid%>';
  var pubKey = '<%=_e2EEPublicKey%>';
  var randomNumber = '<%=_e2EERandomNum%>';
  var oaepHashAlgo = 'SHA-1';
  var pinFormat = 'NONE-SHORT';

  // Encrypt (get RPIN)
  // ame2eea.encryptPinForAM(e2eeSid, publicKey,
randomNumber, pin, oaepHashAlgo, format)
  RPIN.value = ame2eea.encryptPinForAM(e2eeSid, pubKey,
randomNumber, password.value, oaepHashAlgo, pinFormat);
  password.value = ""; // clear unencrypted pin

  return true;
}
</script>
```

Encrypt password for login (JavaScript) for other login module

JavaScript

```
<script language="javascript" src="ame2eea.js"></script>
<script>
function onFormSubmit(form, userId, password, RPIN) {
  // Additional validation if required
  if (password.value.length < 4 || password.value.length >
14){
    alert("The PIN is too short or too long");
    return false;
  }

  // Encryption parameters
  var e2eeSid = '<%=_e2EESid%>';
  var pubKey = '<%=_e2EEPublicKey%>';
  var randomNumber = '<%=_e2EERandomNum%>';
  var oaepHashAlgo = 'SHA-1';

  // Encrypt (get RPIN)
  // ame2eea.encryptPinForAM(e2eeSid, publicKey,
```

Encrypt password for login (JavaScript) for other login module

```
randomNumber, pin, oaepHashAlgo)
  RPIN.value = ame2eea.encryptPinForAM(e2eeSid, pubKey,
randomNumber, password.value, oaepHashAlgo);
  password.value = ""; // clear unencrypted pin

  return true;
}
</script>
```

Change Password

Encrypt password for change password (JavaScript) for Thales payShield login module

JavaScript

```
<script language="javascript" src="ame2eea.js"></script>
<script>
HashMap parms = new HashMap();
  function onFormSubmit(form, flduserid, fldclearpin1,
fldclearpin2, fldclearpin3, fldoldpwd, fldnewpwd) {
    // Additional validation if required
    if (fldclearpin1.value == fldclearpin2.value) {
      alert("New password must be different from old
password");
      return false;
    }

    if (fldclearpin2.value != fldclearpin3.value) {
      alert("New password not match with retype new
password");
      return false;
    }

    if (fldclearpin1.value.length < 4 ||
fldclearpin1.value.length > 14 || fldclearpin2.value.length
< 4
      || fldclearpin2.value.length > 14) {
      alert ('The old PIN or new PIN is too short or too
long');
      return false;
    }

    var regex=/^[0-9A-Za-z]+$/;
    if (!regex.test(fldclearpin2.value)) {
      alert("The new PIN contains illegal character. Please
use alphanumeric ONLY.");
      return false;
    }

    // Encryption parameters
    var e2eeSid = '<%= e2EESid%>';
```

Encrypt password for change password (JavaScript) for Thales payShield login module

```
    var pubKey = '<%=_e2EEPublicKey%>';
    var randomNumber = '<%=_e2EERandomNum%>';
    var passwordQualityPolicy =
'<%=passwordQualityPolicy%>';
    var oaepHashAlgo = 'SHA-1';
    var pinFormat = 'NONE-SHORT';

    // Encrypt (get RPIN)
    // ame2eea.encryptChangePinForAM(e2eeSid, publicKey,
randomNumber, oldPin, newPin, oaepHashAlgo, format,
otherParams)
    var otherParams = {};
    otherParams.passwordQualityPolicy =
passwordQualityPolicy;
    otherParams.userId = flduserid.value;
    fldnewpwd.value = ame2eea.encryptChangePinForAM(e2eeSid,
pubKey, randomNumber, fldclearpin1.value,
    fldclearpin2.value, oaepHashAlgo, pinFormat,
otherParams);
    fldoldpwd.value = "";
    fldclearpin1.value = ""; // clear unencrypted pin
    fldclearpin2.value = "";
    fldclearpin3.value = "";

    return true;
}
</script>
```

Encrypt password for change password (JavaScript) for other login module

JavaScript

```
<script language="javascript" src="ame2eea.js"></script>
<script>
  HashMap parms = new HashMap();

  function onFormSubmit(form, flduserid, fldclearpin1,
fldclearpin2, fldclearpin3, fldoldpwd, fldnewpwd) {
    // Additional validation if required
    if (fldclearpin1.value == fldclearpin2.value) {
      alert("New password must be different from old
password");
      return false;
    }

    if (fldclearpin2.value != fldclearpin3.value) {
      alert("New password not match with retype new
password");
      return false;
    }

    if (fldclearpin1.value.length < 4 ||
```

Encrypt password for change password (JavaScript) for other login module

```
fldclearpin1.value.length > 14 || fldclearpin2.value.length
< 4
      || fldclearpin2.value.length > 14) {
      alert ('The old PIN or new PIN is too short or too
long');
      return false;
    }

    var regex=/^[0-9A-Za-z]+$/;

    if (!regex.test(fldclearpin2.value)) {
      alert("The new PIN contains illegal character. Please
use alphanumeric ONLY.");
      return false;
    }

    // Encryption parameters
    var e2eeSid = '<%=_e2EESid%>';
    var pubKey = '<%=_e2EEPublicKey%>';
    var randomNumber = '<%=_e2EERandomNum%>';
    var oaepHashAlgo = 'SHA-1';

    // Encrypt (get RPIN)
    // ame2eea.encryptChangePinForAM(e2eeSid, publicKey,
randomNumber, oldPin, newPin, oaepHashAlgo)
    fldnewpwd.value = ame2eea.encryptChangePinForAM(e2eeSid,
pubKey, randomNumber, fldclearpin1.value,
    fldclearpin2.value, oaepHashAlgo);
    fldoldpwd.value = "";
    fldclearpin1.value = ""; // clear unencrypted pin
    fldclearpin2.value = "";
    fldclearpin3.value = "";

    return true;
}
</script>
```

Error Codes

The client library may return one of the errors below.

JS Error
Chosen SHA variant is not supported
Invalid b64Pad formatting option
Invalid character in the base-64 string
Invalid '=' found in base-64 string

JS Error
Invalid outputUpper formatting option
Invalid RSA public key
Message too long for RSA
MGF: HASH algorithm is not recognized, hashAlgo=<hashAlgo>
OAEP: HASH algorithm is not recognized, hashAlgo=<hashAlgo>
String of HEX type contains invalid characters
String of HEX type must be in byte increments
The message to be encrypted is too long
Unexpected error in HMAC implementation
Unexpected error in SHA-2 implementation
XOR failure: two binaries have different lengths
charSize must be 8 or 16
format must be HEX or B64
inputFormat must be HEX, TEXT, ASCII, or B64
outputFormat must be HEX or B64
pad block corrupted
srcString of HEX type must be in byte increments

The following error code may be returned from the client library upon RPIN generation for Change Password.

Error Code	Description
58	Denied by password length policy
59	Denied by password consecutive repeating characters policy
60	Denied by password minimum numeric characters policy
61	Denied by password minimum uppercase characters policy

Error Code	Description
62	Denied by password minimum lowercase characters policy
63	Denied by password initial characters policy
64	Denied by password minimum alphabetic characters policy
65	Denied by password repeat of characters policy
66	Denied by password sequence of alphabetic characters policy
67	Denied by password sequence of numeric characters policy
68	Denied by password same as user id policy

Java

Reset

Encrypt password for reset password (Java) for Thales payShield login module

Java

```
AxMxE2EEClient e2ee = new AxMxE2EEClient();
try {
  /**
   * This method is used to encrypt user's pin during login
or password reset operation.
   * @param hashAlgorithmID Hash algorithm used for OAEP
padding.
   * @param e2eeSid E2EE Session Id
   * @param pin Login pin or password reset pin
   * @param publicKey Public key
   * @param serverRandom Server random
   * @param format PIN Block Format
   * @return encrypted login pin (rpin)
   * @throws Exception
   */

  String rpin = e2ee.encryptForLogin(/* Default value*/
AxMxE2EEClient.ALGOID3DES_SHA256,
                    e2eesid,
                    pin,
                    publicKey,
                    serverRandom,
                    AxMxE2EEClient.PINBLOCKFORMATISO0);

  int retCode = e2ee.getRetCode();
  If(retCode ==0) {
    //successful
```

Encrypt password for reset password (Java) for Thales payShield login module

```
  } else {
    If(e2ee.isInvalidPin(retCode)){
      //invalid pin
    } else {
      If(e2ee.isError(retCode))
        // is error
    }
  }
} catch (Exception e) {
  // TODO Auto-generated catch block
}
```

Encrypt password for reset password (Java) for other login module

Java

```
AxMxE2EEClient e2ee = new AxMxE2EEClient();
try {
  /**
   * This method is used to encrypt user's pin during login
or password reset operation.
   * @param hashAlgorithmID Hash algorithm used for OAEP
padding.
   * @param e2eeSid E2EE Session Id
   * @param pin Login pin or password reset pin
   * @param publicKey Public key
   * @param serverRandom Server random
   * @return encrypted login pin (rpin)
   * @throws Exception
   */

  String rpin = e2ee.encryptForLogin(/* Default value*/
AxMxE2EEClient.ALGOID3DES_SHA256,
                    e2eesid,
                    pin,
                    publicKey,
                    serverRandom);

  int retCode = e2ee.getRetCode();
  If(retCode ==0) {
    //successful
  } else {
    If(e2ee.isInvalidPin(retCode)){
      //invalid pin
    } else {
      If(e2ee.isError(retCode))
        // is error
    }
  }
} catch (Exception e) {
  // TODO Auto-generated catch block
}
```

Login

Encrypt password for login (Java) for Thales payShield login module

Java

```
AxMxE2EEClient e2ee = new AxMxE2EEClient();
try {
  /**
   * This method is used to encrypt user's pin during login
or password reset operation.
   * @param hashAlgorithmID Hash algorithm used for OAEP
padding.
   * @param e2eeSid E2EE Session Id
   * @param pin Login pin or password reset pin
   * @param publicKey Public key
   * @param serverRandom Server random
   * @param format PIN Block Format
   * @return encrypted login pin (rpin)
   * @throws Exception
   */
  String rpin = e2ee.encryptForLogin(/** Default value*/
AxMxE2EEClient.ALGO_ID_3DES_SHA256,
                    e2eesid,
                    pin,
                    publicKey,
                    serverRandom,

AxMxE2EEClient.PIN_BLOCK_FORMAT_HASH_MODE_NONE_SHORT);

  int retCode = e2ee.getRetCode();

  If(retCode ==0) {
    //successful
  } else {

    If(e2ee.isInvalidPin(retCode)){
      //invalid pin
    } else {

      If(e2ee.isError(retCode))
        // is error
    }
  }
} catch (Exception e) {
  // TODO Auto-generated catch block
}
```

Encrypt password for login (Java) for other login module

Java

Encrypt password for login (Java) for other login module

```
AxMxE2EEClient e2ee = new AxMxE2EEClient();
try {
  /**
   * This method is used to encrypt user's pin during login
or password reset operation.
   * @param hashAlgorithmID Hash algorithm used for OAEP
padding.
   * @param e2eeSid E2EE Session Id
   * @param pin Login pin or password reset pin
   * @param publicKey Public key
   * @param serverRandom Server random
   * @return encrypted login pin (rpin)
   * @throws Exception
   */
  String rpin = e2ee.encryptForLogin(/** Default value*/
AxMxE2EEClient.ALGO_ID_3DES_SHA256,
                    e2eesid,
                    pin,
                    publicKey,
                    serverRandom);

  int retCode = e2ee.getRetCode();

  If(retCode ==0) {
    //successful
  } else {

    If(e2ee.isInvalidPin(retCode)){
      //invalid pin
    } else {

      If(e2ee.isError(retCode))
        // is error
    }
  }
} catch (Exception e) {
  // TODO Auto-generated catch block
}
```

Change Password

Encrypt password for change password (Java) for Thales payShield login module

Java

```
AxMxE2EEClient e2ee = new AxMxE2EEClient();
try {
  /**
   * This method is used to encrypt user's existing pin and new
pin during password change operation.
   * @param hashAlgorithmID Hash algorithm used for OAEP
```

Encrypt password for change password (Java) for Thales payShield login module

```
padding.
    * @param e2eeSid E2EE Session Id
    * @param oldPin Login pin
    * @param newPin new Pin for password change
    * @param publicKey Public Key
    * @param serverRandom Server random
    * @param format PIN Block Format
    * @param otherParams Other parameters to hold
passwordQualityPolicy or for future enhancement
    * @return encrypted login cum password change pin (rpin)
    * @throws Exception
    */
   // Specify userId and password quality policy returned from
preauthenticate
   Map<String, String> otherParams = new HashMap<String,
String>();

otherParams.put(AxMxE2EEClient.OTHERPARAMSPASSWORDQUALITYPOLICY,
passwordQualityPolicy);
   otherParams.put(AxMxE2EEClient.OTHERPARAMSUSER_ID, userId);

   String rpin = e2ee.encryptForChangePin(/* Default value*/
AxMxE2EEClient.ALGOID3DES_SHA256,
                      e2eesid,
                      oldPin,
                      newPin,
                      publicKey,
                      serverRandom,

AxMxE2EEClient.PINBLOCKFORMATHASHMODENONESHORT,
                      otherParams);

   int retCode = e2ee.getRetCode();
   If(retCode ==0) {
     //successful
   } else {
     If(e2ee.isInvalidPin(retCode)){
       //invalid pin
     } else {
       If(e2ee.isError(retCode))
         // is error
     }
   }
} catch (Exception e) {
   // TODO Auto-generated catch block
}
```

Encrypt password for change password (Java) for other login module

Java

Encrypt password for change password (Java) for other login module

```
AxMxE2EEClient e2ee = new AxMxE2EEClient();
try {
  /**
   * This method is used to encrypt user's existing pin and
new pin during password change operation.
   *
   * @param hashAlgorithmID Hash algorithm used for OAEP
padding.
   * @param e2eeSid E2EE Session Id
   * @param oldPin Login pin
   * @param newPin new Pin for password change
   * @param publicKey Public Key
   * @param serverRandom Server random
   * @return encrypted login cum password change pin (rpin)
   * @throws Exception
   */

  String rpin = e2ee.encryptForChangePin(/* Default value*/
AxMxE2EEClient.ALGOID3DES_SHA256,
                    e2eesid,
                    oldPin,
                    newPin,
                    publicKey,
                    serverRandom);

  int retCode = e2ee.getRetCode();
  If(retCode ==0) {
    //successful
  } else {
    If(e2ee.isInvalidPin(retCode)){
      //invalid pin
    } else {
      If(e2ee.isError(retCode))
        // is error
    }
  }
} catch (Exception e) {
  // TODO Auto-generated catch block
}
```

Error Codes

The client library may return one of the errors below.

Error Code	Description
-1	Invalid hex character
0	No error
1	Invalid hex character

Error Code	Description
10	Invalid PIN length
11	Invalid PIN string
16	Invalid PIN string
20	Invalid PIN block
21	Invalid random number length
22	Invalid random number
30	Invalid pin message
31	Invalid pin message length
40	Invalid encoded message length
41	Invalid RSA key length
42	Invalid RSA key error
43	The public key is not found an error

The following error code may be returned from the client library upon RPIN generation for Change Password.

Error Code	Description
58	Denied by password length policy
59	Denied by password consecutive repeating characters policy
60	Denied by password minimum numeric characters policy
61	Denied by password minimum uppercase characters policy
62	Denied by password minimum lowercase characters policy
63	Denied by password initial characters policy
64	Denied by password minimum alphabetic characters policy
65	Denied by password repeat of characters policy
66	Denied by password sequence of alphabetic characters policy

Error Code	Description
67	Denied by password sequence of numeric characters policy
68	Denied by password same as user id policy

iOS

Reset

Encrypt password for reset password (iOS) for Thales payShield login module

C/C++

```
#import <E2EEA.h>

// create a new E2EEA object
E2EEA * e2eea = [[E2EEA alloc] init];

NSString* rpin = [e2ee encryptForLogin: [e2ee SHA1]
                  session:e2eesid
                  password:password
                  publicKeyValue:publicKeyValue
                  random:serverRandom
                  pinBlockFormat: (unsigned char) fmt];

if([e2eea getReturnCode]==0)
  NSLog(@"RPIN: %@", rpin);

// free the E2EEA object
[e2eea release];
```

Encrypt password for reset password (iOS) for other login module

C/C++

```
#import <E2EEA.h>

// create a new E2EEA object
E2EEA * e2eea = [[E2EEA alloc] init];

NSString* rpin = [e2ee encryptForLogin: [e2ee SHA1]
                  session:e2eesid
                  password:password
                  publicKeyValue:publicKeyValue
                  random:serverRandom];

if([e2eea getReturnCode]==0)
  NSLog(@"RPIN: %@", rpin);
```

Encrypt password for reset password (iOS) for other login module

```
// free the E2EEA object
[e2eea release];
```

Login

Encrypt password for login (iOS) for Thales payShield login module

C/C++

```
#import <E2EEA.h>

// create a new E2EEA object
E2EEA * e2eea = [[E2EEA alloc] init];

NSString* rpin = [e2ee encryptForLogin: [e2ee SHA1]
                  session:e2eesid
                  password:password
                  publicKeyValue:publicKeyValue
                  random:serverRandom
                  pinBlockFormat: (unsigned char) fmt];

if([e2eea getReturnCode]==0)
  NSLog(@"RPIN: %@", rpin);

// free the E2EEA object
[e2eea release];
```

Encrypt password for login (iOS) for other login module

C/C++

```
#import <E2EEA.h>

// create a new E2EEA object
E2EEA * e2eea = [[E2EEA alloc] init];

NSString* rpin = [e2ee encryptForLogin: [e2ee SHA1]
                  session:e2eesid
                  password:password
                  publicKeyValue:publicKeyValue
                  random:serverRandom];

if([e2eea getReturnCode]==0)
  NSLog(@"RPIN: %@", rpin);
```

Encrypt password for login (iOS) for other login module

```
// free the E2EEA object
[e2eea release];
```

Change Password

Encrypt password for change password (iOS) for Thales payShield login module

C/C++

```
#import <E2EEA.h>

// create a new E2EEA object
E2EEA * e2eea = [[E2EEA alloc] init];

NSString* rpin = [e2ee encryptForLogin: [e2ee SHA1]
                  session:e2eesid
                  oldPassword:oldPin
                  newPassword:newPin
                  publicKeyValue:publicKeyValue
                  random:serverRandom
                  pinBlockFormat: (unsigned char) fmt
                  userId: (NSString*) userId
                  passwordPolicy: (NSString*) policy];

if([e2eea getReturnCode]==0)
  NSLog(@"RPIN: %@", rpin);

// free the E2EEA object
[e2eea release];{{/code}}
```

Encrypt password for change password (iOS) for other login module

C/C++

```
#import <E2EEA.h>

// create a new E2EEA object
E2EEA * e2eea = [[E2EEA alloc] init];

NSString* rpin = [e2ee encryptForLogin: [e2ee SHA1]
                  session:e2eesid
                  oldPassword:oldPin
                  newPassword:newPin
                  publicKeyValue:publicKeyValue
                  random:serverRandom];

if([e2eea getReturnCode]==0)
  NSLog(@"RPIN: %@", rpin);
```

Encrypt password for change password (iOS) for other login module

```
// free the E2EEA object
[e2eea release];
```

Error Codes

Error Code	Description
-1	Invalid hex character
0	No error
1	Invalid hex character
10	Invalid PIN length
11	Invalid PIN string
16	Invalid PIN string
20	Invalid PIN block
21	Invalid random number length
22	Invalid random number
30	Invalid pin message
31	Invalid pin message length
40	Invalid encoded message length
41	Invalid RSA key length
42	Invalid RSA key error
43	The public key is not found an error

The following error code may be returned from the client library upon RPIN generation for Change Password.

Error Code	Description
58	Denied by password length policy
59	Denied by password consecutive repeating characters policy

Error Code	Description
60	Denied by password minimum numeric characters policy
61	Denied by password minimum uppercase characters policy
62	Denied by password minimum lowercase characters policy
63	Denied by password initial characters policy
64	Denied by password minimum alphabetic characters policy
65	Denied by password repeat of characters policy
66	Denied by password sequence of alphabetic characters policy
67	Denied by password sequence of numeric characters policy
68	Denied by password same as user id policy

6. Appendices

HSM Native Errors

SafeNet ProtectServer HSM

The Rest APIs may return one of the HSM error codes below in the extendedError field of the response.

Definition in <e2eefm_defs.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_OK	0x000	0	Success, no error.
e2eefm_RV_FAIL	0x001	1	General failure.
e2eefm_RV_RESPONSE_BUFFER_TOO_SMALL	0x002	2	Response buffer is too small.
e2eefm_RV_ENCRYPTION_FAIL	0x003	3	Failure during encryption.
e2eefm_RV_DECRYPTION_FAIL	0x004	4	Failure during decryption.
e2eefm_RV_INIT_VECTOR_FAIL	0x005	5	Failure related to initialization vector.
e2eefm_RV_OUTPUT_BUFFER_TOO_SMALL	0x006	6	Output buffer is too small.
e2eefm_RV_CONTEXT_UNINITIALIZED	0x007	7	Context is uninitialized.
e2eefm_RV_INVALID_SLOT	0x010	16	Slot ID is invalid.
e2eefm_RV_SLOTS_UNINITIALIZED	0x011	17	Slot IDs are uninitialized.

Definition in <e2eefm_defs.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_DIGEST_FAIL	0x020	32	Failure generating hash/digest.
e2eefm_RV_MEMORY_ALLOC_FAIL	0x031	49	Failure during memory allocation.
e2eefm_RV_REQUEST_FORMAT_INVALID	0X053	83	Invalid request format.
e2eefm_RV_FUNCTION_NOT_SUPPORTED	0x054	84	The requested FM function is not supported.
e2eefm_RV_ALGORITHM_NOT_SUPPORTED	0x055	85	The requested algorithm is not supported.
e2eefm_RV_PARSE_FAIL_FORMAT_INVALID	0x060	96	Parsing has failed due to invalid format.
e2eefm_RV_PARSE_FAIL_PARAM_ID	0x061	97	Parsing has failed due to incorrect parameter ID.
e2eefm_RV_PARSE_FAIL_PARAM_LENGTH	0x062	98	Parsing has failed due to incorrect parameter length.
e2eefm_RV_PARSE_FAIL_PARAM_DATA	0x063	99	Parsing has failed due to incorrect parameter data.

Definition in <e2eefm_defs.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_PARSE_FAIL_TERMINATED	0x064	100	Parsing terminated due to error.
e2eefm_RV_PARSE_FAIL_MORE_PARAMS	0x065	101	Parsing has failed due to lack of parameters.
e2eefm_RV_NOT_FOUND_RSA_KEY	0x0A0	160	RSA key is not found.
e2eefm_RV_NOT_FOUND_SECRET_KEY	0x0A1	161	Secret key is not found.
e2eefm_RV_INPUT_MISSING_OR_INVALID	0x100	256	Input is missing or invalid.
e2eefm_RV_INPUT_MISSING_SlotId	0x101	257	Slot ID is not specified.
e2eefm_RV_INPUT_MISSING_KeyLabel	0x102	258	Key label is not specified.
e2eefm_RV_INPUT_MISSING_CipherText	0x103	259	Encrypted data is not specified.
e2eefm_RV_INPUT_MISSING_UserPin	0x104	260	User PIN is not specified.
e2eefm_RV_INPUT_MISSING_AlgoId	0x105	261	Algorithm is not specified.
e2eefm_RV_INPUT_INVALID_AlgoId	0x106	262	Algorithm is invalid.
e2eefm_RV_INPUT_MISSING_PublicKeyLabel	0x109	265	Public key is not specified.
e2eefm_RV_INPUT_MISSING_PrivateKeyLabel	0x110	272	Private key is not specified.

Definition in <e2eefm_defs.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_INPUT_MISSING_ServerRandom	0x111	273	Server random is not specified.
e2eefm_RV_INPUT_MISSING_E2EE_RPIN	0x112	274	E2EE pin is not specified.
e2eefm_RV_INPUT_MISSING_InitVector	0x113	275	IV is not specified.
e2eefm_RV_INPUT_MISSING_ClientRandom	0x114	276	Client random is not specified.
e2eefm_RV_INPUT_MISSING_TransformedPin	0x115	277	Transformed pin is not specified.
e2eefm_RV_INPUT_MISSING_Salt	0x116	278	Salt is not specified.
e2eefm_RV_AppId_Invalid	0x119	281	App ID is invalid.
e2eefm_RV_INPUT_MISSING_PasswordChars	0x121	289	Password chars are not specified.
e2eefm_RV_AccountNumber_Invalid	0x129	297	Account number is invalid.
e2eefm_RV_KCV_Invalid	0x131	305	KCV is invalid.
e2eefm_RV_INPUT_MISSING_OAEP	0x153	339	OAEP is not specified.
e2eefm_RV_INPUT_MISSING_Ticket	0x154	340	Ticket is not specified .
e2eefm_RV_KEY_NOT_FOUND	0x1a0	416	Key is not found.

Definition in <e2eefm_defs.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_KEY_REDEEM_TICKET_FAIL	0x1a1	417	Failure during ticket redemption.
e2eefm_RV_KEY_CREATION_FAIL	0x1a2	418	Failure during key creation.
e2eefm_RV_PIN_BLOCK_NO_PINS	0x200	512	No pin is found in pin block.
e2eefm_RV_PIN_BLOCK_NO_PIN_FOR_INDEX	0x201	513	No pin is found in index.
e2eefm_RV_PIN_BLOCK_INVALID_SERVER_RANDOM	0x202	514	Invalid or mismatched server random.
e2eefm_RV_RPIN_MISSING_ServerRandom	0x230	560	Server random is not specified.
e2eefm_RV_RPIN_MISSING_PIN	0x231	561	Pin is not specified.
e2eefm_RV_RPIN_MISSING_E2EE_Session	0x232	562	E2EE session is not specified.
e2eefm_RV_RPIN_DECRYPTION_FAIL	0x240	576	Failure decrypting RPIN.
e2eefm_RV_RPIN_Customize_DECRYPTION_FAIL	0x241	577	Failure decrypting customized RPIN.
e2eefm_RV_RPIN_DECRYPTION_OAEP_MISMATCH	0x242	578	OAEP verification failed.

Definition in <e2eefm_defs.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_RPIN_DECRYPTION_OAEP_DIGEST_NOT_SUPPORTED	0x243	579	The specified OAEP digest algorithm is not supported.
e2eefm_RV_RPIN_MISMATCH	0x244	580	Mismatched RPIN.
e2eefm_RV_TPIN_MISSING_ClientRandom	0x250	592	Client random value is not specified.
e2eefm_RV_TPIN_MISSING_PIN	0x251	593	Pin is not specified.
e2eefm_RV_TPIN_TransformationFailed	0x252	594	Failure during TPIN transformation.
e2eefm_RV_TPIN_DECRYPTION_FAIL	0x260	608	Failure decrypting TPIN.
e2eefm_RV_TPIN_ENCRYPTION_FAIL	0x261	609	Failure encrypting TPIN.
e2eefm_RV_TPIN_Customize_ENCRYPTION_FAIL	0x262	610	Failure encrypting customized TPIN.
e2eefm_RV_TPIN_Illegal	0x2b0	688	Pin is illegal.
e2eefm_RV_TPIN_TooShort	0x2b1	689	Pin is too short.
e2eefm_RV_TPIN_TooLong	0x2b2	690	Pin is too long.
e2eefm_RV_TPIN_MandatoryChars	0x2b3	691	Pin does not contain

Definition in <e2eefm_defs.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
			mandatory characters.
e2eefm_RV_TPIN_ForbiddenChars	0x2b4	692	Pin contains forbidden characters.
e2eefm_RV_TPIN_AllowedChars	0x2b5	693	Pin contains characters that are not explicitly allowed.
e2eefm_RV_TPIN_MaxConsecutive	0x2b6	694	Pin has more than the allowed number of consecutive characters.
e2eefm_RV_TPIN_ForbiddenWords	0x2b7	695	Pin contains forbidden words.
e2eefm_RV_TPIN_MinUpperCase	0x2b8	696	Pin contains less than the required number of upper case characters.
e2eefm_RV_TPIN_MinLowerCase	0x2b9	697	Pin contains less than the required number of lower case characters.
e2eefm_RV_TPIN_MinNumeric	0x2ba	698	Pin contains less than the required number of numeric characters.

Definition in <e2eefm_defs.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_TPIN_MinAlpha	0x2bb	699	Pin contains less than the required number of alphabetic characters.
e2eefm_RV_TPIN_MinNonAlpha	0x2bc	700	Pin contains less than the required number of non-alphabetic characters.
e2eefm_RV_TPIN_NumRepeatChars	0x2bd	701	Pin contains more than the allowed number of repeated characters.
e2eefm_RV_TPIN_SequentialAlpha	0x2be	702	Pin contains more than the allowed number of sequential alphabetic characters.
e2eefm_RV_TPIN_SequentialNumeric	0x2bf	703	Pin contains more than the allowed number of sequential numeric characters.
e2eefm_RV_VERIFY_FAIL_PIN	0x2c0	704	Pin verification failed.
e2eefm_RV_VERIFY_FAIL_ServerRandom	0x2c1	705	Server random

Definition in <e2eefm_defs.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
			verification failed.
e2eefm_RV_VERIFY_FAIL_ClientRandom	0x2c2	706	Client random verification failed.
e2eefm_RV_VERIFY_OLD_AND_NEW_PINS_SAME	0x2c3	707	New pin is the same as the old pin (changing pin).
e2eefm_RV_VERIFY_NEW_PIN_USED_BEFORE	0x2c4	708	Pin has been used before.
e2eefm_RV_VERIFY_FAIL_Salt	0x2c5	709	Salt verification failed.
e2eefm_RV_VERIFY_PinTooShort	0x2c6	710	Pin is too short.
e2eefm_RV_VERIFY_PinTooLong	0x2c7	711	Pin is too long.
e2eefm_RV_OATH_ErrorBase	0x300	768	OATH error base code: full error code 0x03HH where 0xHH is code from <am_oath.h>.
e2eefm_RV_CustomErrorBase	0xf000	61440	Codes beyond this value are for customization and not used by FM core.

SafeNet Payment HSM

The Rest APIs may return one of the HSM error codes below in the extendedError field of the response.

Definition in <eftapibase.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Message from ErrToString() function
EFT_NO_ERROR	0x00	0	No error
EFT_DES_FAULT	0x01	1	DES Fault
EFT_PM_NOT_ENABLED	0x02	2	Function Not enabled
EFT_INCORRECT_MSG_LEN	0x03	3	Incorrect Message Length
EFT_INVALID_DATA	0x04	4	Invalid Data
EFT_INVALID_KEY_INDEX	0x05	5	Invalid Key Index
EFT_INVALID_PIN_FMT_SPEC	0x06	6	Invalid PIN Format Specifier
EFT_PIN_FORMAT_ERROR	0x07	7	PIN Format Error
EFT_VERIFICATION_FAILED	0x08	8	Verification Failure
EFT_TAMPERED	0x09	9	ESM Tampered
EFT_UNITIALISED_KEY	0x0A	10	Uninitialised Key
EFT_PIN_CHECK_LEN_ERROR	0x0B	11	PIN Check Length Error
EFT_INCONSISTENT_REQ_FIELDS	0x0C	12	Inconsistent Request Fields
EFT_INVALID_VISA_PIN_VER_KEY_IND	0x0F	15	Invalid VISA PIN Verification Key Index
	0x10	16	Internal Error
	0x1a	26	Duplicate Key or Record
EFT_INVALID_KEY_SPECIFIER_LEN	0x20	32	Invalid Key Specifier Length

Definition in <eftapibase.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Message from ErrToString() function
EFT_UNSUPPORTED_KEY_SPECIFIER	0x21	33	Unsupported Key Specifier
EFT_INVALID_KEY_SPECIFIER_CONTENT	0x22	34	Invalid Key Specifier Content
EFT_INVALID_KEY_SPECIFIER_FORMAT	0x23	35	Invalid Key Specifier Format
EFT_INVALID_FUNCTION_MODIFIER	0x24	36	Invalid Function Modifier
	0x30	48	Invalid Admin Signature
	0x31	49	Unknown Error 49
	0x32	50	No Admin Session
	0x36	54	Device Error
	0x39	57	Public Key Pair Unavailable
	0x3a	58	Public Key Pair Generating
	0x3b	59	RSA Cipher Error
	0x40	64	Unsupported SEED Key
	0x70	112	Invalid PIN Block Flag
	0x71	113	Invalid PIN Block Random Flag
	0x72	114	Invalid PIN Block Delimiter
	0x73	115	Invalid PIN Block RB
	0x74	116	Invalid PIN Block RandNum
	0x75	117	Invalid PIN Block RA

Definition in <eftapibase.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Message from ErrToString() function
	0x76	118	Invalid PIN
	0x77	119	Invalid PIN Length
	0x7f	127	Invalid Print Token
	0x80	128	OAEP Decode Error
	0x81	129	OAEP Invalid Header Byte
	0x82	130	OAEP Invalid PIN Block
	0x83	131	OAEP Invalid Random Number
EFT_CL_ERROR_OFFSET	0x100	256	Unknown Error 256
	0x101	257	ESM not found
	0x102	258	Initialisation failed
	0x103	259	Tcp host address error
	0x104	260	Cannot create TCP socket
	0x105	261	Cannot set TCP socket options
	0x106	262	Cannot connect on TCP socket
	0x107	263	Error closing TCP socket
	0x108	264	Error sending on TCP socket
	0x109	265	Timeout sending on TCP socket
	0x10a	266	Error receiving on TCP socket
	0x10b	267	TCP window full error

Definition in <eftapibase.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Message from ErrToString() function
	0x10c	268	API timer thread error
	0x10d	269	API timer thread terminate error
	0x10e	270	API heartbeat thread error
	0x10f	271	API receive thread error
	0x110	272	Receive queue empty
	0x111	273	Error translating error code to string
	0x112	274	Timeout waiting to receive
	0x113	275	ESM not responding
	0x114	276	Callback function not registered
	0x115	277	Error opening serial comms port
	0x116	278	Error getting serial comms port state
	0x117	279	Error setting serial comms port state
	0x118	280	Error setting serial comms port mask
	0x119	281	Error setting serial comms port timeouts
	0x11a	282	Error reading from serial comms port
	0x11b	283	Error sending to serial comms port
	0x11c	284	Cannot find ethernet adapter

Definition in <eftapibase.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Message from ErrToString() function
	0x11d	285	Cannot set ethernet packet filter settings
	0x11e	286	Error sending to ethernet interface
	0x11f	287	Error reading from ethernet interface
	0x120	288	Next ESM found
	0x121	289	Function Parameter Error
	0x122	290	Error Reading ESM
	0x123	291	Error Writing to ESM
EFT_PARAM_ERROR	0x125	293	Parameter Error
EFT_MEMORY_ERROR	0x126	294	Memory Allocation Error
EFT_CL_ERROR	0x127	295	Cryptolink Error
EFT_INTERNAL_ERROR	0x128	296	Internal Error
EFT_INVALID_BUFFER_LENGTH	0x129	297	Invalid Buffer Length
EFT_MAX_MSG_SIZE_EXCEEDED	0x12A	298	Maximum Message Length Exceeded
EFT_NO_SCRIPT_RETURNED_DATA	0x12B	299	Expected Response missing from Test Script
EFT_BAD_RETURNED_DATA	0x12C	300	Returned Data Doesn't Match Expected Response

Thales payShield HSM

The Rest APIs may return one of the HSM error codes below in the extendedError field of the response.

Error Code	Description
01	Verification failure or warning of an imported key parity error.
02	MAC verification failed or key inappropriate length for the algorithm.
03	Invalid public key encoding type.
04	Key Length Error.
05	Invalid key type.
06	Public exponent length error.
08	Supplied public exponent is even.
14	PIN encrypted under LMK pair 02-03 is invalid.
15	Error in input data.
17	HSM is not authorized, or operation prohibited by security settings.
20	PIN block does not contain valid values.
22	Invalid account number.
23	Invalid PIN block format code.
24	The PIN is fewer than 4 or more than 12 digits in length.
47	The algorithm is not licensed.
48	A stronger LMK is required to protect this size RSA key.
68	Command disabled.
69	PIN block format has been disabled.
76	RSA encrypted data length error.
77	Clear data block error.
85	Invalid OAEP MGF.
86	Invalid OAEP MGF Hash Function.
87	OAEP parameter error.
88	OAEP error.

Utimaco Crypto Server HSM

The Rest APIs may return one of the HSM error codes below in the extendedError field of the response.

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_OK	0x000	0	Success, no error.
e2eefm_RV_FAIL	0x001	1	General failure.
e2eefm_RV_RESPONSE_BUFFER_TOO_SMALL	0x002	2	Response buffer is too small.
e2eefm_RV_ENCRYPTION_FAIL	0x003	3	Failure during encryption.
e2eefm_RV_DECRYPTION_FAIL	0x004	4	Failure during decryption.
e2eefm_RV_INIT_VECTOR_FAIL	0x005	5	Failure related to initialization vector.
e2eefm_RV_OUTPUT_BUFFER_TOO_SMALL	0x006	6	Output buffer is too small.
e2eefm_RV_CONTEXT_UNINITIALIZED	0x007	7	Context is uninitialized.
e2eefm_RV_DIGEST_FAIL	0x020	32	Failure generating hash/digest.
e2eefm_RV_MEMORY_ALLOC_FAIL	0x031	49	Failure during memory allocation.

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_REQUEST_FORMAT_INVALID	0X053	83	Invalid request format.
e2eefm_RV_FUNCTION_NOT_SUPPORTED	0x054	84	The requested FM function is not supported.
e2eefm_RV_ALGORITHM_NOT_SUPPORTED	0x055	85	The requested algorithm is not supported.
e2eefm_RV_PARSE_FAIL_FORMAT_INVALID	0x060	96	Parsing has failed due to invalid format
e2eefm_RV_PARSE_FAIL_PARAM_ID	0x061	97	Parsing has failed due to incorrect parameter ID.
e2eefm_RV_PARSE_FAIL_PARAM_LENGTH	0x062	98	Parsing has failed due to incorrect parameter length.
e2eefm_RV_PARSE_FAIL_PARAM_DATA	0x063	99	Parsing has failed due to incorrect parameter data.
e2eefm_RV_PARSE_FAIL_TERMINATED	0x064	100	Parsing terminated due to error.
e2eefm_RV_PARSE_FAIL_MORE_PARAMS	0x065	101	Parsing has failed due to

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
			lack of parameters.
e2eefm_RV_NOT_FOUND_RSA_KEY	0x0A0	160	RSA key is not found.
e2eefm_RV_NOT_FOUND_SECRET_KEY	0x0A1	161	Secret key is not found.
e2eefm_RV_INPUT_MISSING_OR_INVALID	0x100	256	Input is missing or invalid.
e2eefm_RV_INPUT_MISSING_SlotId	0x101	257	Slot ID is not specified.
e2eefm_RV_INPUT_MISSING_KeyLabel	0x102	258	Key label is not specified.
e2eefm_RV_INPUT_MISSING_CipherText	0x103	259	Encrypted data is not specified.
e2eefm_RV_INPUT_MISSING_UserPin	0x104	260	User PIN is not specified.
e2eefm_RV_INPUT_MISSING_AlgoId	0x105	261	Algorithm is not specified.
e2eefm_RV_INPUT_INVALID_AlgoId	0x106	262	Algorithm is invalid.
e2eefm_RV_INPUT_MISSING_PublicKeyLabel	0x109	265	Public key is not specified.
e2eefm_RV_INPUT_MISSING_PrivateKeyLabel	0x110	272	Private key is not specified.
e2eefm_RV_INPUT_MISSING_ServerRandom	0x111	273	Server random is not specified.
e2eefm_RV_INPUT_MISSING_E2EE_RPIN	0x112	274	E2EE pin is not specified.

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_INPUT_MISSING_InitVector	0x113	275	IV is not specified.
e2eefm_RV_INPUT_MISSING_ClientRandom	0x114	276	Client random is not specified.
e2eefm_RV_INPUT_MISSING_TransformedPin	0x115	277	Transformed pin is not specified.
e2eefm_RV_INPUT_MISSING_Salt	0x116	278	Salt is not specified.
e2eefm_RV_AppId_Invalid	0x119	281	App ID is invalid.
e2eefm_RV_INPUT_MISSING_PasswordChars	0x121	289	Password chars are not specified.
e2eefm_RV_AccountNumber_Invalid	0x129	297	Account number is invalid.
e2eefm_RV_KCV_Invalid	0x131	305	KCV is invalid.
e2eefm_RV_INPUT_MISSING_OAEP	0x153	339	OAEP is not specified.
e2eefm_RV_INPUT_MISSING_Ticket	0x154	340	Ticket is not specified.
e2eefm_RV_KEY_NOT_FOUND	0x1a0	416	Key is not found.
e2eefm_RV_KEY_REDEEM_TICKET_FAIL	0x1a1	417	Failure during ticket redemption.
e2eefm_RV_KEY_CREATION_FAIL	0x1a2	418	Failure during key creation.

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_PIN_BLOCK_NO_PINS	0x200	512	No pin is found in pin block.
e2eefm_RV_PIN_BLOCK_NO_PIN_FOR_INDEX	0x201	513	No pin found in the block for given index.
e2eefm_RV_PIN_BLOCK_INVALID_SERVER_RANDOM	0x202	514	Invalid or mismatched server random.
e2eefm_RV_RPIN_MISSING_ServerRandom	0x230	560	Server random is not specified.
e2eefm_RV_RPIN_MISSING_PIN	0x231	561	Pin is not specified.
e2eefm_RV_RPIN_MISSING_E2EE_Session	0x232	562	E2EE session is not specified.
e2eefm_RV_RPIN_DECRYPTION_FAIL	0x240	576	Failure decrypting RPIN
e2eefm_RV_RPIN_Customize_DECRYPTION_FAIL	0x241	577	Failure decrypting customized RPIN.
e2eefm_RV_RPIN_DECRYPTION_OAEP_MISMATCH	0x242	578	OAEP verification failed.
e2eefm_RV_RPIN_DECRYPTION_OAEP_DIGEST_NOT_SUPPORTED	0x243	579	The specified OAEP digest algorithm is not supported.
e2eefm_RV_RPIN_MISMATCH	0x244	580	Mismatched RPIN.

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_TPIN_MISSING_ClientRandom	0x250	592	Client random value is not specified.
e2eefm_RV_TPIN_MISSING_PIN	0x251	593	Pin is not specified.
e2eefm_RV_TPIN_TransformationFailed	0x252	594	Failure during TPIN transformation.
e2eefm_RV_TPIN_DECRYPTION_FAIL	0x260	608	Failure decrypting TPIN.
e2eefm_RV_TPIN_ENCRYPTION_FAIL	0x261	609	Failure encrypting TPIN.
e2eefm_RV_TPIN_Customize_ENCRYPTION_FAIL	0x262	610	Failure encrypting customized pin.
e2eefm_RV_TPIN_Illegal	0x2b0	688	Pin is illegal.
e2eefm_RV_TPIN_TooShort	0x2b1	689	Pin is too short.
e2eefm_RV_TPIN_TooLong	0x2b2	690	Pin is too long.
e2eefm_RV_TPIN_MandatoryChars	0x2b3	691	Pin does not contain mandatory characters.
e2eefm_RV_TPIN_ForbiddenChars	0x2b4	692	Pin contains forbidden characters.
e2eefm_RV_TPIN_AllowedChars	0x2b5	693	Pin contains characters that are not

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
			explicitly allowed.
e2eefm_RV_TPIN_MaxConsecutive	0x2b6	694	Pin has more than the allowed number of consecutive characters.
e2eefm_RV_TPIN_ForbiddenWords	0x2b7	695	Pin contains forbidden words.
e2eefm_RV_TPIN_MinUpperCase	0x2b8	696	Pin contains less than the required number of upper case characters.
e2eefm_RV_TPIN_MinLowerCase	0x2b9	697	Pin contains less than the required number of lower case characters.
e2eefm_RV_TPIN_MinNumeric	0x2ba	698	Pin contains less than the required number of numeric characters.
e2eefm_RV_TPIN_MinAlpha	0x2bb	699	Pin contains less than the required number of alphabetic characters.
e2eefm_RV_TPIN_MinNonAlpha	0x2bc	700	Pin contains less than the required number of

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
			non-alphabetic characters.
e2eefm_RV_TPIN_NumRepeatChars	0x2bd	701	Pin contains more than the allowed number of repeated characters.
e2eefm_RV_TPIN_SequentialAlpha	0x2be	702	Pin contains more than the allowed number of sequential alphabetic characters.
e2eefm_RV_TPIN_SequentialNumeric	0x2bf	703	Pin contains more than the allowed number of sequential numeric characters.
e2eefm_RV_VERIFY_FAIL_PIN	0x2c0	704	Pin verification failed
e2eefm_RV_VERIFY_FAIL_ServerRandom	0x2c1	705	Server random verification failed
e2eefm_RV_VERIFY_FAIL_ClientRandom	0x2c2	706	Client random verification failed
e2eefm_RV_VERIFY_OLD_AND_NEW_PINS_SAME	0x2c3	707	New pin is the same as the old pin (changing pin).

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
e2eefm_RV_VERIFY_NEW_PIN_USED_BEFORE	0x2c4	708	Pin has been used before.
e2eefm_RV_VERIFY_FAIL_Salt	0x2c5	709	Salt verification failed
e2eefm_RV_VERIFY_PinTooShort	0x2c6	710	Pin is too short.
e2eefm_RV_VERIFY_PinTooLong	0x2c7	711	Pin is too long.
e2eefm_RV_OATH_ErrorBase	0x300	768	OATH error base code: full error code 0x03HH where 0xHH is code from <am_oath.h>.
e2eefm_RV_CustomErrorBase	0xf000	61440	Codes beyond this value are for customization and not used by FM core.
E_CSA	0xB900	47360	CryptoServer API
E_CSA_CORE	0xB90000	12124160	CryptoServer API core functions
E_CSA_CORE_BAD_TAG	0xB9000000	3103784960	bad tag in data block
E_CSA_CORE_HANDLE	0xB9000001	3103784961	invalid handle
E_CSA_CORE_INVAL	0xB9000002	3103784962	invalid argument

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
E_CSA_CORE_MEM	0xB9000003	3103784963	can't alloc memory
E_CSA_CORE_STACK	0xB9000004	3103784964	malformed protocol stack
E_CSA_CORE_SIZE	0xB9000005	3103784965	data block too big
E_CSA_CORE_V24_DEV	0xB9000007	3103784967	bad V24 device
E_CSA_CORE_V24_PARAM	0xB9000008	3103784968	bad V24 parameter
E_CSA_CORE_BLK_LEN	0xB9000009	3103784969	can't calculate block length
E_CSA_CORE_EMPTY	0xB900000A	3103784970	empty command block
E_CSA_CORE_BAD_ANSW	0xB900000B	3103784971	malformed answer block from CSLAN
E_CSA_CORE_V24_CTRL	0xB900000C	3103784972	can't set V24 device
E_CSA_CORE_NO_V24	0xB900000D	3103784973	V24 mode not activated
E_CSA_CORE_V24_CRC	0xB900000E	3103784974	V24 crc error on read
E_CSA_CORE_FMT_LEN	0xB9000010	3103784976	bad length within format string (scanf)
E_CSA_CORE_BAD_CMD	0xB9000011	3103784977	bad format of command block

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
E_CSA_CORE_BAD_OUT	0xB9000012	3103784978	bad parameter structure (scanf)
E_CSA_CORE_BAD_FMT	0xB9000013	3103784979	bad format string (scanf)
E_CSA_CORE_SCANF	0xB9000014	3103784980	cs_scanf not supported
E_CSA_CORE_HDL_IN_USE	0xB9000015	3103784981	CSAPI handle still in use
E_CSA_CORE_FCT_NO_MULTICAST	0xB9000016	3103784982	can only use multicast for CSLScanDevices
E_CSA_CMDS	0xB90002	12124162	command layer CMDS for CryptoServer
E_CSA_CMDS_ALEN	0xB9000200	3103785472	length error of answer block
E_CSA_CMDS_CLEN	0xB9000201	3103785473	bad length of command data
E_CSA_CMDS_PARAM	0xB9000202	3103785474	missing parameter structure
E_CSA_CMDS_TAG	0xB9000203	3103785475	bad tag of answer block
E_CSA_AUTH	0xB90004	12124164	authentication layer for CryptoServer
E_CSA_AUTH_ALEN	0xB9000400	3103785984	length error of answer block

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
E_CSA_AUTH_BAD_FC	0xB9000401	3103785985	invalid function code
E_CSA_AUTH_BAD_ANSW	0xB9000402	3103785986	malformed answer block
E_CSA_AUTH_BAD_MECH	0xB9000403	3103785987	invalid authentication mechanism
E_CSA_AUTH_HASH_ERR	0xB9000404	3103785988	error in hash function
E_CSA_AUTH_SIGN_ERR	0xB9000405	3103785989	error in signature function
E_CSA_AUTH_HMAC_ERR	0xB9000406	3103785990	error in HMAC function
E_CSA_SM	0xB90006	12124166	secure messaging layer for CryptoServer
E_CSA_SM_ALEN	0xB9000600	3103786496	length error of answer block
E_CSA_SM_BAD_ANSW	0xB9000601	3103786497	malformed answer block
E_CSA_SM_BAD_MECH	0xB9000602	3103786498	invalid SM mechanism
E_CSA_SM_NO_DATA	0xB9000603	3103786499	zero length data
E_CSA_SM_DES_ERR	0xB9000604	3103786500	en- / decryption / MAC error
E_CSA_SM_UNWRAP	0xB9000605	3103786501	secure messaging unwrap error

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
E_CSA_SM_INTERNAL	0xB9000606	3103786502	internal error in SM lib
E_CSA_LX	0xB901	47361	CryptoServer API LINUX
E_CSA_LX_PATH	0xB9010001	3103850497	path name too long
E_CSA_LX_PORT	0xB9010002	3103850498	bad port number
E_CSA_LX_ADDR	0xB9010003	3103850499	bad IP address
E_CSA_LX_HOSTNAME	0xB9010004	3103850500	bad host name
E_CSA_LX_TERM	0xB9010005	3103850501	connection terminated by remote host
E_CSA_LX_MEM	0xB9010006	3103850502	can't alloc memory
E_CSA_LX_TIMEOUT	0xB9010007	3103850503	timeout occurred
E_CSA_LX_INVAL	0xB9010008	3103850504	invalid argument
E_CSA_LX_ADDRLEN	0xB9010009	3103850505	no space for sockaddr (internal error)
E_CSA_LX_BLKSIZE	0xB901000A	3103850506	bad block size received
E_CSA_LX_NOT_RDY	0xB901000B	3103850507	no ready message from CMDS

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
E_CSA_LX_CRIT_TEMP	0xB901000C	3103850508	cs2 exceeds critical temperature
E_CSA_LX_PROC	0xB901000D	3103850509	error on /proc file
E_CSA_LX_DEV	0xB901000E	3103850510	can't stat device file
E_CSA_LX_BUF_SIZE	0xB901000F	3103850511	buffer size too small
E_CSA_LX_OPEN	0xB9011	757777	can't open device
E_CSA_LX_SOCKET	0xB9012	757778	can't create socket
E_CSA_LX_CONNECT	0xB9013	757779	can't get connection
E_CSA_LX_POLL	0xB9014	757780	error while polling
E_CSA_LX_READ	0xB9015	757781	read error
E_CSA_LX_READ_701	0xB9015701	31038727 69	timeout
E_CSA_LX_READ_706	0xB9015706	31038727 74	operation interrupted by reset
E_CSA_LX_READ_707	0xB9015707	31038727 75	high temperature
E_CSA_LX_READ_70A	0xB901570A	31038727 78	CryptoServer halted
E_CSA_LX_READ_70B	0xB901570B	31038727 79	panic message from CryptoServer
E_CSA_LX_WRITE	0xB9016	757782	write error

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
E_CSA_LX_WRITE_701	0xB9016701	3103876865	timeout
E_CSA_LX_WRITE_703	0xB9016703	3103876867	request rejected by CS2
E_CSA_LX_WRITE_706	0xB9016706	3103876870	operation interrupted by reset
E_CSA_LX_WRITE_707	0xB9016707	3103876871	high temperature
E_CSA_LX_WRITE_70A	0xB901670A	3103876874	CryptoServer halted
E_CSA_LX_WRITE_70B	0xB901670B	3103876875	panic message from CryptoServer
E_CSA_LX_IOCTL	0xB9017	757783	ioctl error
E_CSA_LX_IOCTL_701	0xB9017701	3103880961	timeout
E_CSA_LX_IOCTL_706	0xB9017706	3103880966	operation interrupted by reset
E_CSA_LX_IOCTL_707	0xB9017707	3103880967	high temperature
E_CSA_LX_IOCTL_70A	0xB901770A	3103880970	CryptoServer halted
E_CSA_LX_IOCTL_70B	0xB901770B	3103880971	panic message from CryptoServer
E_CSA_LX_IOCTL_73	0xB901773	193992563	reset of CryptoServer failed

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
E_CSA_LX_LOCK	0xB9018	757784	ioctl error (locking)
E_CSA_LX_LOCK_706	0xB9018706	3103885062	operation interrupted by reset
E_CSA_LX_RECV	0xB9019	757785	tcp receive error
E_CSA_LX_SEND	0xB901A	757786	tcp send error
E_CSA_WIN	0xB902	47362	CryptoServer API Windows
E_CSA_WIN_PATH	0xB9020001	3103916033	path name too long
E_CSA_WIN_PORT	0xB9020002	3103916034	bad port number
E_CSA_WIN_ADDR	0xB9020003	3103916035	bad IP address
E_CSA_WIN_HOSTNAME	0xB9020004	3103916036	bad host name
E_CSA_WIN_TERM	0xB9020005	3103916037	connection terminated by remote host
E_CSA_WIN_MEM	0xB9020006	3103916038	can't alloc memory
E_CSA_WIN_TIMEOUT	0xB9020007	3103916039	timeout occurred
E_CSA_WIN_INVAL	0xB9020008	3103916040	invalid argument
E_CSA_WIN_ADDRLEN	0xB9020009	3103916041	no space for sockaddr (internal error)

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
E_CSA_WIN_BLKSIZE	0xB902000A	3103916042	bad block size received
E_CSA_WIN_CMDS_NOT_RDY	0xB902000B	3103916043	no ready message from CMDS
E_CSA_WIN_CRIT_TEMP	0xB902000C	3103916044	cs2 exceeds critical temperature
E_CSA_WIN_INVALID_PARAM	0xB9020010	3103916048	invalid parameter
E_CSA_WIN_INVALID_HANDLE	0xB9020011	3103916049	invalid handle value
E_CSA_WIN_CREATE_MUTEX	0xB9020013	3103916051	error creating mutex
E_CSA_WIN_LOCK	0xB9020014	3103916052	unable to set lock
E_CSA_WIN_LOCK_TIMEOUT	0xB9020015	3103916053	timeout while waiting for mutex
E_CSA_WIN_LOCK_HANDLE	0xB9020016	3103916054	no valid mutex object
E_CSA_WIN_OPEN	0xB90201	12124673	tcp: can't open device
E_CSA_WIN_SOCKET	0xB90202	12124674	tcp: can't create socket
E_CSA_WIN_CONNECT	0xB90203	12124675	tcp: can't get connection
E_CSA_WIN_POLL	0xB90204	12124676	tcp: error while polling
E_CSA_WIN_CONNECT_FAIL	0xB90204F0	3103917296	can't get connection

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
E_CSA_WIN_READ	0xB90205	12124677	tcp: read error
E_CSA_WIN_WRITE	0xB90206	12124678	tcp: write error
E_CSA_WIN_INIT	0xB90207	12124679	tcp: init error
E_CSA_WIN_IOCTL	0xB90208	12124680	tcp: ioctl error
E_CSA_WIN_DCI_OPEN	0xB9021	757793	dci: can't open device
E_CSA_WIN_DCI_READ	0xB9022	757794	read error
E_CSA_WIN_DCI_READ_RLEN	0xB9022001	3103924225	read returned wrong length
E_CSA_WIN_DCI_READ_TMOUT	0xB90220B5	3103924405	read timeout
E_CSA_WIN_DCI_READ_706	0xB9022706	3103926022	operation interrupted by reset
E_CSA_WIN_DCI_IOCTL_707	0xB9024707	3103934215	high temperature
E_CSA_WIN_DCI_IOCTL_73	0xB902473	193995891	reset failed
E_CSA_WIN_MTX	0xB9025	757797	mutex section
E_CSA_WIN_TCP_STARTUP	0xB9028	757800	tcp: startup error
E_CSA_WIN_TCP_ADDR	0xB9029	757801	tcp: address error
E_CSA_WIN_TCP_SOCKET	0xB902A	757802	tcp: can't create socket

Definition in <e2eefm_defs.h> and <errlist.h>	Error Code (Hexadecimal)	Error Code (Decimal)	Description
E_CSA_WIN_TCP_CONNECT	0xB902B	757803	tcp: can't get connection
E_CSA_WIN_TCP_CONNECT_TIMEOUT	0xB902B03C	3103961148	connection attempt timed out
E_CSA_WIN_TCP_CONNECT_REFUSED	0xB902B03D	3103961149	connection attempt refused
E_CSA_WIN_TCP_SELECT	0xB902C	757804	tcp: error on select
E_CSA_WIN_TCP_RECV	0xB902D	757805	tcp: receive error
E_CSA_WIN_TCP_SEND	0xB902E	757806	tcp: send error
E_CSA_WIN_TCP_IOCTL	0xB902F	757807	tcp: ioctl error